

من الفكرة إلى المتجر دليلك لتطوير تطبيقات الهاتف بمساعدة الذكاء الاصطناعي

دليل تعليمي متدرج لطلاب يريدون أن يفهموا كيف تبني تطبيقات الهاتف
فعلًا، ثم يبنوا تطبيقًا حقيقيًا بـ Flutter ويجربوه وينشروه على Android
و iOS، دون امتلاك جهاز Mac، مع استخدام أدوات الذكاء الاصطناعي بوصفها
مساعدًا لا بديلًا عن الفهم.

قبل أن تبدأ: عن هذا الدليل

هذا الدليل مكتوب لك إن كنت تعرف شيئاً من البرمجة، ربما كتبت برامج صغيرة بلغة Python أو Java أو JavaScript في مقرر جامعي، لكنك لم تبني تطبيق هاتف من قبل، أو حاولت وتنتهت بين عشرات الأدوات والمصطلحات. لا يفترض الدليل أنك تملك جهاز Mac، ولا أنك تملك ميزانية، ولا أنك تعرف ما معنى «توقيع التطبيق» أو «إدارة الحالة». يفترض فقط أنك مستعد للقراءة بتمعن وتجربة ما تقرأ على جهازك.

لماذا هذا الدليل؟

- طالب لديه فكرة تطبيق ويريد أن يعرف المسار الكامل من الفكرة إلى المتجر قبل أن يستثمر شهوياً في الاتجاه الخاطئ.
- طالب يملك Windows أو Linux فقط ويسأل: هل أستطيع أصلاً بناء تطبيق iOS؟ وما الذي سيبقى مستحيلًا دون Mac؟ (الجواب الصادق في الفصل الخامس).
- طالب يستخدم ChatGPT أو Claude أو Copilot لكتابة الكود، ويشعر أن التطبيق «يعمل بطريقة ما» دون أن يفهم لماذا، ويريد أن يستعيد السيطرة على مشروعه.
- مشرف أو زميل يريد مرجعاً عربياً واحداً يشرح المفاهيم بترتيب تعليمي، لا قائمة روابط متفرقة.

ماذا ستستطيع فعله بنهاية الدليل؟

- تشرح الفرق بين التطبيق الأصلي والمتعدد المنصات وتطبيق الويب وPWA، وتختار المناسب لفكرة معينة بحجج واضحة.
- تفهم علاقة نظام التشغيل بالجهاز بالمتجر بحساب المطور، وتعرف ما الملفات التي تنتجها عملية البناء وما معنى توقيعها.
- تجهز بيئة عمل Flutter كاملة على Windows أو Linux، وتشغل تطبيقك على محاكي Android وعلى هاتف حقيقي.
- تكتب متطلبات تطبيق صغير، وترسم شاشاته، وتقسمه إلى مراحل، وتبني أول نسخة قابلة للتجربة (MVP).

- تبني تطبيق «عداد أنكار» كاملاً خطوة خطوة: واجهة، حالة، تخزين محلي، قائمة، معالجة أخطاء، اختبارات، واجهة عربية، ثم إصدار تجريبي.
- تستخدم أدوات الذكاء الاصطناعي بطريقة منهجية: برومبتات محددة، مراجعة كل تغيير، اختبار كل نتيجة، وحماية أسرارك وبياناتك.
- تعرف ما الذي تحتاج إلى تعلمه لاحقاً، وبأي ترتيب، ولماذا.

كيف تقرأ هذا الدليل؟

الفصول مرتبة ترتيباً تعليمياً: الأول إلى الخامس تبني الصورة الكبيرة وتجهز البيئة، والسادس والسابع يعلمانك كيف تفكر وما المفاهيم التي ستحتاجها، والثامن يعلمك العمل مع الذكاء الاصطناعي، والتاسع مشروع كامل تنفذه بيدك، والعاشر والحادي عشر أخطاء شائعة وخطة تعلم. إن كنت متعجلاً فاقرأ الفصلين الأول والخامس ثم انتقل إلى المشروع في الفصل التاسع، وعد إلى بقية الفصول حين تحتاجها. لكن لا تتخط الفصل السادس: أكثر المشاريع الطلابية تفشل قبل كتابة أول سطر كود، لأن صاحبها لم يحدد ما الذي يبنيه.

ستجد في الدليل خمسة أنواع من الصناديق، وكل منها له معنى مختلف. الفصل بين «رأي الشخصي» و«المتطلب الرسمي» مقصود؛ الأول يمكنك مخالفته إن كان لديك سبب، والثاني لا.

نصيحة شخصية

رأي المؤلف أو خبرة عملية. مفيدة عادةً، لكنها ليست قاعدة ملزمة، ويمكن أن يختلف معها مطورون آخرون.

متطلب رسمي

شرط تقني أو سياسة معلنة من Google أو Apple أو مطوري الأداة، مع رابط المصدر وتاريخ التحقق. هذه الأمور تتغير، فراجع الرابط قبل النشر.

انتبه

خطأ شائع أو سوء فهم يقع فيه المبتدئون كثيرًا.

مع الذكاء الاصطناعي

كيف تستخدم أداة الذكاء الاصطناعي في هذه النقطة تحديداً، وما الذي يجب أن يبقى بيدك.

مصطلح

تعريف مختصر لمصطلح يظهر لأول مرة، مع مثال.

ملاحظة عن دقة المعلومات

المفاهيم في هذا الدليل ثابتة نسبياً: ما معنى الحالة، ولماذا نفصل الواجهة عن البيانات، وكيف نفكر في المتطلبات. أما التفاصيل المتغيرة، مثل رسوم حسابات المطورين، وأرقام الإصدارات، ومتطلبات المتاجر، وأسعار خدمات البناء السحابية، فقد تحققت منها من الوثائق الرسمية بتاريخ **2026-09-27** وذكرت الروابط في موضعها وفي الملحق الأول. حين لم أستطع التحقق من معلومة، قلت ذلك صراحةً بدل التخمين. قبل أن تنشر تطبيقك، افتح الروابط وتأكد بنفسك؛ هذه عادة يجب أن تكتسبها من اليوم الأول.

الفهرس

قبل أن تبدأ: عن هذا الدليل

2

الفصل 1: فهم عالم التطبيقات

9

الفصل 2: Android و iOS والمتاجر

14

الفصل 3: رحلة التطبيق: من الفكرة إلى الصيانة

الفصل 4: اختيار التقنية واللغة

26

الفصل 5: الأدوات وبيئة العمل: مسار الطالب الذي لا يملك Mac

الفصل 6: التفكير في التطبيق قبل كتابة الكود

الفصل 7: المفاهيم التي يجب أن تتعلمها، مرتبة حسب مراحل البناء

الفصل 8: استخدام الذكاء الاصطناعي باحتراف

الفصل 9: المشروع التطبيقي: عدّاد أذكار خطوة خطوة

الفصل 10: أخطاء شائعة ونصائح عملية

الفصل 11: خطة تعلم من عشرة أسابيع وقوائم التحقق

الملحق الأول: المصادر الرسمية وتاريخ التحقق

الملحق الثاني: قاموس مصطلحات مختصر

ما الذي يحدث حين تضغط على أيقونة تطبيق؟

التطبيقات الأصلية (Native Apps)

التطبيقات متعددة المنصات (Cross-platform Apps)

تطبيقات الويب وتطبيقات الويب التقدمية (PWA)

الواجهة الأمامية والخلفية (Frontend / Backend)

تطبيق محلي بالكامل أم تطبيق يحتاج إلى خادم؟

9

13	كيف تختار؟ أسئلة تطرحها على فكرتك
13	خلاصة الفصل
16	الجهاز، ونظام التشغيل، والمتجر: ثلاث طبقات
17	أرقام الإصدارات: أي Android وأي iOS ستدعم؟
18	حساب المطور: بطاقة دخولك إلى المتجر
19	ملفات البناء: ما الذي ترفعه فعلياً؟
19	توقيع التطبيق: لماذا، وبأي مفتاح؟
20	أرقام الإصدارات: اسم للناس ورقم للنظام
21	المسار العام للنشر والاختبار والتحديث
21	اختبارات ما قبل النشر التي يجريها المتجر
21	ما الثابت وما المتغير؟
21	خلاصة الفصل
22	المراحل السبع في صورة واحدة
23	1. الفكرة: مشكلة واحدة لشخص محدد
23	2. التصميم: قبل الكود لا بعده
23	3. الكود: أصغر شيء يعمل، ثم توسع
23	4. الاختبار: ما لم يختبر فهو مكسور
24	5. البناء: من كود إلى ملف موقع
24	6. النشر: عمل إداري بقدر ما هو تقني
24	7. الصيانة: هنا يعيش التطبيق معظم عمره
24	أين يقف الذكاء الاصطناعي في كل مرحلة؟
26	المسارات الخمسة
28	لماذا يرشح هذا الدليل Flutter للبداية؟
28	حدود Flutter التي يجب أن تعرفها بصدق
29	متى يكون غير Flutter أفضل؟
29	أسئلة تطرحها على نفسك قبل أن تقرر
30	خلاصة الفصل
31	أربعة أنشطة مختلفة، لا نشاط واحد

32	قائمة الأدوات ودور كل منها
33	الخطوة 1: تثبيت Flutter SDK
34	الخطوة 2: Android Studio و Android SDK والمحاكي
35	الخطوة 3: جهاز Android حقيقي
39	الخطوة 4: المحرر (VS Code أو Android Studio)
40	الخطوة 5: Git و GitHub
41	الخطوة 6: بناء iOS في السحابة بـ Codemagic
42	بدائل Codemagic
42	كيف تختبر iOS فعلاً دون Mac؟ خطة واقعية
43	قائمة تحقق: بيئتك جاهزة حين...
43	خلاصة الفصل
45	ابدأ بمشكلة صغيرة، لا بتطبيق كبير
45	حدد المستخدم المستهدف بدقة مزعجة
46	الوظائف الأساسية وما يمكن تأجيله
48	نموذج المتطلبات: صفحة واحدة تكفي
55	المثال الثاني: حافظلة الفواتير والضمانات
59	المثال الثالث: متابع الأحداث والتواريخ
63	ارسم الشاشات على ورق
65	قسم العمل إلى مراحل صغيرة قابلة للعرض
	خلاصة الفصل
	المحطة الأولى: اللغة التي تفكر بها
	المحطة الثانية: الواجهة التي يلمسها المستخدم
	المحطة الثالثة: البنية والبيانات
	المحطة الرابعة: حواف المنصة
	المحطة الخامسة: الجودة والعمر الطويل
	خريطة المفاهيم في مشروعك الأول
	خلاصة الفصل
	المبدأ الحاكم: أنت المهندس، والأداة مساعد سريع جداً

ما الذي تجيده الأداة في كل مرحلة، وكيف تطلبه؟

68

تشريح البرومبت الجيد

69

متى تقسم المهمة؟

69

كيف تراجع تغييرًا قبل قبوله؟

70

ما لا تشاركه مع الأداة أبدًا

مساعد المحرر مقابل مساعد المحادثة مقابل الوكيل

70

نماذج برومبتات قابلة للنسخ

74

خلاصة الفصل

93

ما بعد الإصدار التجريبي: مراحل لاحقة

94

1. البدء بعشرات الميزات

94

2. نسخ كود الذكاء الاصطناعي دون فهمه

95

3. إهمال Git

95

4. الاختبار على جهاز واحد فقط

95

5. الخلط بين خطأ الواجهة وخطأ البيانات

6. تأجيل الأنونات والخصوصية إلى نهاية المشروع

96

7. أخطاء أخرى تستحق التحذير

نصائح عملية للعمل ضمن مجموعة طلابية

99

ماذا تتعلم بعد ذلك؟

قائمة التحقق الأولى: قبل أن تقول «تطبيقي جاهز للتجربة»

100

قائمة التحقق الثانية: قبل النشر العام

101

كلمة أخيرة

CHAPTER 01

فهم عالم التطبيقات

قبل أن تكتب سطرًا واحدًا من الكود، تحتاج إلى خريطة. كلمة «تطبيق» تطلق اليوم على أشياء مختلفة جدًا: برنامج يعمل بالكامل داخل هاتفك، وموقع ويب يتظاهر بأنه تطبيق، وواجهة صغيرة على الهاتف يقف خلفها عشرون خادماً. في هذا الفصل نفكك هذه الكلمة إلى أنواعها، ونتعلم متى يناسب كل نوع، ونصحح سوء فهم شائعاً: أن كل تطبيق في المتجر يحتاج إلى خادم وقاعدة بيانات سحابية.

ما الذي يحدث حين تضغط على أيقونة تطبيق؟

حين تضغط على أيقونة في هاتفك، يطلب نظام التشغيل (Android أو iOS) من المعالج تحميل ملف تنفيذي مخزن على الجهاز، ويعطيه نافذة ليرسم فيها، وذاكرة ليعمل فيها، وصلاحيات محددة (الكاميرا، الموقع، الإنترنت) لا يتجاوزها. هذا الملف التنفيذي هو «التطبيق». كل ما سنتعلمه في هذا الدليل يدور حول سؤالين: كيف نُنتج هذا الملف؟ وكيف نوصله إلى هواتف الناس؟

الفرق بين أنواع التطبيقات يكمن في الطريقة التي كُتب بها هذا الملف واللغة التي يتحدث بها مع النظام. لنبدأ بالتصنيف الأشهر.

التطبيقات الأصلية (Native Apps)

التطبيق الأصلي يكتب بالأدوات واللغة التي صممها صانع نظام التشغيل نفسه: Kotlin (أو Java القديمة) مع أدوات Android من Google، وSwift (أو Objective-C القديمة) مع Xcode من Apple. الكود يتحدث مباشرة مع واجهات النظام دون وسيط، فيحصل على أفضل أداء، وأسرع وصول إلى الميزات الجديدة يوم صدورها، ومظهر مطابق تماماً لما يتوقعه مستخدم كل منصة.

الثنى: قاعدتا كود منفصلتان. تطبيق «حافضة الفواتير» يحتاج إلى فريق يكتبه مرة بـ Kotlin ومرة بـ Swift، ويصلح كل خطأ مرتين، ويختبر كل ميزة مرتين. الشركات الكبيرة تفعل هذا لأنها تستطيع. الطالب أو الفريق الصغير عادةً لا يستطيع.

مصطلح: واجهة برمجة النظام (Platform API)

مجموعة الدوال التي يوفرها نظام التشغيل للتطبيقات: «افتح الكاميرا»، «أعطني الموقع»، «أرسل إشعارًا». التطبيق الأصلي يستدعيها مباشرة، والتطبيق متعدد المنصات يستدعيها عبر طبقة وسيطة تسمى «جسر» أو «قناة» (platform channel).

التطبيقات متعددة المنصات (Cross-platform Apps)

الفكرة: تكتب الكود مرة واحدة بلغة واحدة، وتنتج منه تطبيقًا لـ Android وآخر لـ iOS. هناك عائلتان رئيسيتان:

- إطار يرسم واجهته بنفسه، مثل **Flutter**: يأتي بمحرك رسم خاص به، فتبدو الشاشة متطابقة على كل الأجهزة، لأنه لا يستخدم أزرار النظام بل يرسم أزراره هو بكسلًا بكسلًا.

- إطار يستخدم عناصر النظام الأصلية عبر جسر، مثل **React Native**: تكتب بـ JavaScript أو TypeScript، ويترجم الإطار أوامرك إلى عناصر واجهة أصلية حقيقية.

في كلتا الحالتين، حين تحتاج إلى ميزة لا يغطيها الإطار (مستشعر نادر، واجهة نظام جديدة جدًا)، ستكتب قطعة صغيرة من الكود الأصلي وتربطها بمشروعك، أو تستخدم حزمة جاهزة كتبها غيرك. هذا ليس عيبًا قاتلًا، لكنه شيء يجب أن تعرفه: «متعدد المنصات» لا يعني «لن ألس Kotlin أو Swift أبدًا».

تطبيقات الويب وتطبيقات الويب التقدمية (PWA)

تطبيق الويب هو موقع يعمل داخل المتصفح. لا يثبت من متجر، ولا يحتاج إلى حساب مطور، ويصل إليه أي جهاز فيه متصفح. تطبيق الويب التقدمي (**Progressive Web App**) هو موقع ويب أضيفت إليه ملفات وإعدادات تجعله يشبه التطبيق: أيقونة على الشاشة الرئيسية، فتح بلا شريط عنوان، وعمل جزئي دون اتصال عبر ما يسمى **Service Worker** (سكربت يعمل في الخلفية ويخزن نسخة من الملفات).

حدود PWA حقيقية ويجب أن تعرفها قبل أن تختارها:

- على iOS، إشعارات الويب (**Web Push**) لا تعمل إلا للموقع الذي أضافه المستخدم إلى الشاشة الرئيسية، وهذا مدعوم منذ iOS 16.4.

- Safari يحذف التخزين المحلي للمواقع التي لم يتفاعل معها المستخدم لمدة 7 أيام، ما لم يكن الموقع مضافاً إلى الشاشة الرئيسية.
- لا وصول إلى Bluetooth أو NFC أو المزامنة في الخلفية على Safari، ولا توجد «ودجات» شاشة رئيسية قياسية.
- لا تظهر في المتاجر إلا إذا غلفتها في حاوية (مثل Trusted Web Activity على Android)، وApple ترفض التطبيقات التي هي «موقع معاد تغليفه» بموجب البند 4.2 من إرشادات المراجعة.

متطلب رسمي ومصادر

دعم Web Push في تطبيقات الشاشة الرئيسية على iOS 16.4: مدونة WebKit. حد 7 أيام للتخزين وإعفاء تطبيقات الشاشة الرئيسية: صفحة سياسة WebKit لمنع التتبع. معايير التثبيت في Chrome: web.dev. بند «الحد الأدنى من الوظائف» في إرشادات App Store: البند 4.2. الروابط في الملحق الأول. تاريخ التحقق: 2026-09-27.

الواجهة الأمامية والخلفية (Frontend / Backend)

هذا تصنيف من زاوية مختلفة تماماً: لا «كيف كتب التطبيق» بل «أين تعمل أجزاؤه».

- الواجهة الأمامية هي كل ما يعمل على جهاز المستخدم: الشاشات، الأزرار، الرسوم، والمنطق الذي يرد فوراً على لمساته.

- الواجهة الخلفية هي كل ما يعمل على خادم بعيد: قاعدة بيانات مركزية، حسابات المستخدمين، منطق لا يجوز أن يعمل على الجهاز (مثل التحقق من الدفع)، وإرسال الإشعارات من الخارج.

تطبيق الهاتف كله «واجهة أمامية» من هذه الزاوية، حتى لو كان فيه منطق معقد. والسؤال المهم: هل يحتاج تطبيقك إلى واجهة خلفية أصلاً؟

تطبيق محلي بالكامل أم تطبيق يحتاج إلى خادم؟

هنا سوء الفهم الأكبر بين المبتدئين. كثيرون يظنون أن أي تطبيق «حقيقي» ينشر في المتجر يحتاج إلى Firebase أو خادم وقاعدة بيانات سحابية وحسابات. هذا خطأ. المتجر لا يهتم بمكان بياناتك؛ يهتم بأن ملف التطبيق سليم وموقع ويحترم السياسات. تطبيق يخزن كل شيء على الهاتف يمكن نشره ويحقق ملايين التنزيلات.

متى يحتاج التطبيق إلى خادم؟ أمثلة

التطبيق	هل يحتاج إلى إنترنت؟	لماذا؟
عداد أنكار	لا	العدد والقائمة تحفظ على الجهاز. لا مستخدمين آخرين، لا بيانات مشتركة.
حافضة فواتير وضمانات (نسخة أولى)	لا	الصور والتواريخ محلية. النسخ الاحتياطي يمكن أن يكون ملفًا يصدره المستخدم بنفسه.
حافضة فواتير مع مزامنة بين هاتفين	نعم	المزامنة تحتاج إلى مكان مشترك يصل إليه الجهازان، وإلى حساب يربط بينهما.
متابعة أحداث وتواريخ شخصية مع تنبيهات	لا	التنبيهات المحلية ي جدولها النظام على الجهاز نفسه دون خادم.
تقويم مشترك لمجموعة طلابية	نعم	عدة مستخدمين يعدلون البيانات نفسها؛ لا بد من مصدر حقيقة مركزي.
مواقيت الصلاة	نعم جزئيًا	يمكن الحساب محليًا من إحداثيات الموقع، لكن جلب بيانات مدينة جديدة أو تحديث الجداول قد يحتاج إلى الشبكة.
محادثة أو تعليقات	نعم	الرسائل تنقل بين أجهزة مختلفة في الوقت الفعلي.

القاعدة العملية: يحتاج التطبيق إلى خادم حين تحتاج البيانات إلى الانتقال بين أجهزة أو أشخاص، أو حين يجب أن يبقى جزء من المنطق بعيدًا عن يد المستخدم. غير ذلك، ابدأ محليًا. الخادم يضيف تكلفة شهرية، وصفحة خصوصية، وإدارة حسابات، وأخطاء شبكة، ومسؤولية قانونية عن بيانات الآخرين. أضفه حين تحتاج إليه فعلاً، لا لأنه «يبدو احترافيًا».

نصيحة شخصية

مشروعك الأول يجب أن يكون محليًا بالكامل. ستتعلم الواجهة والحالة والتخزين والاختبار والنشر، وهي وحدها كافية لتشغل شهوًا. المزامنة والحسابات تأتي في مشروع ثان، حين تصبح أساسياتك ثابتة.

كيف تختار؟ أسئلة تطرحها على فكرتك

لا يوجد خيار «أفضل» مطلقًا، بل خيار مناسب لفكرة ولفريق وليزانية. أجب عن هذه الأسئلة قبل أن تقر:

1. من المستخدم وأين سيستخدمه؟ إن كان الاستخدام سريعًا عشرات المرات يوميًا (عداد، تذكير)، فالتطبيق المثبت أنسب من موقع يجب فتحه في متصفح.
2. هل يحتاج إلى ميزات الجهاز؟ الكاميرا للفواتير، الإشعارات المحلية للتواريخ، الاهتزاز للعداد: كلها تعمل أفضل في تطبيق مثبت. لو كان المحتوى نصيًا يقرأ فقط، فالويب قد يكفي.
3. هل تحتاج إلى iOS وAndroid معًا من اليوم الأول؟ إن كان نعم وأنت وحدك، فمتعدد المنصات هو الخيار الواقعي.
4. ما حجم الفريق وخبرته؟ فريق يعرف JavaScript جيدًا قد يفضل React Native؛ طالب يبدأ من الصفر قد يجد Flutter أوضح لأن كل شيء فيه في مكان واحد.
5. هل يجب أن يكون في المتجر؟ أداة داخلية لمجموعة صغيرة قد تكون PWA أسرع وأرخص. تطبيق يريد أن يجده الجمهور بالبحث في المتجر يحتاج إلى تطبيق مثبت.

مع الذكاء الاصطناعي

اعرض فكرتك على أداة الذكاء الاصطناعي واطلب منها أن تطرح عليك الأسئلة الخمسة أعلاه واحدًا واحدًا، ثم تلخص إجاباتك في توصية مع مبرراتها وبدائلها. لا تقبل توصية بلا بديل؛ اطلب: «ما الحالة التي يكون فيها الخيار الآخر أفضل؟». ستفهم من الجواب أكثر مما تفهم من التوصية نفسها.

خلاصة الفصل

- التطبيق ملف تنفيذي يشغله نظام التشغيل داخل حدود وصلاحيات محددة.
- أصلي = لغة النظام وأدواته، قاعدتنا كود. متعدد المنصات = لغة واحدة، ناتجان، مع جسر للميزات الخاصة. ويب/PWA = يعمل في المتصفح بحدود معروفة خاصة على iOS.
- الواجهة الأمامية على الجهاز، الخلفية على خادم. تطبيق الهاتف كله واجهة أمامية.
- النشر في المتجر لا يستلزم خادمًا. ابدأ محليًا وأضف الخادم حين تحتاج البيانات إلى الانتقال بين أجهزة أو أشخاص.

CHAPTER 02

Android و iOS والمتاجر

حين يقول شخص «أريد تطبيقًا على iPhone»، فهو يتحدث في الواقع عن أربعة أشياء مختلفة: جهاز، ونظام تشغيل، ومتجر، وحساب مطور. الخلط بينها هو سبب معظم الأسئلة المحيرة مثل «لماذا أحتاج إلى Mac؟» أو «لماذا لا أستطيع إرسال ملف التطبيق لصديقي على iPhone كما أفعل على Android؟». هذا الفصل يفكك الأربعة.

الجهاز، ونظام التشغيل، والمتجر: ثلاث طبقات

الطبقة	في عالم Google	في عالم Apple	دورها
الجهاز	هواتف Samsung وXiaomi وPixel وغيرها من مئات الشركات	iPhone وiPad من Apple فقط	العتاد: شاشة ومعالج وكاميرا ومستشعرات.
نظام التشغيل	Android (مفتوح المصدر، تعدله كل شركة)	iOS (مغلق، من Apple فقط)	يدير العتاد ويشغل التطبيقات ويفرض الصلاحيات.
المتجر	Google Play (وتوجد متاجر أخرى مثل Galaxy Store، ويمكن التثبيت المباشر)	App Store (القناة الوحيدة عمليًا لعامة المستخدمين خارج الاتحاد الأوروبي)	يوزع التطبيقات ويراجعها ويتولى الدفع والتحديث.
بوابة المطور	Google Play Console	App Store Connect	حيث ترفع تطبيقك وتكتب وصفه وتدير الاختبار والإصدارات.

لاحظ الفرق الجوهرى: Android يسمح بتثبيت ملف تطبيق مباشرة من خارج المتجر (يسمى **sideloading**)، لذا تستطيع بناء ملف **.apk** وإرساله إلى صديق فيثبته بعد تأكيد تحذير أمني. أما iOS، فلا يشغل إلا تطبيقات موقعة بشهادة تعترف بها Apple، ويصل التطبيق إلى المستخدم العادي إما عبر App Store أو عبر TestFlight (قناة الاختبار الرسمية). هذا القيد هو السبب الحقيقي وراء كثير من «تعقيدات» iOS التي ستقابلها.

أرقام الإصدارات: أي Android وأي iOS ستدعم؟

كل نظام يصدر إصدارًا رئيسيًا سنويًا تقريبًا. Android يرقم إصداراته بأرقام (Android 16، Android 17) ولكل إصدار «مستوى واجهة برمجة» (API level) هو الرقم الذي يهتم المطور فعلاً (Android 16 = API 36، وAndroid 17 = API 37). iOS = (iOS 26، iOS 27) ببساطة.

ستقابل مصطلحين مهمين:

- الحد الأدنى المدعوم (minSdk / Deployment Target): أقدم إصدار يستطيع تثبيت تطبيقك. كلما خفضته وصلت إلى أجهزة أكثر لكن حرمت من ميزات أحدث واضطرت إلى اختبار أوسع.
- الإصدار المستهدف (المستخدم في البناء targetSdk / SDK): الإصدار الذي تعلن أنك اختبرت تطبيقك عليه، والمتاجر تفرض حدًا أدنى له يرتفع كل عام.

متطلب رسمي (تحقق 2026-09-27)

- Google Play: ابتداءً من 31 أغسطس 2026، يجب أن تستهدف التطبيقات الجديدة والتحديثات (Android 16 (API 36) أو أحدث، مع إمكانية طلب تمديد حتى 1 نوفمبر 2026. التطبيقات القائمة يجب أن تستهدف API 35 على الأقل لتبقى متاحة للمستخدمين الجدد على الأجهزة الأحدث.
<https://support.google.com/googleplay/android-developer/answer/11926878>
- Google Play: منذ 1 نوفمبر 2025، التطبيقات الجديدة والتحديثات التي تستهدف Android 15 فأعلى يجب أن تدعم أحجام صفحات الذاكرة 16 KB على أجهزة Flutter-64 bit. التحديث يبني هذا الدعم افتراضياً، لكن الحزم التي تتضمن كوداً أصلياً قديماً قد تحتاج إلى تحديث.
<https://developer.android.com/guide/practices/page-sizes>
- App Store: منذ 28 أبريل 2026، يجب أن تبني التطبيقات المرفوعة إلى App Store Connect بـ Xcode 26 و iOS SDK 26 أو أحدث. وأعلنت Apple أن التطبيقات المرفوعة ابتداءً من أبريل 2027 يجب أن تبني بـ iOS SDK 27.
<https://developer.apple.com/news/upcoming-requirements/>
- Flutter 3.47 يدعم Android من (Android 7.0) API 24 حتى API 37، و iOS من 15 حتى 27.
<https://docs.flutter.dev/reference/supported-platforms>

الدرس العملي: المتطلبات تتحرك سنوياً. لا تحفظ الأرقام؛ احفظ الرابط، وافتحه قبل كل إصدار. حين تستخدم Flutter وأدوات بناء محدثة، يضبط معظم هذه القيم تلقائياً، لكن التطبيق الذي يهمل عامًا كاملاً سيكتشف يوم التحديث أن المتجر يرفضه.

حساب المطور: بطاقة دخولك إلى المتجر

لا تستطيع رفع تطبيق إلى أي من المتجرين دون حساب مطور مدفوع. الحساب مرتبط بهويتك القانونية، والمتجر يعرض بعض بياناتك للجمهور.

حسابا المطور (تحقق 2026-09-27)

Google Play Console		Apple Developer Program
الرسوم	25 دولارًا أمريكيًا، تدفع مرة واحدة	99 دولارًا أمريكيًا سنوياً
التسجيل	عبر الويب من أي جهاز	عبر الويب أو تطبيق Apple Developer على iPhone/iPad. لا تشتري جهاز Mac للتسجيل نفسه
التحقق من الهوية	هوية حكومية وبطاقة ائتمانية باسمك، وتحقق من الهاتف والبريد. الحسابات الشخصية الجديدة يجب أن تثبت وصولها إلى جهاز Android عبر تطبيق Play Console	صورة هوية ضمن التسجيل. المؤسسات تحتاج إلى رقم D-U-N-S
ما يعرض للجمهور	الاسم القانوني والدولة والبريد. إن حققت دخلاً عرض عنوانك الكامل	اسم المطور أو المؤسسة
قيد خاص بالمبتدئين	الحسابات الشخصية المنشأة بعد 13 نوفمبر 2023 يجب أن تجري اختباراً مغلقاً بـ 12 مختبراً على الأقل مشتركين باستمرار 14 يوماً قبل أن يسمح لها بالنشر في الإنتاج	الحساب المجاني يسمح بتشغيل تطبيقك على جهازك عبر Xcode فقط (ينتهي التوقيع بعد 7 أيام)، دون TestFlight أو App Store
إعفاء من الرسوم	لا يوجد	للمؤسسات غير الربحية والتعليمية والحكومية، ليس للأفراد

المصادر

<https://support.google.com/googleplay/android-developer/answer/6112435>
<https://support.google.com/googleplay/android-developer/answer/14151465>
<https://support.google.com/googleplay/android-developer/answer/13628312>
<https://developer.apple.com/support/enrollment/>
<https://developer.apple.com/support/compare-memberships/>
<https://developer.apple.com/support/membership-fee-waiver/>

انتبه: شرط الـ 12 مختبرًا

هذا الشرط يفاجئ كثيرًا من الطلاب. إن أنشأت حساب Google Play شخصيًا جديدًا، فلن تستطيع النشر العام قبل أن يبقى 12 شخصًا على الأقل مشتركين في اختبارك المغلق 14 يومًا متواصلة، ثم تقدم طلبًا للوصول إلى الإنتاج. خطط لهذا مبكرًا: مجموعتكم الطلابية هي المكان المثالي لتبادل الاختبار.

ملفات البناء: ما الذي ترفعه فعليًا؟

حين تكتب أمر البناء، تنتج أدوات التطوير ملفًا واحدًا (أو أكثر) يحتوي على الكود المترجم والصور والخطوط والإعدادات. هذه الملفات هي ما يوقع ويرفع:

الملف	النص	ما هو	متى تستخدمه
.apk	Android	حزمة تطبيق كاملة قابلة للتثبيت مباشرة على الجهاز	الاختبار السريع، الإرسال المباشر لمختبر، المتاجر البديلة
.aab	Android	Android App Bundle: حزمة «خام» يولد منها Google Play ملفات APK مخصصة لكل جهاز (أصغر حجمًا)	الرفع إلى Google Play. مطلوب للتطبيقات الجديدة منذ أغسطس 2021
.ipa	iOS	حزمة تطبيق iOS موقعة	الرفع إلى App Store Connect / TestFlight
.xcarchive	iOS	أرشيف البناء الذي يستخرج منه الـ ipa، ويحتوي على رموز تصحيح الأخطاء	يُنشأ على Mac أو خدمة بناء سحابية

متطلب رسمي

Google Play يشترط رفع التطبيقات الجديدة بصيغة App Bundle.

<https://developer.android.com/guide/app-bundle>

توقيع التطبيق: لماذا، وبأي مفتاح؟

التوقيع الرقمي يجيب عن سؤال النظام: «هل هذا التحديث من صاحب التطبيق الأصلي نفسه؟». لو استطاع أي شخص إنتاج ملف يحمل اسم تطبيقك، لاستطاع استبدال تطبيقك على أجهزة المستخدمين بنسخة خبيثة. لذلك يوقع كل ملف بمفتاح خاص، ويرفض النظام تثبيت تحديث موقع بمفتاح مختلف.

على Android

هناك مفتاحان يجب أن تفرق بينهما:

- **مفتاح توقيع التطبيق (app signing key):** المفتاح الذي توقع به النسخ التي تصل إلى أجهزة المستخدمين. لا يتغير طوال عمر التطبيق. في التطبيقات الجديدة، Google تحتفظ به وتديره عبر **Play App Signing** (الزامي).
- **مفتاح الرفع (upload key):** مفتاح تنشئه أنت وتحتفظ به في ملف **keystore**، وتوقع به الملف قبل رفعه إلى **Play Console**. إن فقدته يمكنك طلب إعادة تعيينه من Google، لكن ذلك يستغرق وقتًا.

احتفظ بملف keystore وكلمة مروره في مكان آمن ومنفصل عن المشروع

لا ترفعه إلى GitHub أبدًا. ولا تعطه لأداة ذكاء اصطناعي. النسخة الاحتياطية الوحيدة في حاسوب واحد ليست نسخة احتياطية. المرجع:

<https://developer.android.com/studio/publish/app-signing>

على iOS

نظام Apple أكثر طبقات، وستقابل ثلاث كلمات:

- **الشهادة (Certificate):** تثبت هويتك كمطور. شهادة تطوير لتشغيل التطبيق على أجهزتك، وشهادة توزيع لرفعه إلى **App Store**.

• **معرف التطبيق (App ID / Bundle ID):** اسم فريد على نمط `sa.edu.kau.students.dhikr` يعرف تطبيقك لدى Apple.

• **ملف التزويد (Provisioning Profile):** يربط الشهادة بمعرف التطبيق (وبقائمة أجهزة في حالة التطوير) ويأذن بتشغيل هذا التطبيق تحديداً.

الخبر الجيد: خدمات البناء السحابية مثل Codemagic تستطيع إنشاء الشهادة وملف التزويد نيابةً عنك باستخدام مفتاح واجهة App Store Connect، دون Mac. سنفعل ذلك في الفصل الخامس. المرجع الرسمي للمفاهيم: <https://developer.apple.com/help/account/certificates/certificates-overview/>

أرقام الإصدارات: اسم للناس ورقم للنظام

كل تطبيق يحمل قيمتين: اسم إصدار يقرؤه المستخدم مثل `1.2.0`، ورقم بناء داخلي يقارن به النظام والمتجر لمعرفة أي نسخة أحدث. في Flutter تكتب الاثنين في سطر واحد داخل `pubspec.yaml`:

```
version: 1.2.0+7 # 1.2.0 هو رقم البناء و7 هو اسم الإصدار،
```

على Android يصبحان `versionName` و `versionCode`، وعلى iOS `CFBundleShortVersionString` و `CFBundleVersion`. القاعدة الوحيدة الصارمة: رقم البناء يجب أن يزيد في كل رفع، وإلا رفضه المتجر. اصطلاح **Semantic Versioning** الشائع لاسم الإصدار: الرقم الأول لتغييرات كبيرة، والثاني لميزات جديدة، والثالث لإصلاحات.

متطلب رسمي

Android: `versionCode` عدد صحيح موجب يجب أن يزيد مع كل إصدار، وأقصى قيمة يقبلها Google Play هي 2100000000.

<https://developer.android.com/studio/publish/versioning>

المسار العام للنشر والاختبار والتحديث

المسار متشابه في المتجرين، مع اختلاف الأسماء:

1. إنشاء سجل التطبيق في البوابة: الاسم، الفئة، اللغة الافتراضية.

2. البناء والتوقيع: إنتاج **.aab** أو **.ipa** موقع.
3. الرفع إلى مسار اختبار: على Google Play: اختبار داخلي (حتى 100 مختبر)، ثم مغلق، ثم مفتوح. على Apple: TestFlight (حتى 100 مختبر داخلي و10,000 خارجي، وتنتهي صلاحية النسخة بعد 90 يومًا).
4. استكمال صفحة المتجر: الوصف، لقطات الشاشة، الأيقونة، سياسة الخصوصية، تصنيف المحتوى، ونموذج أمان البيانات (Play) أو تفاصيل الخصوصية (Apple).
5. المراجعة: Apple تقول إن 90% من الطلبات تراجع في أقل من 24 ساعة. Google تقول إن بعض الحسابات قد تستغرق مراجعتها 7 أيام أو أكثر في حالات استثنائية.
6. الإصدار: فوري، أو مجدول، أو تدريجي لنسبة من المستخدمين.
7. التحديث: زيادة رقم البناء، بناء جديد، رفع، وربما مراجعة مختصرة. التحديثات المنشورة لا يمكن «سحبها» من الأجهزة التي ثبتتها؛ يمكنك فقط إيقاف الطرح أو نشر إصلاح.

المصادر

<https://support.google.com/googleplay/android-developer/answer/9845334>

<https://developer.apple.com/help/app-store-connect/test-a-beta-version/testflight-overview>

<https://developer.apple.com/distribute/app-review/>

<https://support.google.com/googleplay/android-developer/answer/9859751>

اختبارات ما قبل النشر التي يجريها المتجر

Google Play يشغل تلقائيًا **Pre-launch report** عند رفع أي حزمة: يثبت التطبيق على أجهزة مختبره، ويتجول فيه دقائق، ويبلغ عن الانهيارات ومشكلات التوافق والأداء وإمكانية الوصول. اقرأ هذا التقرير؛ كثيرًا ما يكشف انهيارًا على جهاز لم تملكه قط. Apple لا تقدم تقريرًا مماثلًا قبل المراجعة، لكن مراجعة أول نسخة TestFlight خارجية تمر على App Review.

ما الثابت وما المتغير؟

ثابت (افهمه مرة)

- الفصل بين الجهاز والنظام والمتجر والحساب.
- فكرة التوقيع ولماذا لا يتغير مفتاح التطبيق.
- اسم الإصدار للناس ورقم البناء للنظام.
- مسار: بناء ← اختبار ← صفحة متجر ← مراجعة ← إصدار ← تحديث.
- iOS لا يشغل إلا تطبيقات موقعة تعترف بها Apple.

متغير (راجع قبل كل نشر)

- الرسوم وشروط التحقق من الهوية.
- الحد الأدنى لـ targetSdk و iOS SDK.
- نماذج الخصوصية والبيانات ومتطلبات حذف الحساب.
- عدد المختبرين ومدد الاختبار.
- أحجام الصور المطلوبة لصفحة المتجر.

خلاصة الفصل

- Android نظام مفتوح على أجهزة كثيرة يسمح بالتثبيت المباشر؛ iOS نظام مغلق على أجهزة Apple لا يشغل إلا الموقع رسميًا.
- حساب Google Play بـ 25 دولارًا مرة واحدة، وحساب Apple بـ 99 دولارًا سنويًا، وكلاهما يتطلب التحقق من الهوية.
- ترفع .aab إلى Play و .ipa إلى App Store Connect، وكلاهما موقع بمفاتيح يجب أن تحميها.
- المتطلبات التقنية للمتاجر تتغير سنويًا؛ احفظ الروابط لا الأرقام.

CHAPTER 03

رحلة التطبيق: من الفكرة إلى الصيانة

التطبيق ليس شيئاً «يكتب» ثم ينتهي. هو شيء يفكر فيه، يصمم، يكتب، يختبر، ويبني، وينشر، ثم يسان لسنوات. أكثر ما يميز المطور الناضج عن المبتدئ ليس سرعة الكتابة، بل معرفته بأي مرحلة هو فيها الآن، وما الذي تحتاجه هذه المرحلة تحديداً. هذا الفصل يعرض المراحل السبع، وما ينتجها كل منها، وأين يقف الذكاء الاصطناعي في كل واحدة.

المراحل السبع في صورة واحدة

المرحلة	السؤال الذي تجيب عنه	الناتج الملموس	الخطأ الشائع
1. الفكرة	أي مشكلة أحل، ولن؟	فقرة واحدة تصف المشكلة والمستخدم	البدء بقائمة ميزات بدل مشكلة
2. التصميم	ما الشاشات؟ كيف ينتقل المستخدم؟ ما البيانات؟	رسومات شاشات، مخطط تنقل، قائمة كيانات البيانات	القفز إلى الكود «لتوفير الوقت»
3. الكود	كيف أبني أصغر نسخة تعمل؟	مشروع يعمل على المحاكى، بحفظ منتظم في Git	بناء كل الميزات معاً قبل أن يعمل أي شيء
4. الاختبار	هل يعمل كما قصدت، وفي الحالات الصعبة؟	اختبارات آلية + جلسات يدوية على أجهزة مختلفة	الاختبار على جهاز المطور فقط في «المسار السعيد»
5. البناء	كيف أنتج ملفاً موقعاً صالحاً للمتجر؟	.aab و .ipa. موقعان برقم بناء صحيح	ترك التوقيع والأيقونات والأذونات إلى الليلة الأخيرة
6. النشر	كيف يصل إلى المستخدمين قانونياً وبأمان؟	صفحة متجر مكتملة، سياسة خصوصية، مسار اختبار ثم إنتاج	اكتشاف شرط الـ 12 مختبراً أو نموذج البيانات في اللحظة الأخيرة
7. الصيانة	ما الذي ينكسر، ولماذا، وكيف أحدث؟	تقارير أعطال، ملاحظات مستخدمين، إصدارات دورية	الاعتقاد بأن العمل انتهى عند النشر

1. الفكرة: مشكلة واحدة لشخص محدد

الفكرة الجيدة تقال في جملة على هذا النمط: «[شخص محدد] يعاني من [مشكلة ملموسة] وأريد أن أجعل [فعلًا واحدًا] أسهل». مثال: «طالب يريد المحافظة على أذكار الصباح والمساء يفقد العد حين ينشغل؛ أريد عدادًا يتذكر مكانه». لاحظ ما ليس في الجملة: لا إحصاءات، لا حسابات، لا مزامنة، لا تصميم فاخر. كل ذلك قد يأتي لاحقًا أو لا يأتي.

2. التصميم: قبل الكود لا بعده

التصميم هنا ليس تلوين الأزرار. هو الإجابة عن ثلاثة أسئلة على ورقة: ما الشاشات؟ كيف ينتقل المستخدم بينها؟ ما البيانات التي يجب أن تعيش بعد إغلاق التطبيق؟ رسم بالقلم على ورقة يكفي تمامًا في البداية، وأداة مثل Figma أو Excalidraw حين تريد مشاركته. سنتدرب على هذا بالتفصيل في الفصل السادس.

3. الكود: أصغر شيء يعمل، ثم توسّع

المبدأ الحاكم: في كل لحظة يجب أن يكون لديك تطبيق **يعمل**، ولو كان يفعل شيئًا واحدًا تافهًا. تبدأ بشاشة ثابتة تظهر على المحاكى، ثم تجعل زرًا يعمل، ثم تحفظ قيمة، ثم تضيف شاشة ثانية. بين كل خطوة خطوة تحفظ في Git (سنشرحه في الفصل الخامس) حتى تستطيع العودة إن كسرت شيئًا. الفصل التاسع يطبق هذا حرفيًا على عشر مراحل.

4. الاختبار: ما لم يُختبر فهو مكسور

هناك ثلاثة مستويات، ستقابلها كلها في الفصل السابع: اختبارات وحدة لدالة أو صنف واحد، واختبارات واجهة لعنصر شاشة، واختبارات تكامل للتطبيق كاملاً. وهناك الاختبار اليدوي الذي لا غنى عنه: أن تمسك الهاتف وتحاول كسر تطبيقك: تدير الشاشة، تقطع الإنترنت، تدخل نصًا فارغًا، تضغط الزر عشر مرات بسرعة. المتجر أيضًا يختبر: تقرير ما قبل الإطلاق في Play يشغل تطبيقك على أجهزة لم ترها.

5. البناء: من كود إلى ملف موقع

البناء عملية آلية: المترجم يحول كود Dart إلى كود آلة، والأدوات تضم الصور والخطوط، ثم يوقع الناتج. على Android يجري كل هذا على جهازك. على iOS يحتاج إلى macOS وXcode، ولهذا سنستخدم خدمة سحابية. البناء «الإصداري» (release) يختلف عن بناء التطوير (debug): أصغر وأسرع ولا يحتوي أدوات التصحيح، وهو الذي يجب أن تختبره قبل النشر لأنه قد يكشف أخطاء لا تظهر في وضع التطوير.

6. النشر: عمل إداري بقدر ما هو تقني

ستقضي وقتاً مفاجئاً في أمور غير برمجية: كتابة الوصف، التقاط لقطات شاشة بالأحجام المطلوبة، رسم أيقونة، كتابة سياسة خصوصية (مطلوبة حتى لو لم تجمع أي بيانات، فعليك أن تقول ذلك)، وملء نماذج أمان البيانات. ابدأ هذه المهام في منتصف المشروع لا في نهايته.

7. الصيانة: هنا يعيش التطبيق معظم عمره

بعد النشر ستصلك أعطال من أجهزة لم تعرفها، وملاحظات من مستخدمين لم يفهموا شاشة ظننتها بديهية، وتحديثات من Flutter والحزم والمتاجر تتطلب تعديلات. المطور المحترف يترك في تطبيقه من البداية ما يسهل الصيانة: أداة تتبع أعطال، أرقام إصدارات واضحة، سجل تغييرات، وكود منظم يمكن فهمه بعد ستة أشهر.

أين يقف الذكاء الاصطناعي في كل مرحلة؟

المرحلة	ما يجيده المساعد	ما يجب أن يبقى بيدك
الفكرة	طرح أسئلة توضيحية، اقتراح تضيق النطاق، كشف الافتراضات	قرار ما المشكلة التي تستحق الحل، ومن المستخدم
التصميم	اقتراح شاشات وتدفقات ونموذج بيانات، ونقد تصميمك	اختيار ما يدخل النسخة الأولى وما يؤجل
الكود	كتابة قطع صغيرة محددة، شرح كود موجود، اقتراح بنية	قراءة كل سطر تقبله، وفهمه، وتشغيله
الاختبار	اقتراح حالات اختبار، كتابة هيكل الاختبار، تفسير الفشل	تحديد ما المهم اختباره، والاختبار اليدوي على جهاز حقيقي
البناء	تفسير رسائل خطأ البناء، شرح إعدادات التوقيع	المفاتيح وكلمات المرور: لا تمر عبر المساعد أبداً

المرحلة	ما يجيده المساعد	ما يجب أن يبقى بيدك
النشر	مسودة الوصف وسياسة الخصوصية، قائمة تحقق بالمتطلبات	التحقق من الوثائق الرسمية الحالية، وصحة ما تعلنه عن بياناتك
الصيانة	تحليل سجل الأعطال، اقتراح سبب محتمل، صياغة رد على مستخدم	إعادة إنتاج الخطأ فعلياً قبل «إصلاحه»

نصيحة شخصية

ارسم هذا الجدول على ورقة وعلقه أمامك. كلما شعرت بالضياع اسأل: في أي مرحلة أنا؟ وما الناتج الملموس الذي يجب أن أخرج به منها؟ معظم الإحباط في المشاريع الطلابية سببه العمل في ثلاث مراحل معاً دون إنهاء أي منها.

CHAPTER 04

اختيار التقنية واللغة

سؤال «أي تقنية أتعلم؟» هو أكثر ما يستهلك وقت المبتدئين، وأقل ما يستحق. الحقيقة أن معظم ما ستتعلمه (التفكير في الحالة، بنية التطبيق، الاختبار، النشر) ينتقل معك بين التقنيات. لكن الاختيار مهم لسبب عملي: تكلفة البداية وتكلفة الصيانة تختلفان. هذا الفصل يقارن المسارات الخمسة بصدق، ويشرح لماذا يرشح هذا الدليل Flutter لمشروعك الأول، ومتى يكون غيره أفضل.

المسارات الخمسة

مقارنة عامة (الإصدارات الحالية بتاريخ 2026-09-27)

المسار	اللغة	النتائج	يحتاج Mac لبناء iOS؟	أبرز نقاط القوة	أبرز القيود
Android أصلي Kotlin + Jetpack Compose	Kotlin 2.4	Android فقط	لا ينطبق	أفضل تكامل مع Android؛ توصية Google الرسمية؛ Android Studio مبني له	لا iOS إطلاقًا؛ لبناء iOS ستبدأ من الصفر بلغة أخرى
iOS أصلي Swift + SwiftUI	Swift 6.4	iOS (وبقية منصات Apple)	نعم، دائمًا. Xcode 27 يتطلب macOS 26.6 وشريحة Apple silicon	أفضل تكامل مع iOS؛ الوصول الفوري لميزات Apple	مستحيل عمليًا دون Mac؛ لا Android
Flutter Dart	Dart 3.13	Android و iOS والويب وسطح المكتب	للبناء نعم (يمكن عبر خدمة سحابية)؛ للكتابة والاختبار على Android لا	واجهة متطابقة على المنصتين؛ أدوات ممتازة (hot reload، DevTools)؛ لغة واحدة لكل شيء؛ Material 3 مدمج	لا يستخدم عناصر النظام الأصلية (يرسمها)؛ الحزم الأصلية أحيانًا أقل نضجًا؛ الويب ثقيل نسبيًا وغير مناسب للمواقع النصية

المسار	اللغة	الناتج	يحتاج Mac لبناء ؟iOS	أبرز نقاط القوة	أبرز القيود
React Native JavaScript / TypeScript	RN 0.87 عبر Expo	Android و iOS (والويب عبر Expo)	للبناء نعم (EAS) Build يوفر Mac (سحابيًا)	مناسب جدًا لمن يعرف React والويب؛ عناصر واجهة أصلية؛ منظومة JavaScript الضخمة	الجسر بين JS والنظام يضيف طبقة تعقيد؛ تنوع الأدوات قد يربك المبتدئ؛ تحديثات كبيرة متكررة (البنية الجديدة إلزامية منذ 0.82)
ويب / PWA HTML/CSS/ إطار JS +	أي إطار ويب	متصفح؛ قابل للتغليف لـ Play عبر TWA	لا	أرخص وأسرع نشرًا؛ لا حساب متجر؛ تحديث فوري	قيود iOS على الإشعارات والتخزين والوصول للعتاد؛ لا وجود طبيعي في المتاجر؛ Apple ترفض «الموقع المغلف»

المصادر

<https://developer.android.com/compose>

<https://developer.apple.com/support/xcode/>

<https://docs.flutter.dev/reference/supported-platforms>

<https://reactnative.dev/docs/environment-setup>

<https://docs.expo.dev/build/introduction/>

<https://developer.apple.com/app-store/review/guidelines/#minimum-functionality>

توجد مسارات أخرى تستحق الذكر دون تفصيل: **Kotlin Multiplatform / Compose Multiplatform** (أصبح مستقرًا على iOS منذ 2025 لكنه لا يزال يحتاج Mac و Xcode للبناء)، و **.NET MAUI**. لمن يعرف C#، و **Ionic** و **Capacitor** لتغليف تطبيق ويب في حاوية أصلية. لا يرشحها الدليل للمبتدئ لأن موارد التعلم العربية والإنجليزية للمبتدئين فيها أقل، وليس لأنها سيئة.

لماذا يرشح هذا الدليل Flutter للبداية؟

ليس لأنه «الأفضل»، بل لأنه يقلل عدد الأشياء التي يجب أن تتعلمها في الوقت نفسه، وهذا هو العامل الحاسم للمبتدئ:

1. لغة واحدة، أداة واحدة، مشروع واحد: Dart للمنطق والواجهة معًا، ومجلد واحد يخرج منه تطبيقان. في React Native ستتعامل مع JavaScript وأدوات بناء JS ونظام حزم npm وطبقة أصلية، وكلها مجالات بحد ذاتها.
2. Dart لغة مألوفة لمن درس Java أو C# أو JavaScript: أصناف، أنواع ثابتة، دوال، `async/await`. لن تقضي أسابيع على اللغة نفسها.
3. الواجهة قابلة للتوقع: لأن Flutter يرسم كل شيء بنفسه، ما تراه على محاكي Android هو ما سيراه مستخدم iPhone تقريبًا. هذا مهم جدًا لطالب لا يملك iPhone للاختبار المستمر.
4. الوثائق الرسمية ممتازة ومجانية، ومسار التعلم الرسمي (Flutter learning pathway) مصمم للمبتدئ، والأدوات (hot reload) الذي يعرض تعديلك في أقل من ثانية، و DevTools تجعل التعلم بالتجربة سريعًا.
5. أدوات الذكاء الاصطناعي تعرفه جيدًا: كمية الكود المفتوح المكتوب بـ Flutter كبيرة، فالاقترحات دقيقة نسبيًا، بشرط أن تخبر الأداة برقم إصدارك لأن الواجهات تتغير.
6. الدعم العربي و RTL مدمج: مكتبة الترجمة الرسمية تحتوي نصوص العربية، واتجاه العرض ينقلب تلقائيًا حين تضبط اللغة، كما ستري في الفصل التاسع.

حدود Flutter التي يجب أن تعرفها بصدق

- ليس أصليًا في المظهر: تطبيق Flutter بتصميم Material يبدو «تطبيق Android» على iPhone ما لم تبذل جهدًا (مكتبة Cupertino تساعد). معظم المستخدمين لا يلاحظون؛ بعض العملاء يلاحظون.
- حجم التطبيق: تطبيق Flutter فارغ أكبر من نظيره الأصلي بعدة ميغابايتات لأنه يحمل محرك الرسم معه. نادرًا ما يهم اليوم.
- الاعتماد على الحزم للميزات الأصلية: الكاميرا والإشعارات والملفات تأتي عبر حزم على `pub.dev`. الشائعة منها ممتازة ومصانة (من Google أو مجتمع نشط)، والنادرة قد تهمل. تعلم قراءة صفحة الحزمة: تاريخ آخر تحديث، الناشر، عدد المستخدمين، وشارة Flutter Favorite.

- **الويب: Flutter** للويب مناسب للتطبيقات التفاعلية لا للمواقع النصية أو التي تحتاج إلى ظهور في محركات البحث، وهذا ما تقوله وثائقه الرسمية بنفسها. كما أنه لم يعد يولد **Service Worker** تلقائيًا؛ إن أردت PWA تعمل دون اتصال فعليك إضافة ذلك بنفسك.
- **قرارات Google: Flutter** مشروع مفتوح المصدر تقوده Google، وهي تاريخيًا توقف مشاريع. المجتمع كبير بما يكفي ليقفل هذا الخطر، لكنه ليس صفرًا.

متى يكون غير Flutter أفضل؟

الحالة	الخيار الأنسب غالبًا	لماذا
تطبيق Android فقط، بتكامل عميق مع ميزات النظام (ودجات معقدة، خدمات خلفية طويلة، أجهزة خاصة)	Kotlin + Compose	لا طبقة وسيطة، وكل وثائق Google مكتوبة له
تطبيق iOS فقط، وأنت تملك Mac، وتريد أعلى جودة على iPhone	Swift + SwiftUI	التكامل الأصلي مع iOS وميزات Apple الجديدة يوم صدورها
فريقك يجيد React ويبني بالفعل موقع ويب بـ React	React Native + Expo	إعادة استخدام المعرفة وربما جزء من الكود
أداة داخلية أو محتوى يقرأ غالبًا، جمهور على سطح المكتب أيضًا	ويب / PWA	أسرع وأرخص، ولا حاجة إلى متجر
موقع ويب نصي كثيف يحتاج إلى ظهور في Google	ويب تقليدي (ليس Flutter Web)	محركات البحث لا تفهرس Flutter Web جيدًا
ألعاب ثلاثية الأبعاد	Godot أو Unity	محركات ألعاب متخصصة

أسئلة تطرحها على نفسك قبل أن تقرر

- ☐ هل أحتاج فعلًا إلى iOS في النسخة الأولى؟ (إن كان مستخدموك الأوائل زملاءك وكلهم Android، ربما لا.)
- ☐ هل لدي من يختبر على iPhone؟ (بدون شخص واحد يملك iPhone، ستنشر على iOS دون أن ترى تطبيقك عليه إلا عبر TestFlight.)
- ☐ ما اللغة التي أعرفها أفضل؟ (المعرفة السابقة تختصر أسابيع.)
- ☐ هل فكرتي تحتاج إلى ميزة نادرة في العتاد؟ (ابحث عن حزمة Flutter لها قبل أن تلتزم.)
- ☐ ما الذي سيقبله المشرف أو الجهة التي أقدم لها المشروع؟

مع الذكاء الاصطناعي

لا تسأل: «ما أفضل تقنية لتطبيقات الهاتف؟» فستحصل على جواب عام لا يفيد. اسأل: «هذه فكرتي (فقرة)، هذه خبرتي (اللغات التي أعرفها)، هذه أجهزتي (Windows، هاتف Android)، وهذا وقتي (10 ساعات أسبوعيًا لثلاثة أشهر). قارن لي Flutter و React Native و PWA لهذه الحالة تحديدًا، مع ما سأخسره في كل خيار». الجواب المحدد قابل للنقاش؛ الجواب العام لا.

خلاصة الفصل

- الخيارات الخمسة كلها مشروعة؛ ما يختلف هو تكلفة البداية وتكلفة الصيانة ومدى ملاءمتها لفكرتك وفريقك.
- Flutter خيار بداية جيد لأنه يقلل عدد الأشياء التي تتعلمها معًا، ويعطيك المنصتين من قاعدة واحدة، ويعمل بشكل ممتاز على Windows و Linux لكل شيء عدا بناء iOS.
- التقنية الأصلية أفضل حين تكون منصة واحدة وتكامل عميق؛ الويب أفضل حين لا يهم المتجر وتكفي إمكانات المتصفح.
- القرار الذي تتخذه اليوم ليس نهائيًا: المفاهيم تنتقل معك.

CHAPTER 05

الأدوات وبيئة العمل: مسار الطالب الذي لا يملك Mac

هذا الفصل يجيب عن السؤال الذي يمنع كثيرًا من الطلاب من البدء: «أملك جهاز Windows أو Linux وهاتف Android. هل أستطيع فعلًا بناء تطبيق يعمل على iPhone؟». الجواب الصادق: نعم لمعظم الرحلة، بشرط أن تفهم بدقة أين تنتهي قدرات جهازك وتبدأ حاجتك إلى خدمة سحابية أو إلى شخص يملك iPhone. نبدأ بتفكيك «تطوير التطبيق» إلى أربعة أنشطة مختلفة، لأن الجواب يختلف لكل نشاط.

أربعة أنشطة مختلفة، لا نشاط واحد

النشاط	ما معناه	على Windows / Linux أو Android	على Windows /Linux أو iOS
كتابة الكود	تحرير ملفات Dart، تنظيم المشروع، استخدام Git	✓ كامل	✓ كامل
تشغيل التطبيق واختباره	رؤية التطبيق يعمل، تجربة اللمس، تصحيح الأخطاء خطوة بخطوة، hot reload	✓ محاكي + جهاز حقيقي	محلّيًا ✗ لا محاكي iOS خارج macOS. البدائل: اختبارات آلية على محاكي سحابي، أو TestFlight على iPhone حقيقي، أو استعارة Mac
بناء ملف الإصدار	إنتاج .aab أو .ipa موقع	✓ محلّيًا	عبر السحابة Codemagic أو GitHub Actions أو خدمة مماثلة توفر Mac
النشر	الرفع إلى المتجر وإدارة صفحته	✓ من المتصفح	✓ App Store Connect موقع ويب؛ الرفع نفسه تتولاه خدمة البناء

القيد الذي لن يزول: محاكي iOS لا يعمل على Windows أو Linux

محاكي iOS (iOS Simulator) جزء من Xcode، وXcode لا يعمل إلا على macOS. لا توجد طريقة رسمية لتشغيله على Windows أو Linux، وأي أداة تدعي ذلك إما تشغل macOS افتراضياً بما يخالف شروط Apple أو تعرض تطبيقك عن بعد على Mac حقيقي في مكان آخر. حتى وثائق Codemagic تقول صراحةً إنك ستحتاج إلى Mac إن أردت تصحيح تطبيقك تفاعلياً على محاكي أو جهاز iOS. خطط على هذا الأساس: طور واختبر على Android، وابن iOS في السحابة، واختبره على iPhone حقيقي عبر TestFlight بمساعدة زميل إن لم تملك واحداً.

قائمة الأدوات ودور كل منها

الأداة	دورها	هل هي إلزامية؟
Flutter SDK (ويأتي معه Dart)	الإطار نفسه: المترجم، أوامر flutter، المكتبات، أدوات البناء	نعم
Android Studio	ثلاثة أشياء في حزمة واحدة: محرر كود، وAndroid SDK (أدوات بناء Android التي يحتاجها Flutter)، وAndroid Emulator	SDK إلزامي: المحرر اختياري إن استخدمت VS Code: المحاكى اختياري إن استخدمت جهازاً حقيقياً
VS Code + إضافة Flutter	محرر أخف وأسرع بديل عن محرر Android Studio	اختياري (رأيي: ابدأ به)
جهاز Android حقيقي	الاختبار الحقيقي: اللمس، الاهتزاز، الكاميرا، الأداء الفعلي، البطارية	شبه إلزامي قبل أي نشر
Git + GitHub	حفظ تاريخ التغييرات والرجوع عنها ومشاركة الكود، وهو ما تقرأ منه خدمة البناء السحابية	نعم
Codemagic (أو GitHub Actions)	جهاز Mac في السحابة يبني نسخة iOS ويرفعها إلى TestFlight	حين تريد iOS
حساب Google Play وحساب Apple Developer	النشر	حين تريد النشر

الخطوة 1: تثبيت Flutter SDK

الوثائق الرسمية هي المرجع الوحيد الذي يجب أن تتبعه هنا، لأن الخطوات الدقيقة تتغير:

<https://docs.flutter.dev/install>

الفكرة العامة ثابتة: تنزل مجلد Flutter وتضعه في مسار لا يحتوي مسافات أو حروفًا خاصة ولا يحتاج إلى صلاحيات مدير (الوثائق تقترح `%USERPROFILE%\develop` على Windows و `~/develop` على Linux). ثم تضيف مجلد `flutter/bin` إلى متغير `PATH` حتى يعمل أمر `flutter` من أي مكان. على Windows تحتاج إلى Git for Windows مثبتًا مسبقًا؛ على Linux تحتاج إلى حزم أساسية:

```
# Linux (Debian/Ubuntu): وثائق وتطلبها
sudo apt-get install -y curl git unzip xz-utils zip libglu1-mesa
```

ثم شغل الأمر الأهم في حياتك القادمة مع Flutter:

```
flutter doctor
```

`flutter doctor` يفحص بيئتك ويخبرك بما ينقص: Android SDK غير موجود، تراخيص Android غير مقبولة، لا جهاز متصل. اقرأ مخرجاته سطرًا سطرًا ونفذ ما يقترحه. حين تطلب المساعدة من زميل أو من أداة نكاء اصطناعي، أرسل مخرجات `flutter doctor -v` كاملة؛ هي أول ما سيطلبه أي شخص خبير.

حالة الإصدار (تحقق 2026-09-27)

أحدث إصدار مستقر من Flutter هو 3.47.5 (صدر 2026-09-18) ويأتي مع Dart 3.13.4. الإصدارات المستقرة تصدر كل بضعة أشهر؛ ثبت القناة المستقرة (**stable**) دائمًا.

<https://docs.flutter.dev/release/release-notes>

الخطوة 2: Android Studio وAndroid SDK والمحاكي

حتى لو نويت الكتابة في VS Code، ثبت Android Studio لأنه أسهل طريقة للحصول على Android SDK وإدارة تحديثاته والمحاكي. من داخل Android Studio، افتح **SDK Manager** وتأكد من تثبيت: منصة Android الحديثة (API 36) هو ما تطلبه وثائق Flutter (حاليًا)، و **Build-Tools**، و **Command-line Tools**، و **Platform-Tools**، و **Emulator**. ثم اقبل التراخيص:

```
flutter doctor --android-licenses
```

متطلبات النظام (تحقق 2026-09-27)

أحدث Android Studio المستقر هو **2026.1.4 | Quail 4**. الحد الأدنى الرسمي: (8/10/11) Windows 64-bit أو Linux 64-bit بـ glibc 2.31 فأحدث، معالج x86_64 من الجيل الثاني من Intel Core أو أحدث أو AMD يدعم الافتراضية، 8 GB ذاكرة، 8 GB مساحة، شاشة 800×1280. للمحاكي توصي Google بـ 16 GB ذاكرة و 16 GB مساحة.

<https://developer.android.com/studio/install>

<https://developer.android.com/studio/run/emulator>

المحاكي وتسريع العتاد

المحاكي بدون تسريع عتادي بطيء إلى درجة لا تحتمل. التسريع يتطلب تفعيل الافتراضية في BIOS/UEFI (تسمى VT-x عند Intel و SVM أو AMD-V عند AMD). على Windows، الوثائق توصي بـ **WHPX** (منصة Windows Hypervisor، تفعل من «ميزات Windows»); على Linux، تحتاج إلى **KVM** وأن يكون مستخدمك في مجموعة **kvm**. المرجع:

<https://developer.android.com/studio/run/emulator-acceleration>

نصيحة شخصية

إن كان جهازك بذاكرة 8 GB فاستخدم هاتفك الحقيقي بدل المحاكى أساسًا؛ أسرع وأصدق. أنشئ محاكيًا واحدًا فقط بشاشة صغيرة (مثل Pixel 8) لاختبار أحجام شاشات مختلفة عند الحاجة.

الخطوة 3: جهاز Android حقيقي

فعل «خيارات المطور» بالنقر سبع مرات على «رقم الإصدار» في إعدادات الهاتف، ثم فعل **USB debugging**. وصل الهاتف بالكابل، واقبل بصمة الحاسوب على شاشة الهاتف، ثم تحقق:

```
flutter devices
```

يمكن أيضاً التصحيح عبر Wi-Fi (Wireless debugging) على Android 11 فأحدث. الجهاز الحقيقي يكشف ما لا يكشفه المحاكى: الاهتزاز، سطوع الشاشة، حجم الخط الذي كبره المستخدم، ضعف الشبكة، والأداء على معالج متواضع. لا تنشر تطبيقاً لم تختبره على جهاز حقيقي واحد على الأقل.

الخطوة 4: المحرر (VS Code أو Android Studio)

كلاهما يعمل جيداً مع Flutter. VS Code مع إضافة Flutter (تثبت معها إضافة Dart تلقائياً) أخف وأسرع بدءاً، وهو ما أُرشحه للطالب. Android Studio أثقل لكنه أقوى في أمور Android الخاصة (تحرير ملفات Gradle، محلل الأداء الأصلي). كلاهما يعطيك: إكمالاً تلقائياً، وتحذيرات فورية من المحلل، وتصحيحاً خطوة بخطوة، وزر **hot reload**.

المراجع

<https://docs.flutter.dev/tools/vs-code>

<https://docs.flutter.dev/tools/android-studio>

مصطلح: Hot Reload

ميزة تجعل Flutter يحقق تعديلك في التطبيق الجاري تشغيله خلال أقل من ثانية دون إعادة تشغيل وبدون فقدان الحالة. تعدل لون زر وتحفظ الملف فتري النتيجة فوراً. هذا يغير طريقة التعلم كلياً: جرب، انظر، عدل.

الخطوة 5: Git و GitHub

Git نظام يحفظ «لقطات» من مشروعك عبر الزمن، بحيث تستطيع العودة إلى أي لحظة، ومقارنة ما تغير، والعمل مع زملاء دون أن يكتب أحدكم فوق عمل الآخر. GitHub موقع يستضيف نسخة من مستودعك على الإنترنت. خدمات البناء السحابية تقرأ كودك من GitHub (أو GitLab أو Bitbucket)، فهو ليس رفاهية.

الحد الأدنى الذي تحتاجه من Git في مشروعك الأول، خمسة أوامر:

```
git init           # مرة واحدة: ابدأ تتبع المجلد
git add .          # اختر كل التغييرات للحفظ
git commit -m "add counter screen" # احفظ لقطة بوصف قصير واضح
git log --oneline  # اعرض تاريخ اللقطات
git push           # ارفع اللقطات إلى GitHub
```

مشروع Flutter الجديد يأتي بملف `.gitignore` جاهز يستثني ملفات البناء. أضف إليه بنفسك أي ملف يحتوي أسرارًا: `key.properties`، وملف `keystore`، ومفاتيح الواجهات البرمجية. القاعدة: ما زُفِع إلى GitHub ولو لثانية، اعتبره منشورًا.

نصيحة شخصية: متى تحفظ لقطة؟

كلما أنجزت شيئًا صغيرًا يعمل: «زر العد يعمل»، «الحفظ المحلي يعمل». عشر لقطات صغيرة في اليوم أفضل من لقطة كبيرة في الأسبوع، لأن العودة عن خطأ تصبح سهلة ومحددة. وقبل أن تقبل تغييرًا كبيرًا اقترحه الذكاء الاصطناعي، احفظ لقطة أولًا حتى تستطيع التراجع.

الخطوة 6: بناء iOS في السحابة بـ Codemagic

Codemagic خدمة تكامل مستمر (CI/CD) متخصصة في تطبيقات الهاتف، توفر أجهزة Mac في السحابة تبني مشروعك من GitHub وتوقعه وترفعه إلى TestFlight. ليست الخيار الوحيد (انظر البدائل أدناه)، لكنها الأسهل للمبتدئ مع Flutter لأنها تقدم Workflow Editor مرئيًا لمشاريع Flutter لا يتطلب كتابة ملفات إعداد.

الأسعار والحدود (تحقق 2026-09-27؛ تتغير كثيرًا)

- الحساب الشخصي المجاني: 500 دقيقة بناء شهريًا على أجهزة Mac mini M2، بناء واحد متزامن، عدد غير محدود من التطبيقات، ومدة قصوى للبناء 120 دقيقة. الدقائق المجانية لا تتوفر لحسابات الفرق.
- الدفع حسب الاستخدام: 0.095 دولار للدقيقة على M2، و0.114 على M4، و0.045 على Linux/Windows.
- تعلن Codemagic عن حسابات مجانية للمعلمين والطلاب والمنظمات غير الربحية.
- أجهزة البناء تأتي مع Xcode 26.6 افتراضيًا وXcode 27 كخيار حديث، ما يفي بمتطلب Apple الحالي.

<https://codemagic.io/pricing/>

<https://docs.codemagic.io/billing/pricing/>

<https://docs.codemagic.io/specs-macos/xcode-26-6/>

بناء تطبيق Flutter صغير لـ iOS يستغرق عادةً بين 10 و20 دقيقة على هذه الأجهزة، أي أن 500 دقيقة تكفي لنحو 25 إلى 50 بناءً شهريًا. هذا يعني شيئًا مهمًا في طريقة عملك: لا تبني iOS مع كل تعديل. طور واختبر على Android محليًا، وابن iOS عند معالم واضحة فقط.

ما الذي تحتاجه قبل أول بناء iOS؟

1. حساب Apple Developer مدفوع (99 دولارًا سنويًا). لا توجد طريقة حول هذا للوصول إلى TestFlight أو App Store.

2. مفتاح App Store Connect API: من App Store Connect ← Users and Access ← Integrations

App Store Connect API، أنشئ مفتاحًا (توصي وثائق Codemagic بصلاحيات App Manager).

ستحصل على ثلاثة أشياء: Issuer ID، وKey ID، وملف p8. يمكن تنزيله مرة واحدة فقط. احفظه في مدير كلمات مرور.

3. معرف التطبيق (Bundle ID) مسجلًا في حساب Apple، وسجل التطبيق منشأً في App Store Connect.

4. مستودع GitHub يحتوي مشروعك، متصلاً بحساب Codemagic.

بعد ذلك، مع التوقيع التلقائي، تقول وثائق Codemagic حرفيًا إنها «ستنشئ الشهادة وملف التزويد نيابةً عنك دون الحاجة إلى Mac». هذه هي النقطة التي كانت مستحيلة قبل سنوات وأصبحت روتينية اليوم.

المراجع

<https://docs.codemagic.io/flutter-code-signing/ios-code-signing/>

<https://docs.codemagic.io/yaml-code-signing/signing-ios/>

<https://developer.apple.com/help/app-store-connect/get-started/app-store-connect-api>

مثال ملف codemagic.yaml مبسّط

حين تتجاوز محرر سير العمل المرئي، تكتب ملفًا باسم `codemagic.yaml` في جذر المشروع. هذا مثال توضيحي مبني على وثائق Codemagic الرسمية وقت إعداد الدليل؛ راجع صفحة البدء السريع لأحدث صيغة قبل الاستخدام:

```
workflows:
  ios-testflight:
    name: iOS to TestFlight
    instance_type: mac_mini_m2
    max_build_duration: 60
    integrations:
      app_store_connect: my-asc-key # اسم المفتاح كما أضفته في إعدادات الفريق
    environment:
      ios_signing:
        distribution_type: app_store
        bundle_identifier: sa.edu.kau.students.dhikr
      flutter: stable
    scripts:
      - name: Get packages
        script: flutter pub get
      - name: Set up signing
        script: |
          keychain initialize
          app-store-connect fetch-signing-files "$BUNDLE_ID" --type IOS_APP_STORE
    --create
      xcode-project use-profiles
      - name: Build ipa
        script: flutter build ipa --release --export-options-plist=/Users/builder/
export_options.plist
    artifacts:
      - build/ios/ipa/*.ipa
    publishing:
      app_store_connect:
        auth: integration
        submit_to_testflight: true
```

اقرأ الملف كقصة: على جهاز Mac mini M2، بمفتاح App Store Connect المسجل، جهز ملفات التوقيع (وأنشئها إن لم توجد)، ثم ابن ملف ipa، واحتفظ به، وارفعه إلى TestFlight. المرجع:

<https://docs.codemagic.io/yaml-quick-start/building-a-flutter-app/>

وماذا عن Android في السحابة؟

Android لا يحتاج إلى السحابة لأنك تبنيه محليًا. لكن يمكن أتمتة رفعه إلى Google Play أيضًا عبر حساب خدمة من Google Cloud. تنبيه من وثائق Codemagic: أول نسخة من التطبيق يجب أن تُرفع يدويًا إلى Play Console قبل أن تعمل الأتمتة.

بدائل Codemagic

الخدمة	الملخص	التكلفة للمبتدئ (تحقق 2026-09-27)
GitHub Actions	عام (ليس خاصًا بالهاتف)؛ يوفر أجهزة macOS (macos-15 ، macos-26) لكنك تكتب كل شيء بنفسك وتدير التوقيع يدويًا أو عبر أدوات مثل fastlane	2,000 دقيقة شهريًا للمستودعات الخاصة في الخطة المجانية؛ المستودعات العامة مجانية على الأجهزة القياسية. دقيقة macOS تسعر بـ 0.062 دولار مقابل 0.006 لـ Linux، أي أن دقائق Mac تستهلك الرصيد أسرع بكثير
Xcode Cloud	خدمة Apple نفسها؛ 25 ساعة حوسبة شهريًا مشمولة في عضوية المطور	مشمول، لكن إعداد أول سير عمل يجب أن يتم من Xcode على Mac؛ بعدها تدار من App Store Connect على الويب. غير مناسب لمن لا يملك Mac إطلاقًا
Bitrise	خدمة CI للهاتف مشابهة	خطة Hobby مجانية بتطبيق خاص واحد و300 رصيد شهريًا
استئجار Mac عن بُعد (MacStadium وأمثالها)	جهاز Mac حقيقي تتصل به عن بعد؛ يعطيك Xcode والمحاكي كاملين	يبدأ من نحو 109 دولارات شهريًا لـ Mac mini M2 لدى MacStadium. مكلف للطالب، لكنه الخيار الوحيد لتصحيح التفاعلي دون شراء Mac

المراجع

<https://docs.github.com/en/billing/reference/actions-runner-pricing>

<https://developer.apple.com/xcode-cloud/>

<https://developer.apple.com/documentation/xcode/configuring-your-first-xcode-cloud-workflow>

<https://bitrise.io/pricing>

<https://macstadium.com/pricing>

كيف تختبر iOS فعلاً دون Mac؟ خطة واقعية

1. الاختبارات الآلية على محاكي سحابي: Codemagic يستطيع تشغيل اختبارات التكامل على iOS Simulator داخل أجهزته. تكتب الاختبار مرة وتشغله على المنصتين في السحابة.
2. TestFlight على iPhone حقيقي: ابن في السحابة، ارفع إلى TestFlight، وادع زميلاً يملك iPhone كمختبر داخلي. اطلب منه لقطات شاشة وتسجيل شاشة لكل شاشة في التطبيق، وخصوصاً: الحواف السفلية والعلوية (النتوء والمؤشر)، لوحة المفاتيح العربية، الوضع الداكن، وحجم الخط الكبير.
3. الاعتماد على تطابق Flutter: لأن Flutter يرسم واجهته بنفسه، فمعظم مشكلات iOS ستكون في «حواف» المنصة: الأذونات، الإشعارات، الملفات، الحفظ في الخلفية. ركز اختبارك هناك.
4. جلسة Mac مستعارة عند الحاجة: قبل النشر الأول، ساعة واحدة على Mac زميل أو معمل الجامعة لتشغيل التطبيق على المحاكى ورؤية الأخطاء في وحدة التحكم تساوي أياماً من التخمين.

قائمة تحقق: بيئتك جاهزة حين...

- ☐ flutter doctor يظهر علامة صح على Flutter وAndroid toolchain والمحرر (تحذير Xcode طبيعي ومتوقع على Windows/Linux).
- ☐ flutter create hello && cd hello && flutter run يشغل التطبيق الافتراضي على المحاكى أو على هاتفك.
- ☐ تعديل نص في lib/main.dart وحفظه يظهر فوراً بفضل hot reload.
- ☐ لديك مستودع GitHub خاص للمشروع، وأول git push نجح.
- ☐ تعرف مكان ملف .gitignore وما الذي يستثنيه.

- (لاحقًا، عند الحاجة إلى iOS) حساب Apple Developer نشط، ومفتاح App Store Connect API محفوظ في مكان آمن، وحساب Codemagic متصل بالمستودع.

خلاصة الفصل

- فكك «التطوير» إلى كتابة وتشغيل وبناء ونشر. على Windows/Linux تملك الأربعة كاملة لـ Android، وثلاثة منها لـ iOS، والرابع (التشغيل التفاعلي على iOS) يحتاج إلى Mac أو iPhone حقيقي عبر TestFlight.
- Flutter SDK وAndroid Studio (من أجل SDK والمحاكي) وGit هي الأساس؛ VS Code محرر مريح اختياري.
- Codemagic يبني iOS ويوقعه ويرفعه إلى TestFlight دون Mac، بحصة مجانية شهرية تكفي الطالب إن استخدمها بحكمة.
- الأسرار (keystore، مفاتيح API) لا تدخل Git ولا تعطى لأداة نكاء اصطناعي.

CHAPTER 06

التفكير في التطبيق قبل كتابة الكود

أكثر المشاريع الطلابية لا تفشل بسبب خطأ برمجي، بل لأن صاحبها بدأ الكتابة وهو لا يعرف بالضبط ما الذي يبنيه، فاستمر في الإضافة حتى صار المشروع أكبر من قدرته على إنجازه. هذا الفصل يعلمك عادة تفصل بين الهواة والمحترفين: قضاء ساعات قليلة على الورق قبل الكود، تختصر أسابيع بعده. وسنطبق كل خطوة على ثلاث أفكار: عداد أذكار، وحافظة فواتير وضمائمات، ومتابع أحداث وتواريخ.

ابدأ بمشكلة صغيرة، لا بتطبيق كبير

الفكرة الجيدة للمشروع الأول لها ثلاث صفات: تحل مشكلة تعاني منها أنت أو شخص تعرفه (حتى تعرف متى تحل)، ويمكن استخدامها بعد أسبوعين من العمل ولو بشكل بدائي، ولا تحتاج إلى مستخدمين آخرين لتكون مفيدة (لا شبكة اجتماعية، لا سوق). أفكارنا الثلاث تحقق هذه الصفات:

الفكرة	المشكلة بجملة	لماذا مناسبة للبداية
عداد أذكار	أفقد العد حين أنشغل، والمسبحة ليست معي دائماً	شاشة واحدة، بيانات بسيطة، لا شبكة، ونتيجة تستخدم من اليوم الأول
حافظة فواتير وضمائمات	حين يتعطل جهاز لا أجد فاتورته ولا أعرف إن كان الضمان ساريًا	يعلمك الكاميرا والملفات والتواريخ والبحث، ولا يزال محليًا
متابع أحداث وتواريخ	أنسى مواعيد التجديد والمناسبات والمواعيد النهائية	يعلمك الإشعارات المحلية والتقويم والتكرار

حدّد المستخدم المستهدف بدقة مزعجة

«الجميع» ليس مستخدمًا. اكتب شخصًا واحدًا محددًا: عمره، جهازه، متى يستخدم التطبيق، وما الذي يزعجه اليوم. لعداد الأذكار: «طالب جامعي، هاتف Android متوسط، يستخدمه بعد الصلاة وفي المواصلات، أحيانًا بيد واحدة وبدون أن ينظر إلى الشاشة». لاحظ كيف يغير السطر الأخير قرارات التصميم فورًا: زر كبير جدًا، اهتزاز عند بلوغ الهدف، لا حاجة إلى قراءة نص صغير.

الوظائف الأساسية وما يمكن تأجيله

اكتب كل ما يخطر ببالك من ميزات، ثم صنفها في ثلاث مجموعات بقسوة:

- لا يعمل التطبيق بدونها: هذه النسخة الأولى.
- تجعل التطبيق أفضل بوضوح: النسخة الثانية.
- لطيفة: ربما أبدًا.

مثال التصنيف لعدد الأذكار

لا يعمل بدونها (v1)	أفضل بوضوح (v2)	لطيفة (لاحقًا أو أبدًا)
زر يزيد العدد	قائمة أذكار متعددة بأهداف مختلفة	إحصاءات أسبوعية ورسوم بيانية
يتذكر العدد بعد إغلاق التطبيق	اهتزاز عند بلوغ الهدف	تذكير يومي بالأذكار
تصغير مع تأكيد	عد بزر الصوت أو بالنقر على أي مكان في الشاشة	مزامنة بين الأجهزة وحسابات
واجهة عربية صحيحة	الوضع الداكن	ودجت على الشاشة الرئيسية

انتبه

كل ميزة في العمود الأول تضاعف تقريبًا عدد الحالات التي يجب اختبارها. أربع ميزات أساسية تعمل تمامًا أفضل بما لا يقاس من اثنتي عشرة ميزة نصف مكتملة. المستخدم يغفر غياب ميزة، ولا يغفر ميزة تفقد بياناته.

نموذج المتطلبات: صفحة واحدة تكفي

هذا النموذج هو ما يجب أن يكون معك قبل فتح المحرر. املاه بجمل قصيرة. مثال مملوء لعدد الأذكار:

1. هدف التطبيق (جملة واحدة)

عداد أذكار يتذكر مكان توقفك، يستخدم بيد واحدة ودون النظر إلى الشاشة.

2. المستخدم

طالب على Android متوسط الأداء، يستخدمه لدقائق عدة مرات يوميًا.

3. الشاشات الرئيسية

(أ) شاشة العد: اسم الذكر، العدد الحالي، الهدف، زر كبير للعد، زر تصفير، زر لفتح القائمة. (ب) شاشة القائمة: قائمة الأذكار مع الهدف، والنقر على أحدها يختاره ويعود.

4. انتقال المستخدم

يفتح التطبيق ← شاشة العد بآخر ذكر مختار وعدده المحفوظ ← ينقر الزر مرارًا ← عند بلوغ الهدف يتغير لون الرقم ويهتز الهاتف ← يمكنه التصفير (مع تأكيد) أو فتح القائمة لاختيار ذكر آخر ← يعود إلى شاشة العد.

5. البيانات التي يجب حفظها

لكل ذكر: معرف ثابت، نص، هدف، وعدد حالي. إضافة إلى: معرف الذكر المختار حاليًا. حجم البيانات صغير جدًا (بضع مئات من البايتات).

6. الحالات غير المعتادة والأخطاء المحتملة

- المستخدم يغلق التطبيق فجأة أثناء العد: يجب ألا يفقد أكثر من نقرة واحدة على الأكثر.
- نقر سريع جدًا متكرر: يجب ألا يتجمد التطبيق أو يفقد نقرات.
- بيانات محفوظة تالفة (تحديث من إصدار قديم): يبدأ من الصفر بدل الانهيار.
- تجاوز الهدف: هل يستمر العد أم يتوقف؟ (قرار: يستمر، ويبقى اللون مميزًا).
- تصفير بالخطأ: تأكيد قبل التصفير.
- حجم خط النظام كبير جدًا: الرقم يجب ألا يخرج من الشاشة.

7. الحد الأدنى القابل للتجربة (MVP)

الشاشة (أ) فقط بذكر واحد ثابت، مع حفظ العدد. لا قائمة، لا اهتزاز، لا ألوان. إن استطاع زميلي استخدامها يومًا كاملاً فقد نجح الـ MVP.

مصطلح: MVP (Minimum Viable Product)

أصغر نسخة من التطبيق يمكن لشخص حقيقي أن يستخدمها فعليًا ويعطيك رأيًا عنها. ليس «نسخة سيئة»، بل نسخة صغيرة تعمل جيدًا. الهدف منها أن تتعلم مبكرًا إن كانت الفكرة تستحق، قبل أن تستثمر شهرًا.

المثال الثاني: حافظة الفواتير والضمانات

لنطبق النموذج على فكرة أكبر قليلًا لنرى كيف تظهر قرارات جديدة:

- **الهدف:** أحفظ صورة الفاتورة وتاريخ الشراء ومدة الضمان، وأجدها بالبحث حين أحتاجها.
- **الشاشات:** قائمة الفواتير (مع بحث)، شاشة إضافة/تعديل (صورة، اسم، متجر، تاريخ، مدة ضمان، ملاحظات)، شاشة تفاصيل (الصورة بحجم كامل، حالة الضمان).
- **البيانات:** هنا تظهر علاقة لأول مرة: فاتورة واحدة قد تحتوي عدة صور، وقد تنتمي إلى فئة. هذا يرشح قاعدة بيانات محلية (SQLite) بدل تخزين بسيط. والصور ملفات كبيرة تحفظ في مجلد التطبيق ويحفظ مسارها فقط في قاعدة البيانات.
- **الحالات غير المعتادة:** المستخدم يرفض إذن الكاميرا (يجب أن يستطيع اختيار صورة من المعرض أو الإدخال دون صورة)؛ الصورة كبيرة جدًا (ضغطها قبل الحفظ)؛ تاريخ شراء في المستقبل (رفض مع رسالة)؛ حذف بالخطأ (تراجع لبضع ثوان أو سلة محذوفات)؛ تغيير الهاتف (تصدير نسخة احتياطية إلى ملف واحد).
- **MVP:** إضافة فاتورة بصورة واسم وتاريخ، وعرضها في قائمة. بلا بحث ولا فئات ولا ضمان.

لاحظ كيف قادنا سؤال «ما البيانات؟» إلى قرار تقني (قاعدة بيانات بدل تخزين بسيط)، وكيف قادنا سؤال «ماذا لو رفض الإذن؟» إلى قرار تصميم (الإدخال ممكن دون صورة). هذا هو المقصود بـ «التفكير قبل الكود».

المثال الثالث: متابع الأحداث والتواريخ

- **الهدف:** أسجل حدثًا بتاريخ وأتلقى تنبيهًا قبله بوقت أحده.
- **القرار الأول الذي يفرض نفسه:** التنبيهات تعمل عبر إشعارات محلية ي جدولها التطبيق على الجهاز، ولا تحتاج إلى خادم. لكنها تحتاج إلى إذن من المستخدم على Android 13 فأحدث وعلى iOS دائمًا، وقد تتأخر على بعض أجهزة Android بسبب توفير الطاقة. هذه معلومات يجب أن تعرفها قبل أن تعد المستخدم بـ «تنبيه دقيق».

• **الحالات غير المعتادة:** تغيير المنطقة الزمنية أثناء السفر، حدث متكرر سنوياً (تاريخ ميلاد) مقابل حدث لمرة

واحدة، حدث في الماضي، حذف إشعار مجدول عند حذف الحدث.

• **MVP:** قائمة أحداث بتاريخ، مرتبة بالأقرب، بلا إشعارات. الإشعارات في v2 لأنها تجلب أنونات ومنطقاً خاصاً بالمنصة.

ارسم الشاشات على ورق

ارسم كل شاشة مستطيلاً بنسبة الهاتف، وضع فيه العناصر بخطوط بسيطة. لا ألوان ولا أيقونات جميلة. ثم ارسم أسهماً بين الشاشات تمثل الانتقالات، واكتب على كل سهم ما الذي يسببه («نقر على عنصر في القائمة»). هذا الرسم يسمى **wireframe**، وستدهشك كمية الأسئلة التي يطرحها: أين زر الرجوع؟ ماذا يظهر حين تكون القائمة فارغة؟ ما الذي يراه المستخدم أثناء التحميل؟

ثلاث حالات يجب أن ترسمها لكل شاشة تعرض بيانات: **الحالة الفارغة** (لا بيانات بعد)، **وحالة التحميل**، **وحالة الخطأ**. المبتدئ يرسم الحالة «الممتلئة» فقط ثم يفاجأ بشاشة بيضاء عند أول تشغيل.

قسّم العمل إلى مراحل صغيرة قابلة للعرض

خذ الـ MVP وقسمه إلى خطوات، كل خطوة تنتهي بشيء يمكن رؤيته على الشاشة. هذا التقسيم هو بالضبط ما سنتبعه في الفصل التاسع: شاشة ثابتة ← زر يعمل ← حفظ ← قائمة ← حالات غير معتادة ← اختبار ← عربية ← إصدار. كل خطوة يوم أو يومان، وفي نهايتها لقطة Git وتجربة على الهاتف.

مع الذكاء الاصطناعي في هذه المرحلة

المساعد ممتاز هنا كـ «محاوِر صعب». جرب هذه الطلبات بالترتيب:

1. «هذه فكرتي (فقرة). اطرح علي عشرة أسئلة توضيحية لا أستطيع الإجابة عنها بنعم أو لا، ولا تقترح حلولاً بعد».
 2. «بناءً على إجاباتي، املأ نموذج المتطلبات ذا الأقسام السبعة، وضع علامة على كل افتراض افترضته دون أن أخبرك به».
 3. «أعطني عشرين حالة غير معتادة قد تكسر هذا التطبيق، مرتبة بالأرجحية».
 4. «اقترح تقسيمًا إلى مراحل، كل مرحلة تنتهي بشيء يرى على الشاشة ويمكن تجربته».
- ما يبقى بيدك: قرار ما يدخل الـ MVP. المساعد يميل إلى التوسع؛ أنت من يقول «لا، ليس الآن».

خلاصة الفصل

- ابدأ بمشكلة واحدة لشخص محدد، لا بقائمة ميزات للجميع.
- صنف الميزات بقسوة: لا يعمل بدونها، أفضل بوضوح، لطيفة.
- املأ نموذج المتطلبات ذا الأقسام السبعة قبل فتح المحرر، وارسم الشاشات بحالاتها الثلاث.
- أسئلة البيانات والحالات غير المعتادة تقود إلى قرارات تقنية مبكرة وصحيحة.
- قسم الـ MVP إلى خطوات تنتهي كل منها بشيء يرى على الشاشة.

CHAPTER 07

المفاهيم التي يجب أن تتعلمها، مرتبة حسب مراحل البناء

هذا أطول فصل في الدليل، وهو مرجع تعود إليه أكثر من كونه قراءة متصلة. لن أعرف المفاهيم كقائمة قاموس؛ سأرتبها كما ستقابلها فعلاً وأنت تبني تطبيقاً، في خمس محطات: اللغة، ثم الواجهة، ثم البنية والبيانات، ثم حواف المنصة، ثم الجودة والعمر الطويل. في كل مفهوم ستجد ثلاثة أشياء: لماذا تحتاجه، أين يظهر في تطبيق حقيقي، والقرار الذي ستتخذه بشأنه. الأمثلة بلغة Dart، لكن المفاهيم نفسها تنطبق على Kotlin و Swift و TypeScript.

المحطة الأولى: اللغة التي تفكر بها

المتغيرات والأنواع

لماذا؛ كل شيء في التطبيق بيانات تتغير: العدد، اسم الذكر، تاريخ الفاتورة. المتغير اسم لصندوق يحمل قيمة، والنوع يحدد ما الذي يجوز وضعه في الصندوق. Dart لغة ذات أنواع ثابتة: إن قلت إن `count` عدد صحيح، فلن تستطيع وضع نص فيه، ويكتشف المحلل الخطأ قبل التشغيل لا بعده على هاتف المستخدم.

```
int count = 0; // عدد صحيح قابل للتغيير
final String name = 'سُبْحَانَ اللَّهِ'; // يُحدد مرة واحدة عند التشغيل
const int target = 33; // ثابت معروف وقت الترجمة
double progress = count / target; // عدد عشري
bool done = count >= target; // صواب/خطأ
String? note; // علامة ؟ تعني: قد يكون فارغاً
```

القرار الأهم في Dart: القابلية للفراغ (null safety). علامة الاستفهام تعلن أن القيمة قد تكون غائبة، ويجبرك المحلل على التعامل مع هذا الاحتمال قبل استخدامها. معظم انهيئات تطبيقات المبتدئين في اللغات الأقدم سببها «قيمة فارغة لم يتوقعها أحد». في Dart، الفراغ قرار تعلنه لا حادث تكتشفه. استخدم ؟ فقط حين يكون الغياب معنى حقيقياً (ملاحظة اختيارية)، لا كطريقة لتأجيل التفكير.

أين يظهر: نموذج الفاتورة: `DateTime purchaseDate` إلزامي، لكن `int? warrantyMonths` اختياري لأن بعض المشتريات بلا ضمان. هذا القرار في النوع يعني أن شاشة التفاصيل يجب أن تعالج حالة «لا ضمان» صراحةً.

الدوال

لماذا: الدالة قطعة منطق لها اسم ومدخلات ومخرج. تفيدك في ثلاثة أشياء: إعادة الاستخدام، وإعطاء اسم واضح لعملية معقدة، والاختبار المنفصل. الدالة الجيدة تفعل شيئًا واحدًا ويمكن اختبارها دون تشغيل الواجهة.

```
/// هل الضمان ساريًا في تاريخ معيّن؟ دالة نقية: نفس المدخلات تعطي نفس الناتج دائمًا
bool isUnderWarranty({
  required DateTime purchaseDate,
  required int? warrantyMonths,
  required DateTime today,
}) {
  if (warrantyMonths == null) return false;
  final expiry = DateTime(purchaseDate.year, purchaseDate.month + warrantyMonths,
    purchaseDate.day);
  return !today.isAfter(expiry);
}
```

لاحظ أننا مررنا `today` كمدخل بدل استدعاء `DateTime.now()` داخل الدالة. لماذا؟ لأن الاختبار يحتاج إلى تثبيت «اليوم» ليعطي نتيجة متوقعة. هذا قرار صغير يعلمك مبدأً كبيرًا: الدالة التي تعتمد على شيء خارجي (الوقت، الشبكة، الملفات) يصعب اختبارها؛ مرّر ذلك الشيء إليها.

الكائنات والأصناف

لماذا: البيانات المترابطة تجمع في وحدة واحدة. الفاتورة ليست خمسة متغيرات متفرقة، بل «شيء» له اسم وتاريخ وصورة وسلوك (هل ضمانها سار؟). الصنف (`class`) هو القالب، والكائن (`object`) نسخة منه.

```
class Invoice {
  const Invoice({
    required this.id,
    required this.title,
    required this.purchaseDate,
    this.warrantyMonths,
  });

  final String id;
  final String title;
  final DateTime purchaseDate;
  final int? warrantyMonths;

  bool isUnderWarrantyOn(DateTime today) => isUnderWarranty(
    purchaseDate: purchaseDate,
    warrantyMonths: warrantyMonths,
```

```

        today: today,
    );

    /// نسخة معدلة بدل تعديل الأصل: الكائنات غير القابلة للتغيير أسهل تتبعًا
    Invoice copyWith({String? title, int? warrantyMonths}) => Invoice(
        id: id,
        title: title ?? this.title,
        purchaseDate: purchaseDate,
        warrantyMonths: warrantyMonths ?? this.warrantyMonths,
    );
}

```

القرار: اجعل نماذج بياناتك غير قابلة للتغيير (**immutable**) بحقول **final** ودالة **copyWith**. حين تتغير الفاتورة، تنشئ نسخة جديدة بدل تعديل القديمة. هذا يمنع فئة كاملة من الأخطاء التي يصعب تتبعها: «من غير هذه القيمة ومتى؟».

القوائم والمجموعات

لماذا: شاشة القائمة في كل تطبيق هي في الحقيقة **List** من الكائنات معروضة عنصرًا عنصرًا. والخريطة **Map** تربط مفتاحًا بقيمة، وهي ما استخدمناه لحفظ «معرف الذكر ← العدد». تعلم العمليات الأربع الأساسية على القوائم لأنها تظهر في كل مكان:

```

final invoices = <Invoice>[...];
final today = DateTime.now();

final active = invoices.where((i) => i.isUnderWarrantyOn(today)).toList(); //
// تصفية
final titles = invoices.map((i) => i.title).toList(); //
// تحويل
final sorted = [...invoices]..sort((a, b) =>
    b.purchaseDate.compareTo(a.purchaseDate)); // ترتيب (على نسخة)
final byId = {for (final i in invoices) i.id: i}; //
// قائمة ← خريطة

```

البرمجة غير المتزامنة: العمليات التي تستغرق وقتًا

لماذا: قراءة ملف، أو استعلام قاعدة بيانات، أو طلب من الشبكة، كلها تستغرق وقتًا لا يستطيع التطبيق أن «يقف» في انتظاره، وإلا تجمدت الشاشة ولم تستجب للمس. الحل: تبدأ العملية وتعد بأنها ستنتهي لاحقًا، ويواصل التطبيق رسم الشاشة، ثم حين تنتهي تستدعي بقية الكود. هذا الوعد في Dart اسمه **Future**، والكلمتان **async** و **await** تجعلانه يقرأ كأنه كود عادي.

```
Future<Map<String, int>> loadCounts() async {
  final raw = await prefs.getString('counts'); // انتظر دون تجميد الواجهة
  if (raw == null) return {};
  return decode(raw);
}

// الأخطاء في العمليات غير المتزامنة تُمسك بنفس الطريقة:
Future<void> save() async {
  try {
    await store.saveCounts(counts);
  } on FileSystemException catch (e) {
    log('failed to save: $e'); // لا تبتلع الخطأ بصمت
    rethrow;
  }
}
```

مصطلح: الخيط الرئيسي (main/UI thread)

الخيط الذي يرسم الشاشة ويستقبل اللمس. في Flutter، كل كود Dart يعمل عليه افتراضياً، لذلك يجب ألا تشغل عليه حساباً طويلاً (معالجة صورة كبيرة مثلاً)؛ تلك تنقل إلى `Isolate` منفصل عبر `compute()`. أما الانتظار (`await`) فلا يحجز الخيط، لأنه انتظار وليس عملاً.

أين يظهر: في كل مكان يلمس الجهاز أو الشبكة: تحميل العد المحفوظ عند بدء التطبيق (لهذا كان لدينا مؤشر تحميل في الفصل التاسع)، حفظ صورة الفاتورة، جلب موافقت الصلاة. القرار: ماذا يرى المستخدم أثناء الانتظار؟ مؤشر دائري؟ الشاشة القديمة؟ هيكل رمادي؟ وماذا لو فشل؟ صمم الحالتين معاً.

المحلة الثانية: الواجهة التي يلمسها المستخدم

واجهة المستخدم كشجرة من الـ Widgets

لماذا: في Flutter كل شيء على الشاشة `Widget`: النص، الزر، الهامش، العمود، الشاشة كاملة. تتركب الواجهة بتعشيش الـ `widgets` في شجرة، وFlutter يرسمها. الفكرة الجوهرية التي يجب أن تستقر في ذهنك: **الواجهة دالة من الحالة**. لا تقول «غير نص هذا العنصر»، بل تقول «إن كانت الحالة كذا فارسم كذا»، وحين تتغير الحالة يعاد الرسم.

```
Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Text(dhikr.text, style: theme.textTheme.headlineMedium),
    const SizedBox(height: 24),
    Text('$count', style: theme.textTheme.displayLarge),
    FilledButton(onPressed: onTap, child: const Text('سَيِّح')),
  ],
)
```

القرار: متى تقسم الشاشة إلى widgets أصغر؟ القاعدة العملية: حين تتجاوز دالة `build` شاشة واحدة من المحرر، أو حين تتكرر قطعة في مكانين، أو حين تحتاج قطعة إلى حالة خاصة بها. الـ widget الصغير المسمى جيداً (`DhikrTile` ، `CounterDisplay`) يقرأ زميلك ويختبره ويعيد استخدامه.

التنقل بين الشاشات

لماذا: التطبيق ذو الشاشة الواحدة نادر. التنقل يعني إدارة «كومة» من الشاشات: تفتح القائمة فوق شاشة العد، وحين تعود تزال من الكومة. في Flutter الأداة الأساسية هي `Navigator`:

```
// افتح شاشة القائمة فوق الحالية
Navigator.push(context, MaterialPageRoute<void>(builder: (_) => const
DhikrListScreen()));

// عد إلى الشاشة السابقة (ويمكن إرجاع قيمة)
Navigator.pop(context, selectedDhikr);
```

القرار: في التطبيقات الصغيرة (2 إلى 5 شاشات) يكفي `Navigator` المباشر. حين تحتاج إلى روابط عميقة (`deep links`: رابط يفتح شاشة محددة)، أو تنقل بشريط سفلي معقد، أو ويب بعناوين URL، انتقل إلى حزمة `go_router` (من فريق Flutter). لا تبدأ بالأداة الأعقد قبل أن تحتاجها.

إدارة الحالة: أهم مفهوم في تطوير الواجهات

لماذا: «الحالة» هي كل بيانات تتغير مع الزمن وتؤثر في ما يعرض: العدد الحالي، هل القائمة تحمل، أي عنصر مختار، هل المستخدم مسجل دخوله. المشكلة ليست في تخزين هذه القيم بل في إخبار الأجزاء المناسبة من الواجهة حين تتغير، دون تكرار ودون إعادة رسم كل شيء. الفوضى في إدارة الحالة هي السبب الأول في التطبيقات التي «تعمل أحياناً».

هناك نوعان يجب أن تفرق بينهما منذ اليوم الأول:

- **حالة محلية للـ widget**: هل هذا الحقل مفتوح؟ أي تبويب ظاهر؟ لا يهتم بها أحد غير هذا الـ widget. أدواتها: **StatefulWidget** مع **setState**.
- **حالة التطبيق**: العدد المحفوظ، قائمة الفواتير، المستخدم الحالي. تحتاجها شاشات متعددة ويجب أن تعيش خارج أي widget بعينه. أدواتها في هذا الدليل: صنف يرث **ChangeNotifier** (يحمل الحالة ويعلن تغييرها) مع **ListenableBuilder** (يعيد بناء جزء الواجهة المستمع). هذا ما يعلمه مسار Flutter الرسمي للمبتدئين، ولا يحتاج إلى حزمة خارجية.

```
class CounterController extends ChangeNotifier {
  int _count = 0;
  int get count => _count;

  void increment() {
    _count++;
    notifyListeners(); // الحالة تغيّرت
  }
}

// في الواجهة:
ListenableBuilder(
  listenable: controller,
  builder: (context, _) => Text('${controller.count}'),
)
```

خيارات إدارة الحالة الشائعة (حالة الحزم بتاريخ 2026-09-27)

الخيار	متى يناسب	ملاحظة
setState	حالة محلية لعنصر واحد	مدمج؛ ابدأ به دائماً للحالة المحلية
+ ChangeNotifier ListenableBuilder	تطبيقات صغيرة ومتوسطة؛ مشروعك الأول	مدمج؛ توصية مسار التعلم الرسمي
provider	حين تحتاج إلى تمرير الحالة عبر شجرة عميقة دون تمريرها يدوياً	حزمة Flutter Favorite؛ توصي بها وثائق Flutter لحقن التبعيات
flutter_riverpod	تطبيقات أكبر تريد أمان الأنواع وسهولة الاختبار	الإصدار 3 حالي ونشط
flutter_bloc	فرق تفضل فصلاً صارماً بين الأحداث والحالات	Flutter Favorite؛ كود أكثر رسمية

القرار: ابدأ بـ `ChangeNotifier`. حين تجد نفسك تمرر المتحكم عبر خمس طبقات من الـ `widgets`، أضف `provider`. لا تختَر Riverpod أو Bloc في مشروعك الأول لأن مقطع فيديو قال إنها «احترافية»؛ ستتعلم الأداة ولن تتعلم المفهوم. المرجع:

<https://docs.flutter.dev/data-and-backend/state-mgmt/simple>

تصميم تجربة المستخدم وإمكانية الوصول

لماذا: تجربة المستخدم (UX) ليست الجمال، بل أن يحقق المستخدم هدفه بأقل جهد وأقل أخطاء. وإمكانية الوصول (accessibility) تعني أن يستطيع استخدام تطبيقك من يرى بصعوبة أو لا يرى، ومن يكبر خط النظام، ومن يستخدم يدًا واحدة. هذه ليست إضافة أخلاقية فقط؛ المتاجر تفحصها (تقرير ما قبل الإطلاق في Play يتضمن فحص إمكانية الوصول)، والمستخدم الذي يكبر الخط أكثر مما تظن.

قواعد عملية تكفي لمشروعك الأول:

- **هدف اللمس:** `dp 48×48` على الأقل حسب إرشادات Android، و `44×44` نقطة حسب إرشادات Apple. زر العد في مثالنا يملأ عرض الشاشة عمداً.
- **التباين:** نسبة 4.5:1 على الأقل للنص العادي (معيّار WCAG الذي تتبناه المنصتان).
- **لا تعتمد على اللون وحده:** عند بلوغ الهدف غيرنا اللون وأضفنا اهتزازاً؛ أضف نصاً أو أيقونة أيضاً لمن لا يميز الألوان.
- **حجم الخط الديناميكي:** اختبر تطبيقك بأكبر حجم خط في إعدادات النظام. النصوص يجب أن تلتف لا أن تقص أو تخرج من الشاشة.
- **قارئ الشاشة:** كل زر أيقوني يحتاج إلى `tooltip` أو `Semantics(label:)` حتى يقرأه TalkBack أو VoiceOver. جرب تطبيقك بعينين مغمضتين مع TalkBack مرة واحدة على الأقل.
- **اتبع لغة المنصة:** 3 Material على Android (الافتراضي في Flutter منذ 3.16)، وإرشادات Apple (HIG) على iOS. المستخدم يتوقع أن يكون زر الرجوع حيث اعتاده.

المراجع

<https://developer.android.com/guide/topics/ui/accessibility/apps>
<https://developer.apple.com/design/human-interface-guidelines/accessibility>
<https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html>
<https://m3.material.io/>

المحطة الثالثة: البنية والبيانات

بنية التطبيق: لماذا نفصل الواجهة عن المنطق عن البيانات؟

لماذا: التطبيق الذي يضع كل شيء في ملف `main.dart` يعمل في الأسبوع الأول ويصبح مستحيل التعديل في الشهر الثاني. البنية (architecture) هي ببساطة اتفاق على «أي نوع من الكود يعيش أين». الاتفاق الذي توصي به وثائق Flutter الرسمية هو نمط **MVVM** بثلاث طبقات:

الطبقة	ما تحتويه	لا يجوز أن	في مشروع عداد الأذكار
الواجهة (View)	Widgets ترسم وتستقبل اللمس	تحتوي منطق عمل أو تلمس التخزين مباشرة	<code>counter_screen.dart</code>
نموذج العرض (ViewModel)	حالة الشاشة والعمليات عليها؛ يعلن التغيير	يعرف شيئاً عن الـ Widgets	<code>CounterController</code>
البيانات (Repository / Service)	القراءة والكتابة من التخزين أو الشبكة	تعرف شيئاً عن الشاشات وتنفيذه	<code>CounterStore</code>

الفائدة العملية الفورية: لأن `CounterController` لا يعرف الـ `widgets`، اختبرناه في الفصل التاسع بأربعة اختبارات تعمل في أجزاء من الثانية دون تشغيل واجهة. ولأن `CounterStore` واجهة مجردة، استبدلناه بنسخة في الذاكرة داخل الاختبار. هذا هو معنى «بنية قابلة للاختبار».

مصطلح: حقن التبعيات (Dependency Injection)

بدل أن ينشئ الصنف الأشياء التي يحتاجها بنفسه (= `final store`)
 (`SharedPrefsCounterStore()` داخله)، يستقبلها من الخارج عبر المنشئ. الاسم مخيف والفكرة بسيطة: «لا تصنع أدواتك، خذها ممن يناديك». الفائدة: يمكن تمرير أداة وهمية في الاختبار وأداة حقيقية في التطبيق.

القرار: لا تنشئ عشرة مجلدات في مشروع من ثلاث شاشات. بنية مشروع صغير:

```
lib/
  main.dart           // نقطة الدخول: يركّب التبعيات ويشغل التطبيق
  models/             // أصناف بيانات نقية (Dhikr, Invoice)
  data/               // التخزين والشبكة (CounterStore, InvoiceRepository)
  state/              // المتحكمات / نماذج العرض (ChangeNotifier)
  screens/            // شاشة لكل ملف
  widgets/            // قطع واجهة مشتركة
  test/               // lib/ يعكس بنية
```

المرجع:

<https://docs.flutter.dev/app-architecture/guide>

النماذج والتحقق من المدخلات

لماذا: كل بيانات تأتي من الخارج (حقل نصي، ملف، شبكة) مشبوهة حتى تثبت سلامتها. النموذج (Model) يمثل البيانات الصحيحة، والتحقق (validation) هو الحارس على بابه. القاعدة: تحقق مبكرًا عند الحدود، وثق بالبيانات في الداخل.

```
/// نتيجة التحقق: إما نجاح بقيمة أو فشل برسالة قابلة للعرض.
String? validateWarrantyMonths(String input) {
  if (input.trim().isEmpty) return null;           // اختياري: فارغ مقبول
  final n = int.tryParse(input.trim());
  if (n == null) return 'أدخل عددًا صحيحًا';
  if (n < 0 || n > 240) return 'المدة يجب أن تكون بين 0 و240 شهرًا';
  return null;                                     // null = خطأ
}

// TextFormField يُستخدم مباشرة مع Flutter في
TextFormField(
  decoration: const InputDecoration(labelText: 'مدة الضمان بالأشهر'),
  keyboardType: TextInputType.number,
  validator: validateWarrantyMonths,
)
```

ملاحظات خاصة بالعربية: المستخدم قد يكتب الأرقام بالهندية «12» بدل «12»، وقد تضاف مسافات غير مرئية من لوحة المفاتيح؛ طبع المدخلات قبل التحقق. والتاريخ يعرض بالتقويم الميلادي أو الهجري حسب المستخدم، لكنه يخزن دائمًا بصيغة واحدة (ISO 8601 بالتوقيت العالمي).

التخزين المحلي وقواعد البيانات والعلاقات

لماذا: ما لم يحفظ على القرص يختفي عند إغلاق التطبيق. لكن «الحفظ» طيف واسع، والقرار بين خياراته من أهم القرارات المبكرة:

الخيار	يناسب	لا يناسب	الحزمة الشائعة (2026-09-27)
مفاتيح وقيم بسيطة	إعدادات، آخر ذكر مختار، عداد صغير، «هل رأى المستخدم شاشة الترحيب؟»	قوائم كبيرة، بحث، علاقات	<code>shared_preferences</code> (Flutter Favorite)، من فريق Flutter
ملفات	صور، PDF، نسخ احتياطية، ملفات تصدير	بيانات تستعلم أو تصفى	<code>dart:io + path_provider</code>
قاعدة بيانات علائقية (SQLite)	مئات إلى ملايين السجلات، بحث وترتيب وتصفية، علاقات (فاتورة ← صور، فاتورة ← فئة)	ثلاث قيم بسيطة	<code>sqlite</code> (Favorite) للتحكم المباشر، أو <code>drift</code> (Favorite) لأمان الأنواع والاستعلامات المولدة
قواعد كائنات محلية NoSQL	كائنات سريعة بلا مخطط صارم	مشاريع تريد استقراراً طويلاً	<code>isar</code> و <code>hive</code> شائعتان، لكن إصداريهما المستقرين لم يحدثا منذ 2022 و2023: <code>hive_ce</code> استمرار مجتمعي نشط. توخ الحذر

العلاقات: حين تقول «فاتورة واحدة لها عدة صور»، فهذه علاقة «واحد إلى كثير» تمثل في SQLite بجدول `images` فيه عمود `invoice_id` يشير إلى الفاتورة. وحين تقول «فاتورة لها عدة وسوم، والوسم على عدة فواتير»، فهذه «كثير إلى كثير» وتحتاج إلى جدول وسيط. تعلم هذين النمطين ومعهما 80% من نماذج البيانات في التطبيقات الصغيرة.

قرار غالباً ما يُهمل: ترحيل المخطط (migration). في الإصدار 2 تضيف عمود «الفئة». المستخدم الذي يحدث من الإصدار 1 لديه قاعدة بيانات بلا هذا العمود. قاعدة البيانات تحتاج إلى رقم إصدار ودالة ترقية تضيف العمود دون مسح بياناته. تجاهل هذا يعني تقييمات نجمة واحدة بعنوان «التحديث مسح كل فواتيري».

نصيحة شخصية

ابدأ بـ `shared_preferences` لأي شيء يمكن تمثيله في أقل من بضعة كيلوبايتات (كما فعلنا في العداد، وخبزنا الخريطة كنص JSON واحد). انتقل إلى SQLite حين تحتاج إلى بحث أو ترتيب أو علاقات. ولا تلمس NoSQL المحلي في مشروعك الأول.

واجهات البرمجة والتعامل مع الخادم عند الحاجة

لماذا: حين يحتاج تطبيقك إلى بيانات من الخارج (مواقيت صلاة، أسعار، مزامنة)، يتحدث مع خادم عبر واجهة برمجة (API): يرسل طلب HTTP إلى عنوان ويستقبل ردًا غالبًا بصيغة JSON. المفاهيم التي يجب أن تعرفها: طرق الطلب (GET للقراءة، POST للإنشاء)، رموز الحالة (200 نجاح، 404 غير موجود، 401 غير مصرح، 500 خطأ في الخادم)، وتحويل JSON إلى نموذج Dart.

```
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<PrayerTimes> fetchPrayerTimes(double lat, double lng) async {
  final uri = Uri.https('api.example.com', '/v1/timings', {'lat': '$lat', 'lng': '$lng'});
  final res = await http.get(uri).timeout(const Duration(seconds: 10));
  if (res.statusCode != 200) {
    throw HttpException('server returned ${res.statusCode}');
  }
  final json = jsonDecode(res.body) as Map<String, dynamic>;
  return PrayerTimes.fromJson(json); // التحقق من الحقول داخل
}
```

القرارات: (1) مهلة زمنية دائمًا؛ الشبكة تفشل بصمت. (2) ماذا تعرض حين يفشل الطلب؟ آخر بيانات محفوظة مع تنبيه، لا شاشة خطأ فارغة. (3) لا تضع مفاتيح API سرية داخل التطبيق؛ أي شيء داخل ملف التطبيق يمكن استخراجه. المفتاح السري يعيش على خادمك أنت، والتطبيق يتحدث مع خادمك. (4) استخدم HTTPS فقط؛ المنصتان ترفضان HTTP العادي افتراضيًا.

المصادقة وإدارة الحسابات، إن احتجت إليها

لماذا: الحساب ضروري فقط حين تحتاج بيانات المستخدم إلى الانتقال بين أجهزة أو إلى الارتباط بهوية (دفع، محتوى خاص). تطبيقاينا الثلاثة لا تحتاج حسابًا في نسخها الأولى. حين تحتاجه، لا تبني نظام مصادقة بنفسك أبدًا في مشروعك الأول؛ استخدم خدمة (Firebase Authentication، Supabase، أو تسجيل الدخول بـ Google / Apple).

ما يجب أن تعرفه حتى لو استخدمت خدمة: الفرق بين المصادقة (من أنت؟) والتفويض (ما المسموح لك؟)، وأن الرمز (token) الذي يثبت هويتك يجب أن يخزن في التخزين الآمن للنظام (`flutter_secure_storage`) وأمثالها) لا في `shared_preferences`، وأن كلا المتجرين يشترط: إن سمحت بإنشاء حساب فيجب أن تسمح بحذفه من داخل التطبيق، وGoogle تطلب إضافة رابط ويب لطلب الحذف. وApple تشترط تقديم «تسجيل الدخول بـ Apple» إن قدمت تسجيل الدخول بخدمات طرف ثالث أخرى (راجع البند 4.8 من الإرشادات).

متطلب رسمي

<https://support.google.com/googleplay/android-developer/answer/13327111>

<https://developer.apple.com/app-store/review/guidelines/#5.1.1>

المحطة الرابعة: حواف المنصة

«الحواف» هي الأماكن التي يلمس فيها تطبيقك نظام التشغيل نفسه: الأنونات، الإشعارات، الملفات، اللغة. هنا تختلف Android عن iOS أكثر ما تختلف، وهنا تظهر معظم أخطاء iOS التي لن تراها على محاكي Android. اقرأ هذه المحطة مرتين إن كنت تنشر على المنصتين دون Mac.

الأنونات والخصوصية وحماية البيانات

لماذا: النظام لا يعطي تطبيقك الكاميرا أو الموقع أو الإشعارات إلا بإذن صريح من المستخدم، ويمكنه سحب الإذن في أي وقت. والمتاجر تطلب منك أن تعلن مسبقاً ما البيانات التي تجمعها ولماذا، وتعاقب من يكذب.

ثلاث قواعد للأنونات:

1. اطلب الإذن في لحظة الحاجة، لا عند التشغيل. اطلب الكاميرا حين يضغط المستخدم «التقط صورة الفاتورة»، مسبقاً بجملة تشرح لماذا. نسبة القبول تتضاعف.
2. صمّم مسار الرفض. رفض الكاميرا لا يعني نهاية العالم: اسمح باختيار صورة من المعرض أو بالإدخال دون صورة. رفض الإشعارات يعني أن تعرض التواريخ القادمة داخل التطبيق بوضوح.
3. أعلن الإذن في ملفات المنصة: `AndroidManifest.xml` على Android، و `Info.plist` على iOS مع نص شرح لكل إذن (مثل `NSCameraUsageDescription`). نسيان نص الشرح على iOS يسقط التطبيق عند طلب الإذن. هذا خطأ لن تراه على Android أبداً.

الخصوصية: كلا المتجرين يطلب نموذجًا يعلن البيانات المجموعة: **Data safety** في Play (مطلوب لكل تطبيق منشور حتى في مسارات الاختبار المغلق والمفتوح) و**App privacy details** في App Store Connect. وApple تطلب منذ مايو 2024 ملف **Privacy Manifest** يعلن أسباب استخدام بعض الواجهات؛ الحزم الشائعة توفره، لكن تأكد. وسياسة خصوصية على رابط عام مطلوبة حتى لو كان تطبيقك محليًا بالكامل، وعندها تكتب ببساطة أن البيانات لا تغادر الجهاز.

حماية البيانات على الجهاز: البيانات العادية في مجلد التطبيق محمية بعزل النظام. الأسرار (رموز الدخول، مفاتيح) توضع في التخزين الآمن (Keystore على Android وKeychain على iOS عبر حزمة مثل flutter_secure_storage). ولا تسجل بيانات شخصية في السجلات (logs) لأنها قد ترفع إلى أدوات تتبع الأعطال.

المراجع

<https://support.google.com/googleplay/android-developer/answer/10787469>

<https://developer.apple.com/app-store/app-privacy-details/>

<https://developer.apple.com/documentation/bundleresources/privacy-manifest-files>

<https://mas.owasp.org/MASVS/>

الإشعارات المحلية والإشعارات المرسلة من خادم

لماذا: الإشعار هو الطريقة الوحيدة التي يتحدث بها تطبيقك مع المستخدم وهو مغلق. لكن هناك نوعين مختلفين تمامًا في البنية والتكلفة، والخلط بينهما يقود إلى بناء خادم لا حاجة إليه:

إشعار محلي (Local)	إشعار من خادم (Push / Remote)
من يجدوله؟ التطبيق نفسه على الجهاز	خادم بعيد يرسل إلى خدمة المنصة (FCM لـ Android، APNs لـ iOS) التي توصله للجهاز
يحتاج إلى إنترنت؟ لا	نعم، ويحتاج إلى خادم يعرف «رمز الجهاز»
مثال «ينتهي ضمان الغسالة بعد أسبوع»، «حان وقت أذكار المساء»	«وصلتك رسالة جديدة»، «تم شحن طلبك»
الحزمة الشائعة	الحزمة الشائعة
flutter_local_notifications (Flutter Favorite)	firebase_messaging (Flutter Favorite) APNs و FCM

إشعار محلي (Local)	إشعار من خادم (Push / Remote)
تعيين iOS	مفتاح APNs يرفع إلى Firebase، وإعدادات قدرات (capabilities) في مشروع iOS، ولا يعمل على المحاكى إلا بطرق خاصة

القرار: إن كان الحدث معروفًا مسبقًا على الجهاز (تاريخ، وقت يومي)، فالإشعار المحلي يكفي ولا تحتاج إلى خادم. اذهب إلى Push فقط حين يحدث الشيء خارج الجهاز. وفي الحالتين: اطلب إذن الإشعارات في وقته (Android 13) فأحدث يتطلب إذنًا صريحًا مثل iOS)، واعلم أن بعض أجهزة Android تؤخر أو تقتل الإشعارات المجدولة توفيرًا للطاقة، فلا تعد المستخدم بدقة الثانية.

الصور والملفات والاستيراد والتصدير والنسخ الاحتياطي

لماذا: حافظلة الفواتير بلا صور لا معنى لها، وأي تطبيق يحفظ بيانات مهمة يسأل مستخدمه يومًا: «كيف أنقلها إلى هاتفي الجديد؟». إن لم يكن لديك جواب، فقدت المستخدم وبياناته معًا.

- **التقاط واختيار الصور:** `image_picker` (من فريق Flutter) يفتح الكاميرا أو المعرض على المنصتين. اضغط الصورة قبل حفظها؛ صورة هاتف حديث قد تتجاوز 5 MB ولا تحتاج فاتورة إلى أكثر من بضع مئات من الكيلوبايتات.
- **أين تحفظ؟** في مجلد مستندات التطبيق (`path_provider`) يعطيك المسار الصحيح على كل منصة). لا تحفظ المسار المطلق في قاعدة البيانات؛ على iOS يتغير مسار حاوية التطبيق بين التحديثات. احفظ اسم الملف فقط وركب المسار عند القراءة.
- **التصدير:** ملف واحد (ZIP يحتوي JSON + الصور، أو CSV للبيانات الجدولية) يشاركه المستخدم عبر ورقة المشاركة (`share_plus`) إلى Drive أو البريد أو الملفات. هذا أبسط نسخ احتياطي ولا يحتاج إلى خادم ولا حساب.
- **الاستيراد:** `file_picker` لاختيار الملف، ثم تحقق من كل شيء قبل الإدخال: هل هو ملفنا؟ أي إصدار؟ هل الحقوق سليمة؟ الملف المستورد هو مدخل خارجي مثل أي حقل نصي.
- **النسخ الاحتياطي التلقائي للنظام:** Android يعمل نسخًا احتياطيًا لبيانات التطبيق إلى Google Drive افتراضيًا (Auto Backup)، وiOS إلى iCloud ضمن نسخة الجهاز. مفيد، لكنه ليس بديلًا عن التصدير اليدوي لأن المستخدم لا يتحكم فيه ولا يستطيع نقله بين المنصتين.

تعدد اللغات، والعربية، واتجاه العرض من اليمين إلى اليسار

لماذا: حتى لو كان تطبيقك عربيًا فقط، ستحتاج إلى آلية الترجمة لأن نصوص عناصر النظام الجاهزة (أزرار الحوارات، منتقي التاريخ، شريط البحث) تأتي من Flutter بلغة التطبيق المعلنة. وإن أعلنت العربية، ينقلب اتجاه الواجهة كلها تلقائيًا: القوائم تبدأ من اليمين، زر الرجوع على اليمين، والأسهم تنعكس.

الأدوات: حزمة `flutter_localizations` (مدمجة في SDK) تحتوي ترجمات عناصر Material للعربية، وحزمة `intl` لتنسيق الأرقام والتواريخ، وملفات `ARB` لنصوصك أنت مع توليد كود آمن الأنواع عبر `l10n.yaml`. الفصل التاسع يطبق الحد الأدنى.

قواعد RTL التي يخطئ فيها الجميع:

- استخدم `EdgeInsetsDirectional` و `AlignmentDirectional` و `start/end` بدل `left/right`. الهامش «الأيسر» في العربية يجب أن يكون الأيمن، والـ `Directional` يفعل ذلك تلقائيًا.
- الأيقونات ذات الاتجاه (سهم رجوع، سهم «التالي») يجب أن تنعكس؛ Flutter يعكس معظم أيقونات Material تلقائيًا، وما لا ينعكس تلفه بـ `Directionality` أو تختار الأيقونة المناسبة.
- الأرقام: هل تعرض 33 أم 33؟ قرار تصميمي؛ `intl` يعطيك الأرقام الهندية بلغة `ar` والغربية بـ `ar_EG`... تحقق من الناتج على الجهاز. الكثير من المستخدمين العرب يفضلون الأرقام الغربية في التطبيقات.
- النص المختلط (عربي + كلمة إنجليزية + رقم) قد يظهر بترتيب غريب؛ اختبر كل شاشة فيها كود منتج أو بريد إلكتروني أو رقم هاتف.
- اختبر مرة بالعربية ومرة بالإنجليزية إن دعمتهما؛ تخطيط يعمل في واحدة قد ينكسر في الأخرى.

المراجع

<https://docs.flutter.dev/ui/accessibility-and-internationalization/internationalization>

<https://api.flutter.dev/flutter/painting/EdgeInsetsDirectional-class.html>

المحطة الخامسة: الجودة والعمر الطويل

الاختبارات: ثلاثة مستويات

لماذا: الاختبار اليدوي لا يتوسع. بعد الميزة العاشرة لن تعيد تجربة التسع السابقة مع كل تعديل، وسينكسر شيء دون أن تعلم. الاختبار الآلي يعيد التجربة في ثوان في كل مرة. ووثائق Flutter تقسمه إلى ثلاثة:

المستوى	يختبر	سرعته	مثال من مشروعنا
اختبار وحدة	دالة أو صنفًا واحدًا بلا واجهة	أجزاء من الثانية	<code>CounterController</code> يزيد ويحفظ ويستعيد
اختبار Widget	عنصر واجهة أو شاشة في بيئة محاكاة خفيفة	ثوان	النقر على «سبح» يغير الرقم المعروض من 0 إلى 1
اختبار تكامل	التطبيق كاملاً على جهاز أو محاكي حقيقي	دقائق	فتح التطبيق، اختيار ذكر من القائمة، العد، إغلاق وإعادة فتح، التأكد من الاستعادة

القرار: ما الذي يستحق الاختبار؟ ليس كل شيء. اختبر: المنطق الذي يتخذ قرارات (الضمان سار أم لا)، وكل ما يلمس البيانات المحفوظة (الحفظ والاستعادة والترحيل)، والحالات غير المعتادة التي كتبتها في نموذج المتطلبات (البيانات التالفة). لا تختبر أن Flutter يرسم زراً. اختبار وحدة سريع لكل قاعدة عمل + اختبار widget واحد لكل شاشة رئيسية هو حد معقول لمشروع أول. المرجع:

<https://docs.flutter.dev/testing/overview>

تتبع الأخطاء وقراءة سجلات التشغيل

لماذا: على جهازك ترى الخطأ في وحدة التحكم. على جهاز مستخدم في مدينة أخرى لا ترى شيئاً إلا تقييم نجمة واحدة: «ينهار». أداة تتبع الأعطال (crash reporting) ترسل إليك تفاصيل كل انهيار: الجهاز، الإصدار، وتسلسل الاستدعاءات (stack trace). الخياران الشائعان: `firebase_crashlytics` و `sentry_flutter`، وكلاهما Flutter Favorite. أضف أحدهما قبل أول إصدار تجريبي، واذكره في سياسة الخصوصية ونماذج المتجر.

قراءة السجل مهارة: حين ينهار تطبيقك أثناء التطوير، لا تقرأ آخر سطر فقط. ابحث عن أول سطر يذكر ملفاً من ملفاتك أنت (`package:dhikr_counter/...`)؛ ما قبله من Flutter الداخلي غالباً ليس السبب. انسخ تسلسل الاستدعاءات كاملاً حين تطلب المساعدة. واستخدم `debugPrint` أو حزمة `logging` بمستويات (معلومة، تحذير، خطأ) بدل `print` المتناثر الذي ينسى في كود الإنتاج.

مع الذكاء الاصطناعي

تسلسل الاستدعاءات هو أفضل مدخل تعطيه للمساعد على الإطلاق. الصق: رسالة الخطأ كاملة، تسلسل الاستدعاءات كاملاً، والملف المذكور في أول سطر يخصصك، وما فعلته قبل الانهيار مباشرة. ثم اسأل: «ما السبب الأرجح، وكيف أتأكد قبل أن أعدل؟». الجملة الأخيرة مهمة: تريد فرضية قابلة للتحقق، لا تعديلاً عشوائياً.

الأداء واستهلاك الذاكرة والبطارية والعمل دون اتصال

لماذا: المستخدم لا يقرأ كودك؛ يشعر بتطبيقك. الشاشة التي تتلثم أثناء التمرير، والهاتف الذي يسخن، والبطارية التي تنفذ، كلها أسباب حذف. الأرقام التي تهم: 16 مللي ثانية لكل إطار (60 إطاراً في الثانية)؛ أي عمل أطول على الخيط الرئيسي يسقط إطارات.

- الأداء: استخدم `ListView.builder` للقوائم الطويلة (يبني العناصر الظاهرة فقط)، وضع `const` على الـ `widgets` الثابتة، وانقل الحساب الثقيل (ضغط صورة، تحليل ملف كبير) إلى `compute()`. وقس دائماً في وضع `profile` لا `debug`؛ وضع `debug` أبطأ بمرات ولا يمثل الواقع.
- الذاكرة: الصور هي المستهلك الأول. اعرض نسخة مصغرة في القائمة وحمل الأصل في شاشة التفاصيل فقط (`cacheWidth` في `Image`). تسريبات الذاكرة الشائعة: `StreamSubscription` أو `AnimationController` لم يغلق في `dispose()`.
- البطارية: الموقع المستمر، والاستطلاع الدوري للشبكة، والرسوم المتحركة التي لا تتوقف، والعمل في الخلفية. اسأل مع كل ميزة: هل تعمل حين التطبيق مغلق؟ إن كان الجواب نعم فالثمن بطارية، والنظامان يقيدان ذلك بشدة.
- العمل دون اتصال: صمم للانقطاع من البداية. الاستراتيجية الأبسط: المحلي أولاً؛ كل قراءة من التخزين المحلي، والشبكة تحدّثه في الخلفية. تطبيقاتنا الثلاثة محلية أصلاً فتعمل دون اتصال بطبيعتها، وهذه ميزة تنافسية حقيقية لا تحصل عليها التطبيقات التي تبدأ بالخادم.

أداة القياس الرسمية: **Flutter DevTools** (مدمجة) بتبويبات الأداء والذاكرة والشبكة. المرجع:

<https://docs.flutter.dev/tools/devtools>

إدارة الإصدارات والتغييرات بعد النشر

لماذا: بعد النشر تصبح كل تغييرة قراءاً يمس مستخدمين حقيقيين ببيانات حقيقية. لا تستطيع «حذف الجدول وإعادة إنشائه». والتحديث الذي يكسر شيئاً يصل إلى الجميع.

- رقم لكل إصدار وسجل تغييرات: ملف `CHANGELOG.md` يذكر ما تغير في كل إصدار. المستخدمون يقرؤونه في المتجر، وأنت تقرأه بعد ستة أشهر حين تسأل «متى كسرنا هذا؟».
- وسم في Git لكل إصدار منشور (`git tag v1.2.0`) حتى تعرف بالضبط أي كود يعمل عند المستخدمين.
- التوافق مع البيانات القديمة: كل تغيير في صيغة التخزين يحتاج إلى ترحيل. اختبر التحديث من الإصدار السابق فعلاً: ثبت القديم، أدخل بيانات، ثبت الجديد فوقه، تأكد أن البيانات باقية.
- الطرح التدريجي: المتجران يسمحان بنشر التحديث لنسبة صغيرة أولاً. استخدمه؛ خطأ يصيب 5% من المستخدمين أرحم من خطأ يصيب 100%.
- حزم تعتمد عليها تتغير: `flutter pub outdated` يعرض الحزم القديمة. حدث دورياً وفي إصدار مخصص، لا مع ميزة جديدة، حتى تعرف مصدر أي كسر.

خريطة المفاهيم في مشروعك الأول

ليس عليك إتقان كل ما سبق قبل أن تبدأ. هذا ترتيب واقعي لما ستحتاجه ومتى:

ستحتاجه في	المفاهيم
الأسبوع الأول	المتغيرات والأنواع والدوال والأصناف والقوائم؛ شجرة الـ <code>widgets</code> ؛ <code>setState</code>
الأسبوع الثاني والثالث	غير المتزامن: <code>ChangeNotifier</code> ؛ التنقل؛ التخزين البسيط؛ فصل الطبقات الثلاث: Git اليومي
الأسبوع الرابع إلى السادس	النماذج والتحقق: <code>SQLite</code> إن احتجت؛ الاختبارات؛ الأنونات؛ <code>RTL</code> والترجمة
قبل الإصدار التجريبي	تتبع الأعطال؛ الأداء في وضع <code>profile</code> ؛ الخصوصية ونماذج المتجر؛ أرقام الإصدارات
بعد النشر	الترجيلات؛ الطرح التدريجي؛ سجل التغييرات؛ الإشعارات و <code>APIs</code> حين تحتاجها الفكرة

خلاصة الفصل

- اللغة: أنواع ثابتة، قابلية فراغ معلنة، نماذج غير قابلة للتغيير، دوال نقية تستقبل تبعياتها.

- الواجهة: دالة من الحالة؛ فرق بين الحالة المحلية وحالة التطبيق؛ ابدأ بـ `ChangeNotifier`.
- البنية: ثلاث طبقات (واجهة، نموذج عرض، بيانات) تجعل الكود قابلاً للاختبار والتعديل.
- البيانات: تحقق عند الحدود؛ اختر التخزين بحسب الحجم والاستعلامات؛ خطط للترحيل.
- الحواف: اطلب الأذونات في وقتها وصمم الرفض؛ الإشعار المحلي لا يحتاج خادمًا؛ خطط للتصدير؛ استخدم `Directional` للعربية.
- الجودة: اختبر المنطق والبيانات لا الرسم؛ تتبع الأعطال قبل أول إصدار تجريبي؛ قس في وضع `profile`؛ وسم كل إصدار.

CHAPTER 08

استخدام الذكاء الاصطناعي باحتراف

أدوات الذكاء الاصطناعي (ChatGPT و Claude و Gemini و Copilot وأمثالها) غيرت طريقة تعلم البرمجة وممارستها. الطالب اليوم يستطيع أن يحصل في دقيقة على كود كان يحتاج إلى ساعة بحث. لكنها غيرت أيضًا طبيعة الخطأ الشائع: لم يعد «لا أعرف كيف أكتب هذا»، بل «كتبته الأداة ولا أعرف ماذا يفعل». هذا الفصل يعلمك أن تستفيد من السرعة دون أن تفقد الفهم، لأن الفهم هو ما يطلب منك حين ينكسر التطبيق في الساعة الثانية صباحًا قبل التسليم، والأداة لا تعرف تطبيقك كما تعرفه أنت.

المبدأ الحاكم: أنت المهندس، والأداة مساعد سريع جدًا

فكر في الأداة كزميل واسع الاطلاع، سريع الكتابة، لا يتعب، لكنه: لا يرى شاشتك، ولا يعرف قراراتك السابقة إلا إذا أخبرته، ويثق بنفسه أكثر مما ينبغي أحيانًا، وقد يقترح واجهة برمجة ألغيت قبل عامين. زميل كهذا لا تعطيه المشروع كاملاً وتنام؛ تعطيه مهمة محددة، وتراجع ما يعيده، وتشغله، وتسأله لماذا فعل ما فعل. ثلاث قواعد لا تتنازل عنها:

1. لا تقبل كودًا لا تستطيع شرحه لزميل في دقيقتين. إن لم تستطع، اطلب من الأداة شرحه سطرًا سطرًا، ثم أعد الشرح بكلماتك.
2. كل تغيير يُشغّل ويُختبر قبل التغيير التالي. لا تكس خمس اقتراحات ثم تكتشف أن التطبيق لا يترجم.
3. لقطة Git قبل كل تغيير كبير حتى يكون التراجع بأمر واحد.

ما الذي تجيده الأداة في كل مرحلة، وكيف تطلبه؟

المهمة	ما تعطيه للأداة	ما تطلبه منها	ما تفعله أنت بالنتيجة
تحويل الفكرة إلى متطلبات	فقرة الفكرة، المستخدم، القيود (وقت، أجهزة)	أسئلة توضيحية، ثم نموذج المتطلبات ذا الأقسام السبعة مع تمييز الافتراضات	تقرر ما يدخل الـ MVP وتحذف ما توسعت فيه الأداة

المهمة	ما تعطيه للأداة	ما تطلبه منها	ما تفعله أنت بالنتيجة
تخطيط الشاشات والتدفق	المتطلبات	قائمة الشاشات، عناصر كل شاشة، الانتقالات، والحالات الثلاث (فارغة/ تحميل/ خطأ)	ترسمها على ورق وتكتشف الفجوات
اقتراح بنية المشروع	المتطلبات + الإطار وإصداره + قيود الحزم	شجرة مجلدات، مسؤولية كل طبقة، وما الذي لا يجوز أن تفعله كل طبقة	تبسطها لحجم مشروعك
كتابة قطعة كود	الملف ذو الصلة، الواجهات التي يجب أن تتوافق معها، معيار النجاح	قطعة واحدة صغيرة (دالة، صنف، widget) مع شرح القرارات	تقرأ، تشغل، تختبر، وتحفظ لفظة
فهم رسالة خطأ	الرسالة كاملة، تسلسل الاستدعاءات، الملف المذكور، ما فعلته قبلها	السبب الأرجح وطريقة التأكد قبل التعديل	تتحقق من الفرضية أولاً
مراجعة الكود	الملف أو الـ diff	قائمة ملاحظات مرتبة بالخطورة: أخطاء، ثم أمان، ثم أداء، ثم أسلوب	تصلح ما تقتنع به وتناقش ما لا تقتنع به
اقتراح اختبارات	الصنف المراد اختباره + الحالات غير المعتادة من المتطلبات	قائمة حالات اختبار بالأولوية، ثم كود الاختبار لأهم ثلاث	تشغل الاختبارات وتؤكد أنها تفشل حين تكسر الكود عمداً
نصوص الواجهة والترجمة	سياق كل نص (أين يظهر، طول الحقل، نبرة التطبيق)	نصوص عربية قصيرة واضحة، والإنجليزية المقابلة، ملاحظات على النصوص الملتبسة	تراجع اللغة بنفسك؛ الأداة تخطئ في التذكير والتأنيث والهمزات
مواد النشر	وصف التطبيق، لقطات، ما يجمعه من بيانات	مسودة وصف المتجر، مسودة سياسة خصوصية، قائمة تحقق بمتطلبات المتجر مع روابط	تتحقق من كل رابط في الوثائق الرسمية الحالية؛ الأداة قد تذكر شروطاً قديمة

تشریح البرومبت الجيد

البرومبت الجيد ليس طويلاً، بل محدداً. أربعة عناصر تجعل الفرق بين جواب عام وجواب تستطيع استخدامه:

- **الهدف:** ما الذي تريده بالضبط؟ «أضف حفظاً محلياً للعداد» لا «حسن التطبيق».
- **السياق:** الإطار وإصداره (Flutter 3.47, Dart 3.13)، الملفات ذات الصلة، البنية المتبعة، القرارات السابقة.
- **القيود:** ما لا يجوز: «لا حزم جديدة»، «لا تغير توقيع الدالة X»، «لا تلمس ملفات الاختبار»، «الزم بالبنية الحالية».

- معيار النجاح: كيف سنعرف أنه نجح؟ `flutter analyze` بلا تحذيرات، والاختبارات الحالية تمر، والعدد يبقى بعد إغلاق التطبيق».

انتبه: «التطبيق لا يعمل» ليس وصفًا

قارن: «التطبيق لا يعمل» مقابل: «عند النقر على زر التصغير في `counter_screen.dart` يظهر الحوار، وعند تأكيد التصغير يبقى الرقم كما هو ولا يظهر خطأ في وحدة التحكم. الكود المرفق. Flutter 3.47. توقعت أن يعود الرقم إلى 0». الثاني يحل في دقيقة؛ الأول يقود إلى عشر رسائل من التخمين.

متى تقسم المهمة؟

قاعدة عملية: إن كان الجواب المتوقع أطول من شاشة واحدة من الكود، فالمهمة كبيرة جدًا. «ابن لي تطبيق الفواتير» ينتج كودًا كثيرًا لن تراجع له ولن يعمل من أول مرة ولن تعرف أين انكسر. قسمها كما قسمنا المشروع في الفصل التاسع: نموذج البيانات أولاً (واختبره)، ثم المستودع (واختبره)، ثم شاشة القائمة، ثم شاشة الإضافة. كل خطوة برومبت مستقل يحمل سياق ما قبله.

علامات أنك تحتاج إلى التقسيم: الأداة تكتب «...» أو «أكمل بقية الحالات بنفس الطريقة»، أو تنسى قيدًا ذكرته في البداية، أو يفشل الكود في الترجمة بأخطاء متعددة في ملفات مختلفة.

كيف تراجع تغييرًا قبل قبوله؟

1. اقرأ الـ `diff` لا الملف: ما الذي تغير بالضبط؟ إن لمست الأداة أسطرًا لم تطلب تعديلها، اسأل لماذا.
2. ابحث عن الافتراضات الصامتة: هل افترضت حزمة غير موجودة؟ إصدارًا مختلفًا؟ أن القيمة لا تكون فارغة أبدًا؟
3. شغل المحلل والاختبارات: `flutter analyze` ثم `flutter test`. تحذير جديد = سؤال جديد.
4. جرّب يدويًا الحالة غير المعتادة التي يخصصها التغيير، لا المسار السعيد فقط.
5. اطلب شرح القرارات: «لماذا اخترت X بدل Y؟ ما البديل ومتى يكون أفضل؟». إن لم يقنعك الجواب، فلا تقبل التغيير.

ما لا تشاركه مع الأداة أبدًا

- ملفات `keystore` وكلمات مرورها، ومفاتيح `p8`. الخاصة بـ App Store Connect، وملفات `key.properties`.
 - مفاتيح واجهات البرمجة السرية، وملفات إعدادات Firebase التي تحتوي معرفات خاصة إن كان مشروعك حساسًا.
 - بيانات مستخدمين حقيقيين (أسماء، أرقام، صور فواتير حقيقية). استخدم بيانات وهمية دائمًا في الأمثلة.
 - كود لا تملك حق مشاركته (مشروع شركة، مشروع زميل).
- حين تحتاج إلى مساعدة في ملف يحتوي سرًا، استبدل السر بـ `<REDACTED>` قبل اللصق. وراجع سياسة الأداة: كثير من الخدمات تستخدم المحادثات في التدريب ما لم تعطل ذلك.

مساعد المحرر مقابل مساعد المحادثة مقابل الوكيل

ستقابل ثلاثة أشكال، ولكل منها استخدام:

- **الإكمال داخل المحرر** (مثل Copilot): يقترح السطر التالي وأنت تكتب. ممتاز للكود المتكرر (`copyWith`، `fromJson`). خطره: تقبل بـ Tab دون قراءة.
- **المحادثة**: للشرح والتخطيط والتشخيص. أفضل مكان للتعلم لأنك تستطيع السؤال «لماذا».
- **الوكيل** (أداة تعدل ملفاتك وتشغل الأوامر بنفسها، مثل Claude Code أو Cursor agent): أسرع كثيرًا لكنه يحتاج إلى انضباط أكبر: مهام محددة، لقطة Git قبلها، ومراجعة الـ diff بعدها. لا تعط الوكيل مفاتيحك ولا صلاحية النشر.

نماذج برومبتات قابلة للنسخ

كل نموذج يحدد الهدف والسياق والقيود ومعياري النجاح. الأجزاء بين أقواس معقوفة تملؤها أنت. النصوص عربية، ويمكنك كتابتها بالإنجليزية إن كانت الأداة تعطيك نتائج أفضل بها: المهم البنية.

1. بدء مشروع

أنت مهندس Flutter خبير تساعد طالبًا يبني مشروعه الأول.

الهدف

اقترح بنية مشروع Flutter لتطبيق [عداد أنكار محلي بالكامل]، بحجم يناسب [شاشتين وبلا خادم].

السياق

[Flutter 3.47] و [Dart 3.13]. المتطلبات: [الصق نموذج المتطلبات ذا الأقسام السبعة]. أعمل على [Windows وهاتف Android]، ولا أملك Mac.

القيود

- بنية ثلاثية الطبقات (واجهة / نموذج عرض بـ ChangeNotifier / بيانات) بدون حزم إدارة حالة خارجية.
- الحزم المسموحة: shared_preferences فقط.
- لا تكتب كود الشاشات الآن؛ فقط الشجرة ومسؤولية كل ملف وواجهات الأصناف الرئيسية (الأسماء والتوقييع).

معيار النجاح

أستطيع أن أشرح لزميل دور كل مجلد في دقيقة، وكل صنف له مسؤولية واحدة، وطبقة البيانات قابلة للاستبدال بنسخة في الذاكرة للاختبار. في النهاية اذكر ثلاثة افتراضات افترضتها وسألتني عنها لو استطعت.

2. تصميم شاشة

الهدف

صمم شاشة [العد] لتطبيق [عداد الأناكر] كوصف نصي دقيق، ثم كود Flutter لها.

السياق

المستخدم [يستخدمها بيد واحدة ودون النظر إلى الشاشة]. البيانات المتاحة من المتحكم: [نص الذكر، العدد الحالي، الهدف، هل بلغ الهدف]. الشاشة عربية RTL. Material 3.

القيود

- زر العد هدف لمس كبير جدًا؛ التباين 4.5:1 على الأقل؛ بلوغ الهدف يشار إليه بلون ونص معًا لا باللون وحده.
- استخدم EdgeInsetsDirectional لا EdgeInsets حيث يهتم الاتجاه.
- لا منطق عمل داخل الـ widget؛ كل شيء عبر [CounterController] الذي واجهته: [الصق الصنف].
- صف الحالات الثلاث: التحميل، والعادية، وبلوغ الهدف.

معيار النجاح

flutter analyze بلا تحذيرات، الشاشة تعمل بأكبر حجم خط في إعدادات النظام دون قص نص، وكل زر أيقوني له tooltip.

3. إضافة ميزة

الهدف

أضف ميزة [حفظ العد محليًا واستعادته عند فتح التطبيق] إلى المشروع.

السياق

[3.47] Flutter. الملفات ذات الصلة: [الصق counter__controller.dart و counter__store.dart و main.dart].

البنية: [واجهة / ChangeNotifier / مخزن مجرد].

القيود

- عدل أقل عدد ممكن من الملفات واذكر كل ملف عدلته ولماذا.
- لا تغير تواريخ الدوال العامة الموجودة.
- البيانات المحفوظة قد تكون تالفة أو من إصدار قديم: يجب ألا ينهار التطبيق، بل يبدأ من الصفر.
- اشرح متى تحفظ (بعد كل نقرة؟ عند الخروج؟) ولماذا.

معيار النجاح

أغلق التطبيق تمامًا وأعيد فتحه فأجد العدد نفسه. الاختبارات الحالية تمر. أضف اختبار وحدة واحدًا على الأقل للاستعادة وآخر للبيانات التالفة.

4. إصلاح خطأ

الهدف

ساعدني في تشخيص هذا الخطأ قبل إصلاحه.

السياق

ما فعلت: [نقرت على زر التصفير ثم أكدت]. ما توقعت: [يعود الرقم إلى 0]. ما حدث: [بقي الرقم، ولا رسالة خطأ]. [3.47] Flutter، على [محاكي Pixel 8 API 36 وهاتف Samsung حقيقي، ويحدث في كليهما].

رسالة الخطأ وتسلسل الاستدعاءات كاملاً (إن وجد): [الصق].

الملف المذكور في أول سطر يخصني: [الصق الملف].

القيود

- اذكر الأسباب المحتملة مرتبة بالأرجحية، ولكل سبب طريقة أتتحقق بها منه (سطر تسجيل أو نقطة توقف أو اختبار) قبل أي تعديل.
- لا تقترح إعادة كتابة الملف؛ أصغر تعديل يصلح السبب الحقيقي.

معيار النجاح

أفهم السبب الجذري وأستطيع كتابة اختبار يفشل قبل الإصلاح وينجح بعده.

5. مراجعة كود

الهدف

راجع هذا الكود كمراجع صارم قبل دمج.

السياق

[الصق الملف أو الـ diff]. الغرض من التغيير: [إضافة قائمة الأذكار]. البنية المتبعة: [ثلاث طبقات]. Flutter [3.47].

القيود

- رتب الملاحظات: (1) أخطاء ستظهر للمستخدم، (2) أمان وخصوصية، (3) أداء، (4) قابلية الصيانة، (5) أسلوب. لا تخلط بينها.
- لكل ملاحظة: السطر، المشكلة، لماذا تهتم، والإصلاح المقترح في سطرين على الأكثر.
- إن لم تجد شيئاً في فئة فقل ذلك صراحةً بدل اختلاق ملاحظات.
- لا تعد كتابة الكود كاملاً.

معيار النجاح

أستطيع اتخاذ قرار مستقل بشأن كل ملاحظة: أقبلها أو أرفضها بسبب.

6. كتابة اختبار

الهدف

اقترح اختبارات لـ [CounterController] ثم اكتب أهم ثلاثة.

السياق

[الصق الصنف وواجهة CounterStore]. الحالات غير المعتادة من متطلباتي: [إغلاق مفاجئ، نقر سريع، بيانات تالفة، تجاوز الهدف].

القيود

- استخدم InMemoryCounterStore بدل التخزين الحقيقي.
- اختبارات وحدة بـ flutter_test فقط، بلا حزم إضافية.
- لكل اختبار اسم عربي يصف السلوك، وسطر واحد يشرح لماذا يهم.
- اذكر حالة واحدة على الأقل لا يمكن اختبارها كاختبار وحدة وتحتاج إلى اختبار widget أو تكامل، ولماذا.

معيار النجاح

flutter test يمر، وإذا عطلت السطر الذي يحفظ البيانات عمداً يفشل اختبار واحد على الأقل.

7. فحص جاهزية الإصدار

الهدف

أعد قائمة تحقق لجاهزية الإصدار التجريبي الأول لتطبيقي على Google Play (اختبار مغلق) وTestFlight.

السياق

التطبيق: [عداد أذكار محلي بالكامل، لا يجمع أي بيانات، لا حسابات، لا إنترنت]. [Flutter 3.47]. أبني iOS عبر Codemagic دون Mac. حساب Google Play شخصي أنشئ في [2026].

القيود

- افصل بوضوح بين: متطلب رسمي من المتجر (مع رابط الوثيقة الرسمية)، وممارسة موصى بها.
- لكل بند من المتطلبات الرسمية ضع تاريخ آخر مرة تحققت فيها منه، وقل صراحةً إن كانت معلوماتك قد تكون قديمة وما الذي يجب أن أراجع به بنفسي.
- غط: التوقيع، أرقام الإصدار، الأيقونات ولقطات الشاشة، سياسة الخصوصية ونموذج أمان البيانات وتفاصيل الخصوصية، الأنونات المعلنة، شرط المختبرين للحساب الشخصي، وحد SDK الأدنى الحالي.

معيار النجاح

قائمة يمكنني وضع علامة صح أمام كل بند فيها، وأعرف لكل بند أين أتحقق منه في الوثائق الرسمية.

نصيحة شخصية: اجعل البرومبت جزءًا من المشروع

أنشئ ملفًا باسم `AI_CONTEXT.md` في جذر المشروع يحتوي: وصف التطبيق في فقرة، البنية والقواعد (الطبقات الثلاث، الحزم المسموحة، RTL)، إصدار Flutter، والقرارات المهمة مع أسبابها. الصقه في بداية كل محادثة جديدة، أو وجه أداة الوكيل إليه. ستوفر عشرات الدقائق من إعادة الشرح، وستحصل على اقتراحات متسقة مع مشروعك بدل اقتراحات عامة.

خلاصة الفصل

- الأداة مساعد سريع جدًا، لا مهندسًا مسؤولًا. المسؤولية عن الفهم والاختبار والقرار تبقى معك.
- البرومبت الجيد: هدف محدد، سياق (إصدار وملفات وقرارات)، قيود صريحة، ومعيار نجاح قابل للتحقق.
- قسم المهام إلى قطع بحجم شاشة واحدة، وراجع الـ diff، وشغل المحلل والاختبارات، واطلب تفسير القرارات.
- الأسرار والبيانات الشخصية لا تدخل المحادثة. لقطة Git قبل كل تغيير كبير.

CHAPTER 09

المشروع التطبيقي: عداد أنكار خطوة خطوة

في هذا الفصل تبني تطبيقًا كاملاً بيدك عبر عشر مراحل، كل مرحلة تنتهي بشيء يعمل على شاشتك ولقطة Git. الكود الوارد هنا ليس مقتطفات نظرية: هو مشروع حقيقي جمع بـ Flutter 3.41 واجتاز `flutter analyze` بلا ملاحظات و7 اختبارات آلية أثناء إعداد هذا الدليل. المشروع الكامل مرفق مع الدليل في مجلد `example_dhikr_counter`. لكن لا تنسخه دفعة واحدة؛ اكتبه مرحلة مرحلة كما هو مرتب، فالهدف أن تفهم لماذا تنمو الفكرة بهذا الترتيب، لا أن تحصل على تطبيق.

قبل أن تبدأ

تأكد أن `flutter doctor` راض عن بيئتك (الفصل الخامس)، وأن هاتفك أو المحاكى يظهر في `flutter devices`. وأنشئ مستودع Git من اللحظة الأولى.

المرحلة 1: صياغة الفكرة والنطاق الأول

لن نكتب كودًا. نأخذ نموذج المتطلبات من الفصل السادس ونثبت النطاق الأول بجملة واحدة: «شاشة عدّ واحدة بذكر ثابت، زر كبير يزيد العدد، تصفير مع تأكيد، والعدد يبقى بعد إغلاق التطبيق». لا قائمة، لا إحصاءات، لا إشعارات. اكتب هذه الجملة في ملف `README.md` في مجلد المشروع مع قائمة «مؤجل» تضع فيها كل فكرة تخطر لك أثناء العمل بدل تنفيذها.

لماذا هذا النطاق؟ لأنه يجبرنا على المرور بالمفاهيم الأربعة الأساسية (واجهة، حالة، تخزين، حالات غير معتادة) في أصغر شكل ممكن. كل ما سيأتي بعده توسيع لا تأسيس.

المرحلة 2: رسم الشاشات وتحديد التنقل

على ورقة: مستطيل بنسبة الهاتف. في الأعلى شريط عنوان فيه «عداد الأذكار» ووزان أيقونيان (قائمة، تصفير). في الوسط نص الذكر، تحته الرقم كبيرًا جدًا، تحته «الهدف: 33». في الأسفل زر يملأ العرض بارتفاع كبير مكتوب عليه «سبح». مستطيل ثان لشاشة القائمة: قائمة عمودية، كل صف فيه نص الذكر وهدفه، وعلامة صح أمام المختار. سهم من زر القائمة إلى الشاشة الثانية، وسهم عودة من أي صف إلى الأولى.

ثلاثة قرارات تخرج من الرسم فوراً: (1) الزر في أسفل الشاشة حيث يصل الإبهام بيد واحدة؛ (2) الرقم هو أكبر عنصر لأنه ما يقرأ من بعيد؛ (3) شاشة القائمة تفتح فوق شاشة العد وتغلق بالاختيار، فلا نحتاج إلى تنقل معقد.

المرحلة 3: بناء الواجهة الأساسية (ثابتة)

أنشئ المشروع. اسم الحزمة (org) يصبح جزءاً من معرف التطبيق في المتجرين ويصعب تغييره لاحقاً، فاختره بعناية على نمط نطاق معكوس:

```
flutter create --project-name dhikr_counter --org sa.edu.kau.students --
platforms android,ios dhikr_counter
cd dhikr_counter
git init && git add . && git commit -m "flutter create"
```

احذف محتوى `lib/main.dart` الافتراضي واكتب شاشة ثابتة لا تفعل شيئاً. الهدف الوحيد: رؤية الشكل على الشاشة والتأكد أن حلقة التطوير (تعديل ← حفظ ← hot reload) تعمل.

```
import 'package:flutter/material.dart';

/// المرحلة 3: واجهة ثابتة بلا أي منطق. الهدف: رؤية الشكل على الشاشة أولاً
class StaticCounterScreen extends StatelessWidget {
  const StaticCounterScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('عداد الأذكار')),
      body: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Text('سُبْحَانَ اللَّهِ', style: TextStyle(fontSize: 28)),
          const SizedBox(height: 16),
          const Text('0', style: TextStyle(fontSize: 72)),
          const SizedBox(height: 32),
          FilledButton(
            onPressed: () {}, // لا يفعل شيئاً بعد
            child: const Text('سَيِّح'),
          ),
        ],
      ),
    );
  }
}
```

```

    ),
  ],
),
);
}
}

```

واستدعها من `main` مؤقتًا:

```
void main() => runApp(const MaterialApp(home: StaticCounterScreen()));
```

شغل `flutter run`. غير حجم الخط واحفظ؛ يجب أن ترى التغيير خلال ثانية. `git commit -am "static counter screen"`

ما تعلمته: `Scaffold` هيكل الشاشة (شريط علوي + جسم)، `Column` يرص الأبناء عموديًا، `SizedBox` مسافة، و `const` أمام الـ `widgets` الثابتة تحسين مجاني.

المرحلة 4: جعل زر العدّ يعمل وإدارة حالة العدّاد

الآن أول «حالة» حقيقية. الرقم لم يعد ثابتًا؛ يتغير مع كل نقرة. في Flutter، الـ `widget` الذي يحمل حالة متغيرة هو `StatefulWidget`، ودالة `setState` تخبر Flutter بأن يعيد الرسم:

```

import 'package:flutter/material.dart';

/// المرحلة 4: أول "حالة" حقيقية. StatefulWidget + setState.
class SimpleCounterScreen extends StatefulWidget {
  const SimpleCounterScreen({super.key});

  @override
  State<SimpleCounterScreen> createState() => _SimpleCounterScreenState();
}

class _SimpleCounterScreenState extends State<SimpleCounterScreen> {
  الحالة: قيمة تتغير مع الزمن وتؤثر في ما يُعرض //
  int _count = 0;

  void _increment() {
    // setState تخبر Flutter: "الحالة تغيّرت، أعد بناء هذه الشاشة".
    setState(() => _count++);
  }

  @override

```

```
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('عداد الأذكار')),
    body: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text('$_count', style: const TextStyle(fontSize: 72)),
        const SizedBox(height: 32),
        FilledButton(onPressed: _increment, child: const Text('سَيِّح')),
      ],
    ),
  );
}
```

شغل وانقر. الرقم يزداد. انقر بسرعة عشرين مرة: هل ضاعت نقرات؟ (لا؛ Flutter يعالج اللمسات بالتسلسل). أدر الشاشة: الرقم عاد إلى الصفر؟ لا، لأن حالة الـ widget تبقى عند الدوران. لكن أغلق التطبيق تمامًا وافتحه: الرقم صفر. هذه هي مشكلة المرحلة التالية.

لقطة Git. ما تعلمته: الفرق بين `StatelessWidget` (يرسم من مدخلاته فقط) و `StatefulWidget` (يملك حالة تتغير)، وأن `setState` لا «تغير النص» بل تعلن أن الحالة تغيرت فيعيد بناء الواجهة من الحالة الجديدة.

انتبه: لماذا لن نبقى على `setState`؟

`setState` ممتازة للحالة المحلية. لكن العدد الآن يحتاج إلى الحفظ (طبقة بيانات) وستحتاجه شاشة القائمة لاحقًا (شاشة أخرى). حالة تحتاجها أكثر من شاشة وتلمس التخزين هي «حالة تطبيق»، ومكانها خارج الـ widget. لذلك سنعيد التنظيم في المرحلة التالية قبل أن يكبر الكود، لا بعده.

المرحلة 5: حفظ العداد محليًا واستعادته عند فتح التطبيق

هنا نبني البنية التي ستحمل بقية المشروع: نموذج، ومخزن مجرد بتنفيذين، ومتحكم يحمل الحالة. نضيف حزمة التخزين البسيط:

```
flutter pub add shared_preferences
```

أولاً النموذج `lib/models/dhikr.dart`. نكتبه الآن رغم أننا ما زلنا بذكر واحد، لأن «العدد» يخص ذكرًا ما، والمعرف الثابت سيسمح لنا بحفظ عدد لكل ذكر لاحقًا دون تغيير صيغة التخزين:

```
/// يمثل ذكرًا واحدًا في القائمة (Model) نموذج.
/// النموذج "غبي" عمدًا: يحمل البيانات فقط ولا يعرف شيئًا عن الشاشات أو التخزين.
class Dhikr {
  const Dhikr({required this.id, required this.text, required this.target});

  /// معرف ثابت لا يتغير حتى لو عدّلنا النص لاحقًا.
  final String id;

  /// نص الذكر كما يظهر للمستخدم.
  final String text;

  /// العدد المستهدف (مثل 33 أو 100).
  final int target;
}

/// قائمة مبدئية مضمّنة في التطبيق. في مرحلة لاحقة يمكن نقلها إلى ملف
/// أو السماح للمستخدم بإضافة أذكاره الخاصة.
const List<Dhikr> defaultAdhkar = [
  Dhikr(id: 'subhanallah', text: 'سُبْحَانَ اللَّهِ', target: 33),
  Dhikr(id: 'alhamdulillah', text: 'الْحَمْدُ لِلَّهِ', target: 33),
  Dhikr(id: 'allahuakbar', text: 'اللَّهُ أَكْبَرُ', target: 34),
  Dhikr(id: 'istighfar', text: 'أَسْتَغْفِرُ اللَّهَ', target: 100),
  Dhikr(id: 'hawqala', text: 'لَا حَوْلَ وَلَا قُوَّةَ إِلَّا بِاللَّهِ', target: 100),
];
```

ثانيًا طبقة البيانات `lib/data/counter_store.dart`. لاحظ ثلاثة قرارات: عقد مجرد `CounterStore` حتى نستبدل التخزين في الاختبار؛ التخزين كخريطة «معرف ← عدد» في نص JSON واحد لأن البيانات صغيرة؛ ومعالجة البيانات التالفة من الآن (سنعود إليها في المرحلة 7):

```
import 'dart:convert';

import 'package:shared_preferences/shared_preferences.dart';

/// عقد (واجهة مجردة) يصف ما نحتاجه من "مخزن العدادات" دون تحديد كيفية
/// التخزين.
/// فائدة العقد: يمكن استبدال التخزين الحقيقي بنسخة في الذاكرة أثناء الاختبار.
abstract class CounterStore {
  /// يُرجع خريطة: معرف الذكر -> العدد المحفوظ.
  Future<Map<String, int>> loadCounts();

  /// يحفظ الخريطة كاملة. الحجم صغير جدًا فلا حاجة لقاعدة بيانات هنا.
}
```

```

Future<void> saveCounts(Map<String, int> counts);
}

/// shared_preferences تنفيذ حقيقي يعتمد على حزمة.
/// واحد تحت مفتاح واحد JSON نخزن الخريطة كنص.
class SharedPrefsCounterStore implements CounterStore {
  SharedPrefsCounterStore([SharedPreferencesAsync? prefs])
    : _prefs = prefs ?? SharedPreferencesAsync();

  final SharedPreferencesAsync _prefs;
  static const _key = 'dhikr_counts_v1';

  @override
  Future<Map<String, int>> loadCounts() async {
    final raw = await _prefs.getString(_key);
    if (raw == null || raw.isEmpty) return {};
    try {
      final decoded = jsonDecode(raw);
      if (decoded is! Map) return {};
      // نتحقق من نوع كل قيمة: أي قيمة تالفة تُهمل بدل أن تُسقط التطبيق.
      return {
        for (final entry in decoded.entries)
          if (entry.value is int && (entry.value as int) >= 0)
            entry.key.toString(): entry.value as int,
      };
    } on FormatException {
      // بيانات تالفة (مثلاً من إصدار قديم): نبدأ من الصفر بدل الانهيار.
      return {};
    }
  }

  @override
  Future<void> saveCounts(Map<String, int> counts) {
    return _prefs.setString(_key, jsonEncode(counts));
  }
}

/// تنفيذ في الذاكرة فقط، للاختبارات وللتجربة السريعة.
class InMemoryCounterStore implements CounterStore {
  final Map<String, int> _data = {};

  @override
  Future<Map<String, int>> loadCounts() async => Map.of(_data);

  @override

```

```
Future<void> saveCounts(Map<String, int> counts) async {
  _data
    ..clear()
    ..addAll(counts);
}
}
```

ثالثًا المتحكم `lib/state/counter_controller.dart`. يرث `ChangeNotifier`. يحمل الحالة، ويحفظ بعد كل تغيير. لا يعرف شيئًا عن الـ `widgets`:

```
import 'package:flutter/foundation.dart';

import '../data/counter_store.dart';
import '../models/dhikr.dart';

/// عقل الشاشة: يحمل الحالة (الذكر الحالي وعدّه) ويبلغّ الواجهة عند تغييرها.
/// لذا يمكن اختباره دون تشغيل واجهة Widgets، لا يعرف شيئًا عن الـ
class CounterController extends ChangeNotifier {
  CounterController({required CounterStore store, List<Dhikr>? adhkar})
    : _store = store,
      adhkar = adhkar ?? defaultAdhkar;

  final CounterStore _store;
  final List<Dhikr> adhkar;

  Map<String, int> _counts = {};
  Dhikr? _current;
  bool _loaded = false;

  bool get isLoading => !_loaded;
  Dhikr get current => _current ?? adhkar.first;
  int get count => _counts[current.id] ?? 0;
  bool get reachedTarget => count >= current.target;

  /// يُستدعى مرة واحدة عند فتح التطبيق لاستعادة ما حُفظ سابقًا.
  Future<void> load() async {
    _counts = await _store.loadCounts();
    _loaded = true;
    notifyListeners();
  }

  void selectDhikr(Dhikr dhikr) {
    _current = dhikr;
    notifyListeners();
  }

  /// يزيد العدّاد ويحفظ. نحفظ بعد كل نقرة لأن الكتابة صغيرة ورخيصة،
```

```

///ولأن المستخدم قد يغلق التطبيق في أي لحظة
Future<void> increment() async {
  _counts[current.id] = count + 1;
  notifyListeners();
  await _store.saveCounts(_counts);
}

Future<void> reset() async {
  _counts.remove(current.id);
  notifyListeners();
  await _store.saveCounts(_counts);
}
}

```

لماذا نحفظ بعد كل نقرة لا عند الخروج؟ لأن الأنظمة تقتل التطبيقات في الخلفية دون إنذار، ولا يوجد حدث «قبل الإغلاق» موثوق على الهاتف. الكتابة هنا بضع مئات من البايتات ولا تلاحظ. في تطبيق يحفظ صورًا كبيرة كان القرار سيختلف (حفظ عند اكتمال الإدخال).

رابعًا نقطة الدخول `lib/main.dart`: ننشئ التبعيات في مكان واحد ونمررها للأسفل (هذا هو حقن التبعيات بأبسط صورته). السطور الخاصة باللغة سنشرحها في المرحلة 9، اتركها الآن كما هي:

```

import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';

import 'data/counter_store.dart';
import 'screens/counter_screen.dart';
import 'state/counter_controller.dart';

void main() {
  //ننشئ التبعيات في مكان واحد (نقطة الدخول) ونمررها للأسفل.
  final controller = CounterController(store: SharedPrefsCounterStore());
  controller.load();
  runApp(DhikrApp(controller: controller));
}

class DhikrApp extends StatelessWidget {
  const DhikrApp({super.key, required this.controller});

  final CounterController controller;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'عداد الأذكار',
      debugShowCheckedModeBanner: false,

```

```

Material ونصوص عناصر RTL اللغة العربية افتراضيًا: يضبط اتجاه العرض //
الجاهزة.
locale: const Locale('ar'),
supportedLocales: const [Locale('ar'), Locale('en')],
localizationsDelegates: GlobalMaterialLocalizations.delegates,
theme: ThemeData(
  colorSchemeSeed: const Color(0xFF315B45),
  useMaterial3: true,
),
home: CounterScreen(controller: controller),
);
}
}

```

وستكتب شاشة العد الجديدة في المرحلة التالية مع القائمة. مؤقتًا، بدل `StaticCounterScreen` بنسخة `ListenableBuilder` من المتحكم عبر `ListenableBuilder`. اختبر: عد، أغلق التطبيق تمامًا (اسحبه من قائمة التطبيقات الأخيرة)، افتح: العدد باق. لقطة `Git`.

المرحلة 6: إضافة قائمة أنكار بسيطة

البنية جاهزة، فالإضافة الآن صغيرة: شاشة قائمة تعرض `controller.adhkar`، والنقر على صف `selectDhikr` يستدعي `lib/screens/dhikr_list_screen.dart` وينشئ.

```

import 'package:flutter/material.dart';

import '../state/counter_controller.dart';

class DhikrListScreen extends StatelessWidget {
  const DhikrListScreen({super.key, required this.controller});

  final CounterController controller;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('اختر ذكرًا')),
      body: ListView.separated(
        itemCount: controller.adhkar.length,
        separatorBuilder: (context, index) => const Divider(height: 1),
        itemBuilder: (context, index) {
          final dhikr = controller.adhkar[index];
          final selected = dhikr.id == controller.current.id;
          return ListTile(

```

```

        title: Text(dhikr.text),
        subtitle: Text('الهدف: ${dhikr.target}'),
        trailing: selected ? const Icon(Icons.check) : null,
        selected: selected,
        onTap: () {
          controller.selectDhikr(dhikr);
          Navigator.pop(context);
        },
      );
    },
  ),
);
}
}

```

ثم شاشة العد الكاملة `lib/screens/counter_screen.dart`، وفيها أيضًا حوار تأكيد التصفير ومؤشر التحميل والاهتزاز (المرحلة 7 تشرح تلك الأجزاء):

```

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import '../state/counter_controller.dart';
import 'dhikr_list_screen.dart';

class CounterScreen extends StatelessWidget {
  const CounterScreen({super.key, required this.controller});

  final CounterController controller;

  Future<void> _confirmReset(BuildContext context) async {
    final ok = await showDialog<bool>(
      context: context,
      builder: (context) => AlertDialog(
        title: const Text('تصفير العدّاد؟'),
        content: const Text('سيبدأ العدّ من الصفر لهذا الذكر'),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context, false),
            child: const Text('إلغاء'),
          ),
          FilledButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('تصفير'),
          ),
        ],
      ),
    );
  }
}

```

```

    if (ok == true) await controller.reset();
  }

Future<void> _onTap() async {
  final wasBelowTarget = !controller.reachedTarget;
  await controller.increment();
  اهتزاز خفيف عند بلوغ الهدف (يعمل على الجهاز الحقيقي، لا على المحاكى
  غالبًا).
  if (wasBelowTarget && controller.reachedTarget) {
    HapticFeedback.mediumImpact();
  } else {
    HapticFeedback.selectionClick();
  }
}

@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  return ListenableBuilder(
    listenable: controller,
    builder: (context, _) {
      if (!controller.isLoading) {
        return const Scaffold(
          body: Center(child: CircularProgressIndicator()),
        );
      }
      final dhikr = controller.current;
      return Scaffold(
        appBar: AppBar(
          title: const Text('عداد الأذكار'),
          actions: [
            IconButton(
              tooltip: 'اختيار ذكر',
              icon: const Icon(Icons.list_alt),
              onPressed: () => Navigator.push(
                context,
                MaterialPageRoute<void>(
                  builder: (_) => DhikrListScreen(controller: controller),
                ),
              ),
            IconButton(
              tooltip: 'تصفير',
              icon: const Icon(Icons.refresh),
              onPressed: () => _confirmReset(context),
            ),
          ],
        ),
        body: SafeArea(

```

```

child: Column(
  children: [
    const Spacer(),
    Padding(
      padding: const EdgeInsets.symmetric(horizontal: 24),
      child: Text(
        dhikr.text,
        textAlign: TextAlign.center,
        style: theme.textTheme.headlineMedium,
      ),
    ),
    const SizedBox(height: 24),
    Semantics(
      // نص يقرأه قارئ الشاشة (TalkBack / VoiceOver).
      label: 'العدد الحالي ${controller.count} من $
{dhikr.target}',
      child: Text(
        '${controller.count}',
        style: theme.textTheme.displayLarge?.copyWith(
          fontWeight: FontWeight.w800,
          color: controller.reachedTarget
            ? theme.colorScheme.primary
            : null,
        ),
      ),
    ),
    Text('الهدف: ${dhikr.target}',
      style: theme.textTheme.bodyLarge),
    const Spacer(),
    // زر كبير يملأ عرض الشاشة تقريبًا: هدف اللمس سهل حتى دون النظر
    Padding(
      padding: const EdgeInsets.all(24),
      child: SizedBox(
        width: double.infinity,
        height: 160,
        child: FilledButton(
          onPressed: _onTap,
          child: const Text('سَيِّح', style: TextStyle(fontSize:
32))),
      ),
    ),
  ],
),

```

```

    ),
    );
},
);
}
}

```

شغل: افتح القائمة، اختر «أستغفر الله»، عد خمسًا، ارجع إلى القائمة واختر «سبحان الله»، عد ثلاثًا، ثم عد إلى الاستغفار: يجب أن تجد الخمسة باقية، لأن الحفظ لكل ذكر بمعرفه. **لقطة Git**.

ما تعلمته: **ListView.separated** للقوائم، و **Navigator.push/pop** للتنقل، وأن الشاشتين تتشاركان المتحكم نفسه فتبقيان متسقتين دون أي كود مزامنة.

المرحلة 7: معالجة الحالات غير المتوقعة

عد إلى قائمة الحالات غير المعتادة في نموذج المتطلبات وتحقق من كل واحدة في الكود:

الحالة	أين عولجت	كيف تختبرها يدويًا
إغلاق مفاجئ أثناء العد	الحفظ بعد كل نقرة في increment	عد ثم اسحب التطبيق من قائمة التطبيقات الأخيرة فورًا
بيانات محفوظة نالفة	try/on FormatException والتحقق من نوع كل قيمة في loadCounts	اختبار الوحدة في المرحلة 8 يحقن نصًا غير صالح
تصفير بالخطأ	حوار تأكيد _confirmReset	انقر تصفير ثم «إلغاء»؛ يجب ألا يتغير شيء
تجاوز الهدف	يستمر العد، reachedTarget يبقى صحيحًا، اللون مميز + اهتزاز عند لحظة البلوغ فقط	عد حتى 33 ثم استمر
فتح التطبيق قبل اكتمال التحميل	isLoading يعرض مؤشرًا بدل رقم خاطئ	لن تلاحظها على جهاز سريع، لكنها تمنع ومضة «0» ثم «57»
حجم خط النظام كبير جدًا	الرقم في Column مرّن مع Spacer ؛ نص الذكر في Padding ويلتف	إعدادات الهاتف ← حجم الخط ← الأكبر، ثم افتح التطبيق
قارئ الشاشة	Semantics(label:) على الرقم، و tooltip على الأزرار الأيقونية	فعل TalkBack وتنقل

لاحظ أن معالجة الحالات غير المعتادة لم تكن «مرحلة إضافية» بقدر ما كانت مراجعة: معظمها بني في مكانه الطبيعي حين كتبنا كل طبقة. هذا هو الوضع الصحي؛ ما يجب أن تفعله هنا هو التحقق المنهجي لا الترقيع. نقطة Git.

المرحلة 8: اختبار التطبيق على المحاكى وجهاز حقيقي

الاختبارات الآلية أولاً. لأن المتحكم لا يعرف الواجهة، نختبره مباشرة بمخزن في الذاكرة. `test/`

`:counter_controller_test.dart`

```
import 'package:dhikr_counter/data/counter_store.dart';
import 'package:dhikr_counter/models/dhikr.dart';
import 'package:dhikr_counter/state/counter_controller.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  const testAdhkar = [Dhikr(id: 'a', text: 'ذكر أ', target: 3)];

  group('CounterController', () {
    test('يبدأ من الصفر ويزيد بمقدار واحد', () async {
      final controller =
        CounterController(store: InMemoryCounterStore(), adhkar:
testAdhkar);
      await controller.load();

      expect(controller.count, 0);
      await controller.increment();
      await controller.increment();
      expect(controller.count, 2);
      expect(controller.reachedTarget, isFalse);
    });

    test('يبلغ الهدف عند العدد المستهدف', () async {
      final controller =
        CounterController(store: InMemoryCounterStore(), adhkar:
testAdhkar);
      await controller.load();
      for (var i = 0; i < 3; i++) {
        await controller.increment();
      }
      expect(controller.reachedTarget, isTrue);
    });

    test('يستعيد العدد المحفوظ عند إعادة الفتح', () async {
```

```

    final store = InMemoryCounterStore();
    final first = CounterController(store: store, adhkar: testAdhkar);
    await first.load();
    await first.increment();

    // إعادة فتح التطبيق: متحكم جديد بنفس المخزن "
    final second = CounterController(store: store, adhkar: testAdhkar);
    await second.load();
    expect(second.count, 1);
  });

  test('التصفير يعيد العدد إلى الصفر ويحفظ', () async {
    final store = InMemoryCounterStore();
    final controller = CounterController(store: store, adhkar: testAdhkar);
    await controller.load();
    await controller.increment();
    await controller.reset();
    expect(controller.count, 0);
    expect(await store.loadCounts(), isEmpty);
  });
});
}

```

اختبار widget واحد يتأكد أن النقر يغير ما يعرض فعلاً، `test/counter_screen_test.dart`:

```

import 'package:dhikr_counter/data/counter_store.dart';
import 'package:dhikr_counter/screens/counter_screen.dart';
import 'package:dhikr_counter/state/counter_controller.dart';
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

void main() {
  testWidgets('النقر على الزر يزيد الرقم المعروض', (tester) async {
    final controller = CounterController(store: InMemoryCounterStore());
    await controller.load();

    await tester.pumpWidget(
      MaterialApp(home: CounterScreen(controller: controller)),
    );

    expect(find.text('0'), findsOneWidget);
    await tester.tap(find.text('سَيِّح'));
    await tester.pump();
    expect(find.text('1'), findsOneWidget);
  });
}

```

واختبار لطبقة التخزين الحقيقية ببديل في الذاكرة توفره الحزمة نفسها، وفيه اختبار البيانات التالية، `test/`
`flutter pub add` إلى `counter_store_test.dart`
`:(dev:shared_preferences_platform_interface`

```
import 'package:dhikr_counter/data/counter_store.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:shared_preferences/shared_preferences.dart';
import 'package:shared_preferences_platform_interface/
in_memory_shared_preferences_async.dart';
import 'package:shared_preferences_platform_interface/
shared_preferences_async_platform_interface.dart';

void main() {
  setUp(() {
    // نستبدل التخزين الحقيقي بنسخة في الذاكرة داخل الاختبار .
    SharedPreferencesAsyncPlatform.instance =
      InMemorySharedPreferencesAsync.empty();
  });

  test('يحفظ ويسترجع الأعداد', () async {
    final store = SharedPrefsCounterStore(SharedPreferencesAsync());
    await store.saveCounts({'a': 5, 'b': 0});
    expect(await store.loadCounts(), {'a': 5, 'b': 0});
  });

  test('يتجاهل البيانات التالية بدل الانهيار', () async {
    final prefs = SharedPreferencesAsync();
    await prefs.setString('dhikr_counts_v1', '{ ليس json }');
    final store = SharedPrefsCounterStore(prefs);
    expect(await store.loadCounts(), isEmpty);
  });
}
```

```
flutter analyze # يجب : No issues found!
flutter test    # يجب : All tests passed! (7 اختبارات)
```

ثم الاختبار اليدوي على جهاز حقيقي بقائمة ثابتة تعيدها مع كل إصدار: التثبيت من الصفر (حالة فارغة)،
 العد، الاهتزاز عند الهدف (لن تشعر به على المحاكى)، التصفير وإلغاء، التبديل بين الأذكار، الإغلاق التام
 وإعادة الفتح، تدوير الشاشة، أكبر حجم خط، الوضع الداكن، وTalkBack لدقيقة. سجل ما وجدته في
`README.md` . لقطة `Git`.

مع الذكاء الاصطناعي

هذه أفضل لحظة لاستخدام برومبت «كتابة اختبار» من الفصل الثامن: أعط الأداة `CounterController` واطلب حالات إضافية لم تخطر لك. ثم لكل اختبار تقترحه اسأل نفسك: هل يختبر سلوكًا يهم المستخدم، أم يختبر أن Dart تعمل؟

المرحلة 9: تحسين الواجهة العربية

عد إلى `main.dart`. السطور الثلاثة الخاصة باللغة تفعل الكثير: `locale: Locale('ar')` يجعل العربية لغة التطبيق، و `GlobalMaterialLocalizations.delegates` (من حزمة `flutter_localizations` المدمجة، أضفها بـ `flutter pub add flutter_localizations --sdk=flutter`) يوفر النصوص العربية لعناصر Material الجاهزة (أزرار الحوارات، tooltips الرجوع) ويقلب اتجاه الواجهة كلها إلى RTL. لاحظ في التطبيق: زر الرجوع في شاشة القائمة صار على اليمين، وعلامة الصح على اليسار، دون أن نكتب سطرًا لذلك. قائمة مراجعة عربية لهذا المشروع:

- ☐ كل نص عربي مكتوب بحركات حيث تهتم القراءة (نصوص الأذكار) وبدونها حيث لا تهتم (الأزرار) للوضوح.
- ☐ الأرقام: قررنا عرضها بالأرقام الغربية ('`controller.count`') لوضوحها من بعيد. لو أردت الأرقام الهندية فاستخدم `NumberFormat.decimalPattern('ar')` من حزمة `intl` واختبر الناتج على الجهاز.
- ☐ لا يوجد في هذا المشروع `EdgeInsets.only(left:)` ولا `Alignment.centerLeft`؛ استخدمنا `symmetric` و `Directional`. راجع كودك بالبحث عن `left` و `right`.
- ☐ الخط: الخط الافتراضي في Flutter للعربية مقبول لكنه عام. لخط مخصص (مثل خطوط Google العربية) أضف ملفات الخط إلى `pubspec.yaml` واضبطه في `ThemeData.fontFamily`. تأكد من رخصة الخط.

□ جرب التطبيق بلغة الجهاز الإنجليزية أيضًا: لأننا ثبتنا `locale` على العربية فسيبقى عربيًا، وهذا مقصود في تطبيق عربي. لو أردت لغتين فاحذف `locale` الثابت واستخدم ملفات ARB كما في وثائق الترجمة.

نقطة Git.

المرحلة 10: تجهيز إصدار تجريبي

ثلاثة أشياء لـ Android يمكنك إتقانها بالكامل على جهازك:

(أ) الأيقونة والاسم. اسم التطبيق الظاهر في `android/app/src/main/AndroidManifest.xml` (`android:label`) وفي `ios/Runner/Info.plist` (`CFBundleDisplayName`). الأيقونة عبر حزمة `flutter_launcher_icons` من صورة 1024×1024 واحدة.

(ب) مفتاح الرفع. أنشئ `keystore` مرة واحدة واحفظه خارج المشروع وفي نسخة احتياطية:

```
keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA -keysize 2048 -
validity 10000 -alias upload
```

ثم ملف `android/key.properties` (وأضفه إلى `.gitignore` فورًا) واربطه في `android/app/` `build.gradle.kts` كما تشرح وثائق Flutter خطوة بخطوة:

<https://docs.flutter.dev/deployment/android>

(ج) رقم الإصدار والبناء. في `pubspec.yaml` : `version: 0.1.0+1` . ثم:

```
flutter build appbundle --release
# الناتج: build/app/outputs/bundle/release/app-release.aab
flutter build apk --release --split-per-abi
# لإرسالها مباشرة لمختبرين خارج المتجر APK ملفات
```

ثبت نسخة الإصدار على هاتفك واختبرها بالقائمة اليدوية نفسها؛ وضع `release` قد يكشف ما لا يظهر في `debug`. ثم ارفع الـ `aab` إلى مسار الاختبار الداخلي في `Play Console` وأضف زملاءك بعنوانين بريدهم. اقرأ تقرير ما قبل الإطلاق حين يصدر.

لـ iOS اتبع الفصل الخامس: مستودع Codemagic ← GitHub ← مفتاح App Store Connect ← بناء ورفع إلى TestFlight ← زميل يملك iPhone. قبل البناء الأول راجع `ios/Runner/Info.plist`: هذا التطبيق لا يستخدم أي إذن فلا نصوص شرح مطلوبة، لكن تطبيق الفواتير سيحتاج إلى `NSCameraUsageDescription` و `NSPhotoLibraryUsageDescription`.
وسم الإصدار: `git tag v0.1.0 && git push --tags`. وسجل في `CHANGELOG.md` ما يتضمنه.

ما بعد الإصدار التجريبي: مراحل لاحقة

الآن، وفقط الآن، افتح قائمة «مؤجل». كل بند فيها يمكن أن يصبح إصدارًا صغيرًا بالمنهج نفسه (متطلب ← رسم ← كود ← اختبار ← نقطة):

- **الإحصاءات (v0.2):** تتطلب حفظ تاريخ كل جلسة عد لا العدد الحالي فقط. هذا تغيير في صيغة التخزين، فهو أول ترحيل بيانات ستكتبه. ربما يستدعي الانتقال إلى SQLite.
- **التنبهات (v0.3):** إشعار محلي يومي لأنكار الصباح والمساء عبر `flutter_local_notifications`. يجلب معه أول إذن (الإشعارات) وأول اختلاف حقيقي بين المنصتين.
- **العد بطرق متعددة (v0.4):** النقر على أي مكان في الشاشة، أو زر خفض الصوت (يتطلب كودًا أصليًا عبر platform channel على كل منصة، وهو مدخل جيد لفهم «الجسر»).
- **أنكار مخصصة:** إضافة وتعديل وحذف؛ يجلب التحقق من المدخلات ونماذج الإدخال.
- **التصدير والاستيراد:** ملف JSON عبر ورقة المشاركة؛ أول تعامل مع الملفات.

لاحظ كيف تفتح كل ميزة بابًا واحدًا جديدًا من مفاهيم الفصل السابع. هذا ليس صدفة؛ هكذا يتعلم المطورون فعلاً: ميزة تجلب مفهومًا، لا فصل نظري يجلب عشرة.

لماذا لا نضيف كل شيء في اليوم الأول؟

لأن كل مرحلة أعلاه كشفت قرارًا لم يكن ليتضح قبلها: المعرف الثابت في النموذج لم يكن ليخطر لنا لولا التفكير في القائمة، والحفظ بعد كل نقرة لم يكن ليتضح لولا سؤال «ماذا لو أغلق فجأة»، والمخزن المجرد أثبت قيمته في الاختبار. لو كتبنا كل شيء معًا لاتخذنا هذه القرارات بالخطأ ودفعنا ثمنها في إعادة الكتابة. الصغير الذي يعمل يعلمك أكثر من الكبير الذي لا يعمل.

CHAPTER 10

أخطاء شائعة ونصائح عملية

كل خطأ في هذا الفصل رأيته يتكرر في مشاريع طلابية، وارتكبت معظمها بنفسى. لكل خطأ: كيف يبدو من الداخل (لأنه لا يبدو خطأ وأنت فيه)، ولماذا يضر، وطريقة عملية واحدة على الأقل لتجنبه. اقرأه الآن، ثم أعد قراءته بعد أسبوعين من بدء مشروعك؛ ستفهمه بطريقة مختلفة.

1. البدء بعشرات الميزات

كيف يبدو من الداخل: «التطبيق سيكون رائعًا: عداد، وإحصاءات، ومزامنة، وحسابات، وثيمات، وودجت». تشعر بالحماس وتفتح عشر ملفات في الأسبوع الأول.

لماذا يضر: لا شيء يعمل كاملاً، فلا تستطيع تجربة التطبيق ولا إعطاءه لأحد، فيموت الحماس في الأسبوع الثالث. وكل ميزة نصف مكتملة تعقد الميزات الأخرى.

كيف تتجنبه: قائمة «مؤجل» في `README.md`. كل فكرة جديدة تكتب فيها فوراً بدل تنفيذها. اسمح لنفسك بميزة جديدة فقط حين تعمل السابقة على هاتفك واختبرها زميل. اسأل مع كل ميزة: «لو حذفها، هل يفقد التطبيق سبب وجوده؟».

2. نسخ كود الذكاء الاصطناعي دون فهمه

كيف يبدو من الداخل: «يعمل، لماذا أقرأه؟». ثم بعد أسبوع تريد تعديلاً صغيراً فتكتشف أنك لا تعرف أين، فتطلب من الأداة التعديل، فتكسر شيئاً آخر لا تعرفه أيضاً.

لماذا يضر: تفقد القدرة على التصحيح، وتتراكم أكواد متعارضة لأن كل محادثة تقترح أسلوباً مختلفاً، وفي المناقشة أو المقابلة لا تستطيع شرح مشروعك.

كيف تتجنبه: قاعدة «اشرحه في دقيقتين» من الفصل الثامن. تمرين مفيد: بعد قبول أي كود، أغلق الأداة واكتب في تعليق فوق كل دالة سطرًا يشرح ما تفعله بكلماتك. إن لم تستطع، لم تفهمه بعد. وامنع نفسك من قبول أكثر من ملف واحد في كل مرة.

3. إهمال Git

كيف يبدو من الداخل: «المشروع صغير، سأبدأ Git لاحقاً». أو: لقطة واحدة كل أسبوع برسالة «update». أو: مجلدات `project_v2_final_FINAL`.

لماذا يضر: أول مرة يكسر فيها تغيير كبير التطبيق ولا تعرف ما الذي تغير، ستضيع يوماً كاملاً كان يمكن أن يكون أمر `git diff` واحداً. وأول مرة يتعطل فيها الحاسوب، تفقد كل شيء.

كيف تتجنبه: `git init` قبل أول سطر كود. لقطة لكل إنجاز صغير برسالة تصف السلوك («save count after each tap») لا الملف («edit controller»). دفع إلى GitHub يوميًا. وقبل أي تجربة جريئة: فرع جديد `git switch` `-c try-sqlite`؛ إن نجحت دمجه وإن فشلت حذفته دون أثر.

4. الاختبار على جهاز واحد فقط

كيف يبدو من الداخل: «يعمل على هاتفي تمامًا». وهاتفك حديث، بشاشة كبيرة، وخط افتراضي، ولغة واحدة، وذاكرة فارغة.

لماذا يضر: المستخدم الأول لديه هاتف عمره أربع سنوات، وحجم خط «كبير جدًا»، وشاشة أقصر، ونظام أقدم بإصدارين. أول تقييم سيكون «الأزرار مقطوعة».

كيف تتجنبه: محاكي واحد بمواصفات متواضعة (شاشة صغيرة، أقدم API تدعمه) إلى جانب هاتفك. جولة «التخريب» قبل كل إصدار: أكبر حجم خط، الوضع الداكن، الدوران، انقطاع الشبكة، لغة ثانية. مجموعتكم الطلابية هي مختبر أجهزة مجاني: تبادلو ملفات APK ونسخ TestFlight. وقرأ تقرير ما قبل الإطلاق في Play الذي يجرب تطبيقك على أجهزة لم تملكها.

5. الخلط بين خطأ الواجهة وخطأ البيانات

كيف يبدو من الداخل: الرقم على الشاشة خاطئ، فتعدل في الـ widget. لا يتغير شيء، فتعدل أكثر. الخطأ في الحقيقة كان في دالة الحفظ.

لماذا يضر: تضيع ساعات في المكان الخطأ، وتضيف «إصلاحات» في الواجهة تخفي الخطأ الحقيقي بدل حله (مثل إعادة قراءة القيمة مرتين).

كيف تتجنبه: هذه هي الفائدة العملية للطبقات الثلاث. حين يظهر خطأ اسأل بالترتيب: (1) هل البيانات المحفوظة صحيحة؟ (اطبعها أو افحص الملف). (2) هل حالة المتحكم صحيحة؟ (اختبار وحدة أو سطر `debugPrint` في المتحكم). (3) هل الواجهة تعرض الحالة صحيحة؟ ابدأ من الأسفل لا من الشاشة. والاختبارات الآلية للمتحكم والمخزن تجيب عن (1) و(2) في ثوان.

6. تأجيل الأنونات والخصوصية إلى نهاية المشروع

كيف يبدو من الداخل: «سأضيف نصوص الأنونات وسياسة الخصوصية قبل النشر بيوم». ثم قبل النشر بيوم تكتشف أن iOS ينهار عند فتح الكاميرا لغياب نص الشرح، وأن نموذج أمان البيانات يسأل أسئلة لا تعرف إجابتها عن حزمة أضفتها قبل شهرين، وأن التصميم لا يحتوي مساراً لمن يرفض الإذن.

لماذا يضر: الأنونات قرار تصميم يغير الشاشات (مسار الرفض)، والخصوصية قرار معماري (أي بيانات تخرج من الجهاز وإلى أين). تأجيلهما يعني إعادة تصميم في أسوأ وقت.

كيف تتجنبه: في نموذج المتطلبات قسم دائم: «الأنونات التي أحتاجها ولماذا، وماذا يحدث عند الرفض». ومع كل حزمة تضيفها، اكتب في `README.md` ما تجمعه من بيانات (صفحة الحزمة تذكر ذلك عادةً). سياسة الخصوصية اكتب مسودتها في الأسبوع الثاني؛ تحديثها لاحقاً أسهل من كتابتها تحت الضغط.

7. أخطاء أخرى تستحق التحذير

الخطأ	البديل العملي
اختيار أعقد أداة لإدارة الحالة أو أعقد بنية «لأنها احترافية»	أبسط أداة تحل مشكلتك الحالية؛ ترق حين تشعر بالألم فعلاً
تجاهل تحذيرات <code>flutter analyze</code> لأنها «مجرد تحذيرات»	صفر تحذيرات كقاعدة؛ كل تحذير جديد إما يصلح أو يفهم سبب تجاهله ويوثق
وضع الأسرار في الكود أو في Git «مؤقتاً»	<code>.gitignore</code> قبل إنشاء الملف السري؛ لا يوجد «مؤقتاً» في Git
تحديث Flutter وكل الحزم في منتصف ميزة	التحديث في لقطة مستقلة وفي وقت هادئ، ثم تشغيل كل الاختبارات
الاعتماد على وضع debug في الحكم على الأداء	القياس في وضع <code>--profile</code> على جهاز حقيقي
حفظ المسارات المطلقة للملفات	حفظ الاسم فقط وتركيب المسار عند القراءة (iOS يغير مسار الحاوية)
«سأكتب الاختبارات في النهاية»	اختبار واحد مع كل قاعدة عمل وقت كتابتها؛ الكتابة لاحقاً لا تحدث أبداً

الخطأ	البديل العملي
وعد المستخدم بتنبيه «في الثانية المحددة»	صياغة صادقة («سندرك حوالي الساعة 7») وشرح إعدادات البطارية عند الحاجة
مطالبة الأداة بـ «إصلاح كل الأخطاء» دفعة واحدة	خطأ واحد، فرضية، تحقق، إصلاح، اختبار، لقطة، ثم التالي
تجاهل صفحة الحزمة على pub.dev قبل الاعتماد عليها	تحقق من: تاريخ آخر إصدار، الناشر، عدد المستخدمين، شارة Flutter Favorite، والمشكلات المفتوحة

نصائح عملية للعمل ضمن مجموعة طلابية

- تبادلو الاختبار لا الكود: كل واحد يختبر تطبيق الآخر بقائمة «التخريب» ويكتب ما وجد. هذا يحل مشكلة الجهاز الواحد وشرط الـ 12 مختبرًا معًا.
- مراجعة كود أسبوعية: عشرون دقيقة يعرض فيها أحدكم `git diff` أسبوعه ويشرحه. من يشرح يتعلم أكثر ممن يستمع.
- سجل قرارات مشترك: ملف واحد يجمع «واجهنا كذا، اخترنا كذا، لأن كذا». بعد شهرين سيكون أثنان ما لديكم.
- من يملك iPhone يصبح موردًا للجميع في اختبار TestFlight، ومن يملك Mac يستطيع تنظيم جلسة تصحيح iOS شهرية.

CHAPTER 11

خطة تعلم من عشرة أسابيع وقوائم التحقق

هذه خطة واقعية لطالب يخصص من 8 إلى 12 ساعة أسبوعيًا. كل أسبوع له هدف واحد ومخرج ملموس يمكن أن تريه لزميل، والمخرجات تراكمية: ما تبنيه في الأسبوع الثالث هو ما تختبره في السادس وتنشره في العاشر. إن تأخرت أسبوعًا فلا بأس؛ لا تتخط مخرجًا لتلحق بالجدول، فالمخرجات هي التعلم. وإن كنت أسرع، وسع المشروع لا الجدول.

الأسبوع	الهدف	المخرج الملموس	من الدليل
1	البيئة تعمل وفهم الصورة الكبيرة	<code>flutter doctor</code> سليم؛ التطبيق الافتراضي يعمل على هاتفك؛ مستودع GitHub؛ جواب مكتوب عن: ما الفرق بين الأصلي والمتعدد والويب، ولماذا اخترت Flutter	الفصول 1 إلى 5
2	أساسيات Dart و Widgets	إكمال مسار Dart التعليمي الرسمي وأول درسين من مسار Flutter؛ شاشة ثابتة لمشروعك (المرحلة 3)	الفصل 7 (المحطة 1 و 2)، المرحلة 3
3	نموذج المتطلبات والحالة	نموذج المتطلبات ذو الأقسام السبعة ورسومات الشاشات؛ زر يعمل بـ <code>setState</code> (المرحلة 4)	الفصل 6، المرحلة 4
4	تنظيم الكود في طبقات	نموذج + مخزن مجرد + متحكم <code>ChangeNotifier</code> ؛ الواجهة تقرأ من المتحكم؛ لا منطق في الـ <code>widgets</code>	الفصل 7 (البنية)، المرحلة 5
5	تخزين البيانات	البيانات تبقى بعد إغلاق التطبيق؛ معالجة البيانات التالفة؛ شاشة ثانية وتنقل	الفصل 7 (التخزين)، المرحلتان 5 و 6
6	الاختبارات والحالات غير المعتادة	3 اختبارات وحدة + اختبار <code>widget</code> تمر كلها؛ جدول الحالات غير المعتادة مع مكان معالجة كل منها؛ <code>flutter analyze</code> بلا تحذيرات	المرحلتان 7 و 8
7	العربية وإمكانية الوصول والتحسينات	واجهة RTL صحيحة؛ يعمل بأكبر حجم خط؛ <code>TalkBack</code> يقرأ الأزرار؛ قائمة «مؤجل» مرتبة بالأولوية	الفصل 7 (RTL)، الوصول، المرحلة 9
8	العمل مع الذكاء الاصطناعي بانضباط	ملف <code>AI_CONTEXT.md</code> ؛ ميزة واحدة من قائمة «مؤجل» أضيفت بمساعدة الأداة مع مراجعة <code>diff</code> واختبار؛ مراجعة كود من الأداة أخذ منها ما يستحق	الفصل 8

الأسبوع	الهدف	المخرج الملموس	من الدليل
9	تجربة بناء إصدار Android	keystore محفوظ بأمان: <code>aab</code> . موقع: أيقونة واسم؛ مسودة سياسة خصوصية؛ نسخة <code>release</code> مختبرة على هاتفك وهاتف زميل؛ اختبار داخلي في Play Console إن كان لديك حساب	الفصل 2، المرحلة 10
10	iOS في السحابة والإغلاق	بناء iOS ناجح على Codemagic (إن توفر حساب Apple) أو على الأقل سير عمل يبني ويشغل الاختبارات: <code>CHANGELOG.md</code> ووسم <code>v0.1.0</code> ؛ عرض عشر دقائق أمام المجموعة: ما بنيت، ما تعلمت، ما التالي	الفصل 5

إن كان لديك 12 أسبوعًا

أضف أسبوعًا بعد الخامس لـ `SQLite` (إن كان مشروعك يحتاج بحثًا أو علاقات)، وأسبوعًا بعد الثامن للإشعارات المحلية أو الكاميرا حسب فكرتك. هذان يفتحان بابي الأذونات والاختلاف بين المنصتين، وهما أكثر ما يفيدك رؤيته قبل مشروعك الثاني.

ماذا تتعلم بعد ذلك؟

بعد الأسبوع العاشر ستعرف ما تحتاجه أكثر مما يعرفه أي دليل. لكن هذه ترتيبات معقولة بحسب اتجاهك:

- إن أردت التعمق في Flutter: بقية مسار التعلم الرسمي، ثم `go_router`، ثم `provider` أو `Riverpod`، ثم الرسوم المتحركة، ثم `platform channels`.
- إن أردت الخادم: ابدأ بخدمة جاهزة (`Supabase` أو `Firebase`) لتطبيقك الحالي: مزامنة الفواتير بين جهازين. ستتعلم المصادقة وقواعد الأمان والعمل دون اتصال مع المزامنة.
- إن أردت الجودة: اختبارات التكامل على `Codemagic`، وتتبع الأعطال، وقياس الأداء، ثم أتمتة كاملة من الدفع إلى `TestFlight`.
- إن أردت فهم ما تحت Flutter: تعلم أساسيات `Kotlin` و `Compose` أو `Swift` و `SwiftUI` بمشروع صغير؛ ستفهم ما يفعله Flutter نيابةً عنك ومتى يقف.

قائمة التحقق الأولى: قبل أن تقول «تطبيقي جاهز للتجربة»

هذه القائمة لنسخة تعطيها لزملاء (اختبار داخلي أو APK مباشر أو TestFlight). ليست للنشر العام.

- ☐ الـ MVP كما كتبته في نموذج المتطلبات يعمل كاملاً، ولا توجد ميزة نصف مكتملة ظاهرة في الواجهة.
- ☐ البيانات تبقى بعد الإغلاق التام وإعادة الفتح، وبعد تحديث التطبيق فوق نسخة سابقة.
- ☐ كل حالة غير معتادة في نموذج المتطلبات إما معالجة أو موثقة كمعروفة.
- ☐ flutter analyze بلا تحذيرات، و flutter test تمر كلها.
- ☐ اختبر على جهاز حقيقي واحد على الأقل غير جهازك، وفي وضع release.
- ☐ جولة التخريب: أكبر حجم خط، الوضع الداكن، الدوران، لغة النظام المختلفة، انقطاع الشبكة (إن كانت مستخدمة).
- ☐ الشاشات الفارغة وحالات التحميل والخطأ لها محتوى مفهوم لا شاشة بيضاء.
- ☐ كل زر أيقوني له tooltip، و TalkBack يقرأ العناصر الرئيسية.
- ☐ الاسم والأيقونة ورقم الإصدار مضبوطة (version: في pubspec).
- ☐ لا أسرار في المستودع (git log -p | grep -i secret) وتفقد .gitignore).
- ☐ للمختبرين: ورقة من خمسة أسطر تشرح ما تريدهم أن يجربوه وكيف يبلغون عن مشكلة.

قائمة التحقق الثانية: قبل النشر العام

تفترض اجتياز القائمة الأولى. البنود المعلمة بـ **رسمي** متطلبات متجر تحققت منها بتاريخ 2026-09-27 وقد تتغير؛ راجع الروابط في الملحق الأول قبل النشر.

Google Play

- ☐ **رسمي** ملف .aab موقع بمفتاح الرفع، و Play App Signing مفعّل.
- ☐ **رسمي** targetSdk يفي بالحد الحالي (API 36 ابتداءً من 31 أغسطس 2026)، ودعم صفحات 16 KB.
- ☐ **رسمي** للحساب الشخصي الجديد: اختبار مغلق بـ 12 مختبراً على الأقل مشتركين 14 يوماً متواصلة، ثم طلب الوصول إلى الإنتاج.
- ☐ **رسمي** نموذج أمان البيانات مكتمل وصادق، ورابط سياسة خصوصية عام.

- ☐ رسمي إن كان فيه حسابات: حذف الحساب من داخل التطبيق ورابط ويب لطلب الحذف.
- ☐ رسمي تصنيف المحتوى، وبيانات الاتصال، وصفحة المتجر بلقطات شاشة بالأحجام المطلوبة.
- ☐ قرأت تقرير ما قبل الإطلاق وعالجت الانهيارات.
- ☐ رقم البناء أعلى من أي رقم رفع سابقًا، ووسم Git للإصدار، وسطر في `CHANGELOG.md`.
- ☐ أداة تتبع أعطال مفعلة ومذكورة في سياسة الخصوصية ونموذج البيانات.
- ☐ خطة طرح تدريجي (نسبة صغيرة أولاً) ومن يراقب التقارير في أول 48 ساعة.

App Store

- ☐ رسمي بني بـ Xcode 26 و SDK iOS 26 أو أحدث (خدمة البناء تستخدم صورة حديثة).
- ☐ رسمي تفاصيل الخصوصية في App Store Connect مكتملة، وملف Privacy Manifest للواجهات التي تتطلب سببًا.
- ☐ رسمي نصوص شرح لكل إذن في `Info.plist`.
- ☐ رسمي إن كان فيه حسابات: حذف الحساب داخل التطبيق؛ وإن قدمت تسجيل دخول بطرف ثالث فقدم «تسجيل الدخول بـ Apple».
- ☐ رسمي التطبيق ليس «موقعًا مغلفًا» ويقدم وظيفة حقيقية (البند 4.2).
- ☐ اختبر عبر TestFlight على iPhone حقيقي واحد على الأقل، وخاصة: الحواف والنتوء، لوحة المفاتيح العربية، الوضع الداكن، حجم الخط الكبير.
- ☐ لقطات شاشة بالأحجام المطلوبة، ووصف، وكلمات مفتاحية، ورابط دعم، ورابط سياسة خصوصية.
- ☐ ملاحظات للمراجع تشرح كيف يجرب التطبيق (وحساب تجريبي إن كان فيه تسجيل دخول).

كلمة أخيرة

ستقرأ بعد عام هذا الدليل وتبتسم من بعض ما فيه: ستكون بعض الأرقام قد تغيرت، وبعض الحزم قد استبدلت، وربما ظهرت أدوات نكاء اصطناعي تفعل ما كان يستغرق فصلًا كاملاً. لكن ما لن يتغير هو ما حاول الدليل تعليمه بين السطور: أن تبدأ صغيرًا، وأن تفهم ما تبنيه، وأن تختبر ما تظنه يعمل، وأن تتحقق من المصدر الرسمي قبل أن تعد أحدًا بشيء، وأن تحتفظ بالقرار لنفسك مهما كان مساعدك سريعًا. هذه عادات مطور، لا معلومات عن أدوات. ابدأ هذا الأسبوع، بشاشة ثابتة واحدة.

الملحق الأول: المصادر الرسمية وتاريخ التحقق

كل المعلومات المتغيرة في هذا الدليل جرى التحقق منها من الصفحات التالية بتاريخ **2026-09-27**. قبل أن تعتمد على رقم أو شرط، افتح الرابط. حين تعذر التحقق من معلومة، ذكر ذلك في الجدول الأخير.

Android و Google Play

الموضوع	الرابط
رسوم التسجيل (25 دولارًا مرة واحدة) والتحقق من الهوية والجهاز	https://support.google.com/googleplay/android-developer/answer/6112435
شرط الاختبار المغلق للحسابات الشخصية الجديدة (12 مختبرًا / 14 يومًا)	https://support.google.com/googleplay/android-developer/answer/14151465
التحقق من الهوية و D-U-N-S وما يعرض للجمهور	https://support.google.com/googleplay/android-developer/answer/13628312
الحد الأدنى لـ targetSdk (API 36) من 31 أغسطس (2026)	https://support.google.com/googleplay/android-developer/answer/11926878
دعم صفحات KB 16	https://developer.android.com/guide/practices/page-sizes
Android App Bundle	https://developer.android.com/guide/app-bundle
توقيع التطبيق ومفتاحا الرفع والتوقيع	https://developer.android.com/studio/publish/app-signing
Play App Signing	https://support.google.com/googleplay/android-developer/answer/9842756
نموذج أمان البيانات	https://support.google.com/googleplay/android-developer/answer/10787469
سياسة حذف الحساب	https://support.google.com/googleplay/android-developer/answer/13327111

الموضوع	الرابط
مسارات الاختبار (داخلي/مغلق/مفتوح/إنتاج)	https://support.google.com/googleplay/android-developer/answer/9845334
تقرير ما قبل الإطلاق	https://support.google.com/googleplay/android-developer/answer/9842757
مدة المراجعة	https://support.google.com/googleplay/android-developer/answer/9859751
versionName و versionCode	https://developer.android.com/studio/publish/versioning
Android Studio (Quail 4 2026.1.4) والمتطلبات	https://developer.android.com/studio https://developer.android.com/studio/install
المحاكي وتسريع العتاد	https://developer.android.com/studio/run/emulator https://developer.android.com/studio/run/emulator-acceleration
Android 17 (API 37)	https://developer.android.com/about/versions/17
إرشادات إمكانية الوصول (48dp، التباين)	https://developer.android.com/guide/topics/ui/accessibility/apps
Kotlin و Jetpack Compose أولاً	https://developer.android.com/compose https://developer.android.com/kotlin/first

iOS و Apple

الموضوع	الرابط
التسجيل في برنامج المطورين (99 دولارًا سنويًا) وطرقه	https://developer.apple.com/support/enrollment/
مقارنة العضويات (ما يتيح الحساب المجاني)	https://developer.apple.com/support/compare-memberships/
D-U-N-S والإعفاء من الرسوم	https://developer.apple.com/support/D-U-N-S/ https://developer.apple.com/support/membership-fee-waiver/

الموضوع	الرابط
إصدارات Xcode ومتطلبات macOS وأهداف النشر	https://developer.apple.com/support/xcode/
المتطلبات القادمة لرفع التطبيقات (iOS SDK) (26 منذ 28 أبريل 2026)	https://developer.apple.com/news/upcoming-requirements/
TestFlight (100 داخلي، 10,000 خارجي، 90 يومًا)	https://developer.apple.com/help/app-store-connect/test-a-beta-version/testflight-overview
App Store Connect والأدوار	https://developer.apple.com/app-store-connect/ https://developer.apple.com/help/app-store-connect/reference/role-permissions
إرشادات المراجعة (البنود 4.2 و4.8 و5.1.1)	https://developer.apple.com/app-store/review/guidelines/
مدة المراجعة (90% في أقل من 24 ساعة)	https://developer.apple.com/distribute/app-review/
تفاصيل الخصوصية وPrivacy Manifest	https://developer.apple.com/app-store/app-privacy-details/ https://developer.apple.com/documentation/bundleresources/privacy-manifest-files
حذف الحساب	https://developer.apple.com/support/offering-account-deletion-in-your-app/
مفاتيح App Store Connect API	https://developer.apple.com/help/app-store-connect/get-started/app-store-connect-api
رفع النسخ (Xcode، altool، Transporter)	https://developer.apple.com/help/app-store-connect/manage-builds/upload-builds
الشهادات وملفات التزويد ومعرف التطبيق	https://developer.apple.com/help/account/certificates/certificates-overview/ https://developer.apple.com/help/account/manage-identifiers/register-an-app-id/
Xcode Cloud (25 ساعة شهريًا؛ الإعداد الأول من Xcode)	https://developer.apple.com/xcode-cloud/ https://developer.apple.com/documentation/xcode/configuring-your-first-xcode-cloud-workflow

الموضوع	الرابط
العمولات وبرنامج الأعمال الصغيرة	https://developer.apple.com/programs/whats-included/ https://developer.apple.com/app-store/small-business-program/
إرشادات الواجهة البشرية وإمكانية الوصول (44pt)	https://developer.apple.com/design/human-interface-guidelines/ https://developer.apple.com/design/human-interface-guidelines/accessibility
إصدارات iOS (iOS 27 في 14 سبتمبر 2026)	https://developer.apple.com/news/releases/

Dart و Flutter

الموضوع	الرابط
التثبيت (Windows/Linux) والإعداد لـ Android	https://docs.flutter.dev/install https://docs.flutter.dev/platform-integration/android/setup
المنصات المدعومة (iOS 15، Android API 24-37) (27)	https://docs.flutter.dev/reference/supported-platforms
ملاحظات الإصدار (3.47)	https://docs.flutter.dev/release/release-notes https://flutter.dev/blog/whats-new-in-flutter-3-47
بناء ونشر Android	https://docs.flutter.dev/deployment/android
بناء ونشر iOS (يتطلب macOS وXcode)	https://docs.flutter.dev/deployment/ios
الويب (لم يعد يولد Service Worker)	https://docs.flutter.dev/deployment/web https://docs.flutter.dev/platform-integration/web/faq
إدارة الحالة البسيطة والبنية (MVVM)	https://docs.flutter.dev/data-and-backend/state-mgmt/simple https://docs.flutter.dev/app-architecture/guide https://docs.flutter.dev/app-architecture/recommendations

الموضوع	الرابط
مسار التعلم الرسمي ودروس ChangeNotifier وListenableBuilder	https://docs.flutter.dev/learn/pathway https://docs.flutter.dev/learn/pathway/tutorial/change-notifier
مسار Dart التعليمي	https://dart.dev/learn/tutorial
الترجمة RTL	https://docs.flutter.dev/ui/accessibility-and-internationalization/internationalization https://api.flutter.dev/flutter/painting/EdgeInsetsDirectional-class.html
الاختبارات وDevTools	https://docs.flutter.dev/testing/overview https://docs.flutter.dev/testing/integration-tests https://docs.flutter.dev/tools/devtools
Impeller (محرك الرسم الافتراضي)	https://docs.flutter.dev/perf/impeller
Android Studio وVS Code	https://docs.flutter.dev/tools/vs-code https://docs.flutter.dev/tools/android-studio
Material 3 افتراضي منذ 3.16	https://docs.flutter.dev/release/breaking-changes/material-3-default
الحزم المذكورة (تحقق من الصفحة قبل الاعتماد)	https://pub.dev/packages/shared_preferences https://pub.dev/packages/sqflite https://pub.dev/packages/drift https://pub.dev/packages/flutter_local_notifications https://pub.dev/packages/firebase_messaging https://pub.dev/packages/provider https://pub.dev/packages/go_router https://pub.dev/packages/image_picker https://pub.dev/packages/share_plus https://pub.dev/packages/sentry_flutter https://pub.dev/packages/firebase_crashlytics

خدمات البناء السحابية

الموضوع	الرابط
أسعار 500 Codemagic دقيقة M2 مجانية للحساب الشخصي	https://codemagic.io/pricing/ https://docs.codemagic.io/billing/pricing/
مواصفات أجهزة Xcode 26.6 macOS (افتراضياً)	https://docs.codemagic.io/specs-macos/xcode-26-6/
توقيع iOS التلقائي (دون Mac) واليدوي	https://docs.codemagic.io/flutter-code-signing/ios-code-signing/ https://docs.codemagic.io/yaml-code-signing/signing-ios/
بدء سريع لـ Flutter بملف codemagic.yaml	https://docs.codemagic.io/yaml-quick-start/building-a-flutter-app/
النشر إلى Google Play و TestFlight	https://docs.codemagic.io/yaml-publishing/app-store-connect/ https://docs.codemagic.io/yaml-publishing/google-play/
الاختبارات على محاكي iOS في السحابة	https://docs.codemagic.io/yaml-testing/testing/
مقال Codemagic: بناء iOS دون Mac (وحدوده)	https://blog.codemagic.io/how-to-build-and-distribute-ios-apps-without-mac-with-flutter-codemagic/
GitHub Actions: الدقائق المشمولة وأسعار macOS	https://docs.github.com/en/billing/concepts/product-billing/github-actions https://docs.github.com/en/billing/reference/actions-runner-pricing
MacStadium و Bitrise	https://bitrise.io/pricing https://macstadium.com/pricing
Expo EAS Build (15 بناء iOS مجاناً شهرياً)	https://expo.dev/pricing https://docs.expo.dev/build/introduction/

الويب وPWA والأمان

الموضوع	الرابط
Web Push لتطبيقات الشاشة الرئيسية على iOS 16.4	https://webkit.org/blog/13878/web-push-for-web-apps-on-ios-and-ipados/
حد 7 أيام للتخزين في Safari	https://webkit.org/tracking-prevention/
معايير التثبيت في Chrome	https://web.dev/articles/install-criteria
قدرات PWA وحدودها	https://web.dev/learn/pwa/capabilities
PWABuilder و Trusted Web Activity	https://developer.chrome.com/docs/android/trusted-web-activity https://docs.pwabuilder.com/
APNs و FCM	https://firebase.google.com/docs/cloud-messaging https://developer.apple.com/documentation/usernotifications/setting-up-a-remote-notification-server
WCAG التباين	https://www.w3.org/WAI/WCAG22/Understanding/contrast-minimum.html
Mobile Top 10 و OWASP MASVS	https://mas.owasp.org/MASVS/ https://owasp.org/www-project-mobile-top-10/
Expo، Kotlin و React Native	https://reactnative.dev/docs/environment-setup
Multiplatform، Swift	https://kotlinlang.org/docs/multiplatform/quickstart.html https://www.swift.org/blog/swift-6.4-released/

ما لم أستطع التحقق منه

ذكرت هذه النقاط بحذر في الدليل أو حذفت، لأن الصفحة الرسمية لم تكن متاحة أو لا تحتوي نصًا صريحًا:

- لا توجد جملة صريحة من Apple تقول إن التسجيل في برنامج المطورين «لا يتطلب Mac»؛ الاستنتاج مبني على أن التسجيل متاح عبر الويب وتطبيق Apple Developer على iPhone/iPad.
- لا توجد جملة صريحة من Apple تقول إن Simulator غير متاح لـ Windows/Linux؛ الاستنتاج مبني على أنه جزء من Xcode المتاح «لـ Mac فقط».

- لم يذكر على صفحة أسعار Codemagic صراحةً أن الحساب المجاني لا يتطلب بطاقة ائتمانية؛ الوثائق تشير إلى طلب البطاقة عند تفعيل الفوترة فقط.
- وثائق GitHub الحالية لم تعد تذكر «مضاعف 10x» لدقائق macOS؛ اكتفينا بالسعر المعلن للدقيقة.
- متطلبات Flutter القديمة (مساحة القرص، إصدار PowerShell) لم تعد على الوثائق الحالية فلم تذكر.
- حزمتا **hive** و **isar** لا تحملان علامة «متوقفة» رسمية على pub.dev، لكن إصداريهما المستقرين لم يحدثا منذ 2022 و2023 على التوالي.

الملحق الثاني: قاموس مصطلحات مختصر

(Application Programming Interface) API

واجهة برمجة: مجموعة دوال أو نقاط اتصال يقدمها نظام أو خادم ليستخدمها تطبيقك.

AAB / APK / IPA

صيغ ملفات التطبيق: AAB حزمة الرفع إلى Google Play، APK حزمة التثبيت المباشر على IPA، Android، حزمة iOS.

Async / await / Future

أدوات Dart للعمليات التي تستغرق وقتًا دون تجميد الواجهة.

Bundle ID / Application ID

المعرف الفريد للتطبيق على نمط نطاق معكوس، مثل `sa.edu.kau.students.dhikr`.

ChangeNotifier

صنف في Flutter يحمل حالة ويعلن تغييرها لمن يستمع إليه.

CI/CD

التكامل والتسليم المستمر: خدمات تبني وتختبر وتنشر تطبيقك آليًا عند كل تغيير (مثل Codemagic).

Debug / Profile / Release

أوضاع البناء: للتطوير، ولقياس الأداء، وللنشر.

Deep link

رابط يفتح شاشة محددة داخل التطبيق.

Dependency Injection

تمرير ما يحتاجه الصنف من الخارج بدل إنشائه داخله، لتسهيل الاستبدال والاختبار.

Hot reload

حقن التعديل في التطبيق الجاري تشغيله دون إعادة تشغيل.

Immutable

غير قابل للتغيير بعد الإنشاء؛ ينشأ بدل التعديل نسخة جديدة.

Keystore / Certificate / Provisioning Profile

ملفات التوقيع: الأول على Android، والآخران على iOS.

Migration

ترحيل: تحديث صيغة البيانات المحفوظة عند تغييرها بين إصدارات دون فقدانها.

minSdk / targetSdk

أقدم إصدار Android يدعمه التطبيق، والإصدار الذي يعلن أنه اختبر عليه.

MVP

أصغر نسخة قابلة للاستخدام الحقيقي من التطبيق.

MVVM

نمط بنية يفصل الواجهة (View) عن حالتها ومنطقها (ViewModel) عن البيانات (Model/Repository).

Null safety

نظام أنواع يجبرك على إعلان أي قيمة قد تكون فارغة والتعامل معها.

Platform channel

الجسر بين كود Dart والكود الأصلي (Kotlin/Swift) عند الحاجة إلى ميزة لا يغطيها Flutter.

Push / Local notification

إشعار مرسل من خادم عبر FCM/APNs، مقابل إشعار يحدوله التطبيق على الجهاز.

PWA

تطبيق ويب تقدمي: موقع قابل للتثبيت على الشاشة الرئيسية بقدرات محدودة تشبه التطبيق.

RTL

اتجاه العرض من اليمين إلى اليسار، كما في العربية.

Sideload

تثبيت تطبيق من خارج المتجر الرسمي (متاح على Android).

State

الحالة: كل بيانات تتغير مع الزمن وتؤثر في ما يعرض.

TestFlight

قناة Apple الرسمية لتوزيع نسخ تجريبية على مختبرين.

Widget

وحدة بناء الواجهة في Flutter؛ كل شيء على الشاشة widget.

Wireframe

رسم تخطيطي بسيط للشاشة يوضح العناصر ومواضعها دون تصميم بصري.